

- ❑ Topic of the session: Advanced Java Programming
- ❑ UNIT 1: GRAPHIC USER INTERFACE
- ❑ **University of Kigali (UoK)**
- ❑ Trainer name: Dr. NTEZIRIZA NKERABAHIZI Josbert

My Introduction

- Higher-level education: Ph.D. degree in Computer Science and Technology.
- Postdoctoral Candidate in Computer Science and Technology.
- Master Degree: Master's degree in Computer Science and Technology.
- Research interests: Software-Defined Networking (SDN), Internet of Things (IoT), Industrial IoT (IIoT), Machine Learning (ML), Cloud/Fog/Edge Computing, and Software Development.

My Introduction

➤ Research Gate:

[Home](#) > [Chongqing University of Posts and Telecommunications](#) > Computer Science > Nteziriza Nkerabahizi Josbert



Nteziriza Nkerabahizi Josbert

Chongqing University of Posts and Telecommunications · Computer Science

About

Publications **10**

Network

Connect with experts

Join ResearchGate to
connect with your science

Join for free

Learn more

About

10

Publications

1,711

Reads ⓘ

56

Citations

My Introduction

5

➤ Google Scholar



Dr. NTEZIRIZA NKERABAHIZI Josbert

FOLLOWING

Postdoctoral researcher in Chongqing University of Posts and Telecommunications
Verified email at stu.cqupt.edu.cn

Software-Defined Networki... Internet of Things (IoT) Industrial IoT (IIoT)

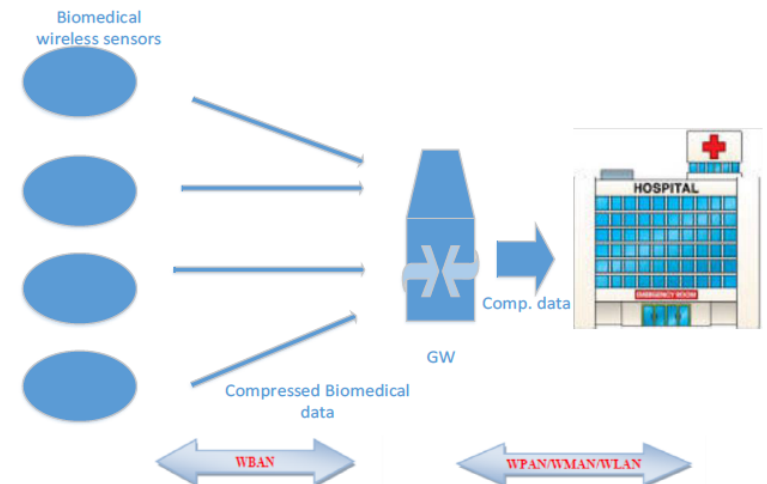
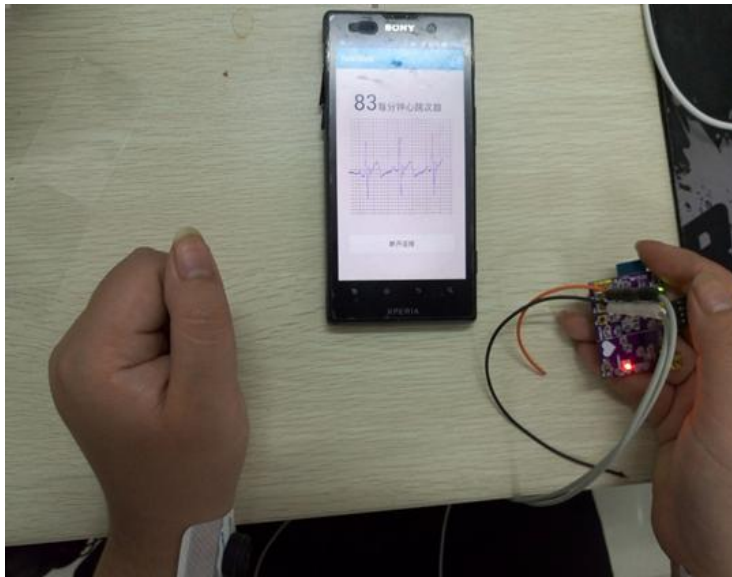
☐	TITLE	☐	CITED BY	YEAR
☐	Clustering optimization of LoRa networks for perturbed ultra-dense IoT networks MSA Muthanna, P Wang, M Wei, A Rafiq, NN Josbert Information 12 (2), 76		13	2021
☐	Optimizing energy consumption and latency based on computation offloading and cell association in MEC enabled Industrial IoT environment A Rafiq, W Ping, W Min, SH Hong, NN Josbert 2021 6th International Conference on Intelligent Computing and Signal ...		12	2021
☐	[Retracted] Mitigation Impact of Energy and Time Delay for Computation Offloading in an Industrial IoT Environment Using Levenshtein Distance Algorithm A Rafiq, P Wang, M Wei, MS Ali Muthanna, NN Josbert Security and Communication Networks 2022 (1), 6469380		10	2022
☐	Solution for industrial networks: Resilience-based sdn technology NN Josbert, W Ping, M Wei, A Rafiq 2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence ...		10	2021
☐	Industrial networks driven by SDN technology for dynamic fast resilience NN Josbert, W Ping, M Wei, Y Li		7	2021

Fall-2026-Dr. Josbert

UoK

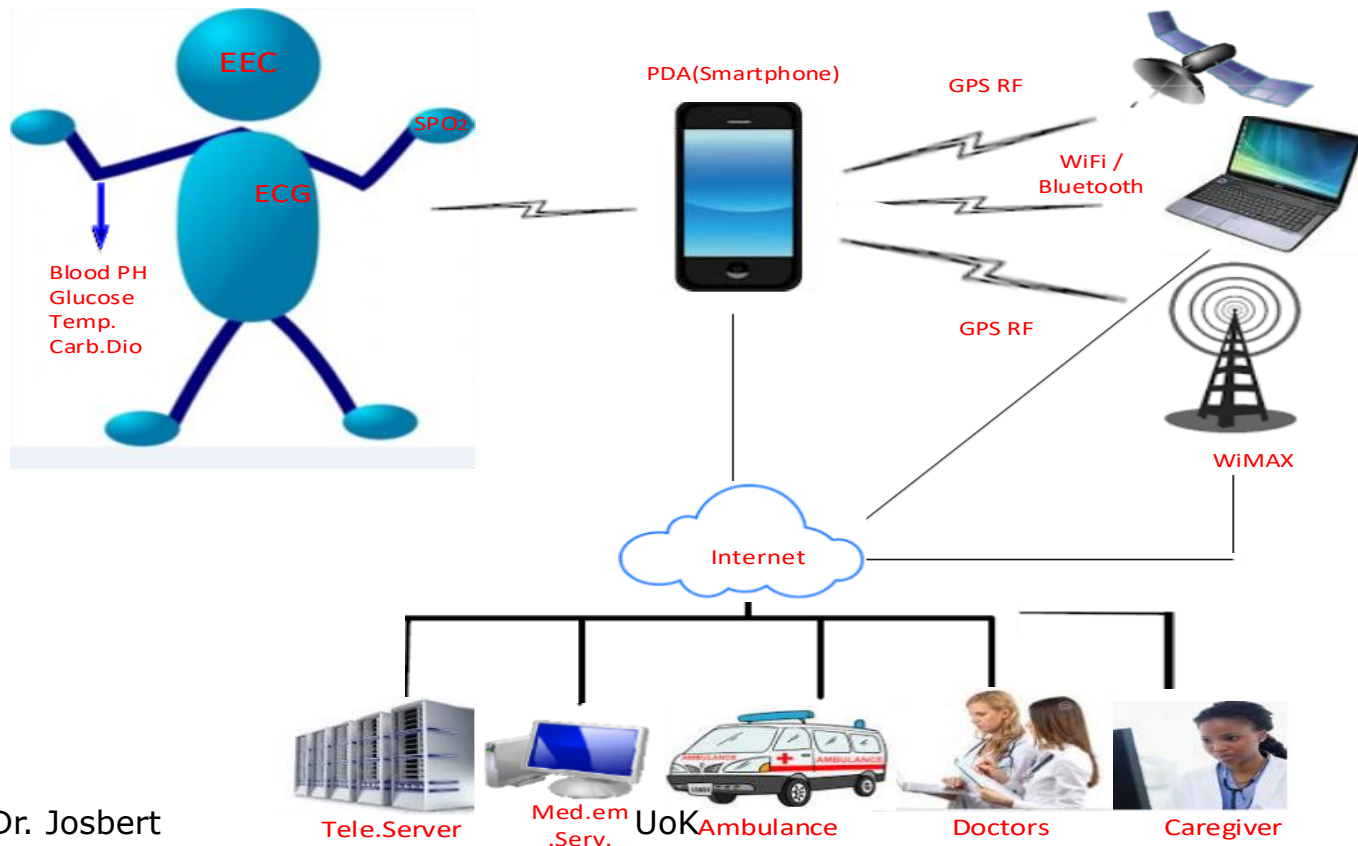
My Experience in JAVA

- **Wireless Body Area Network (WBAN) based on compressed sensing in MFOCUSS algorithm**
- A WBAN consists of several wireless sensor and actuator nodes placed on, near, or in a human body. The WBAN nodes sense and process vital signs (blood pressure, ECG, etc.)



My Experience in JAVA

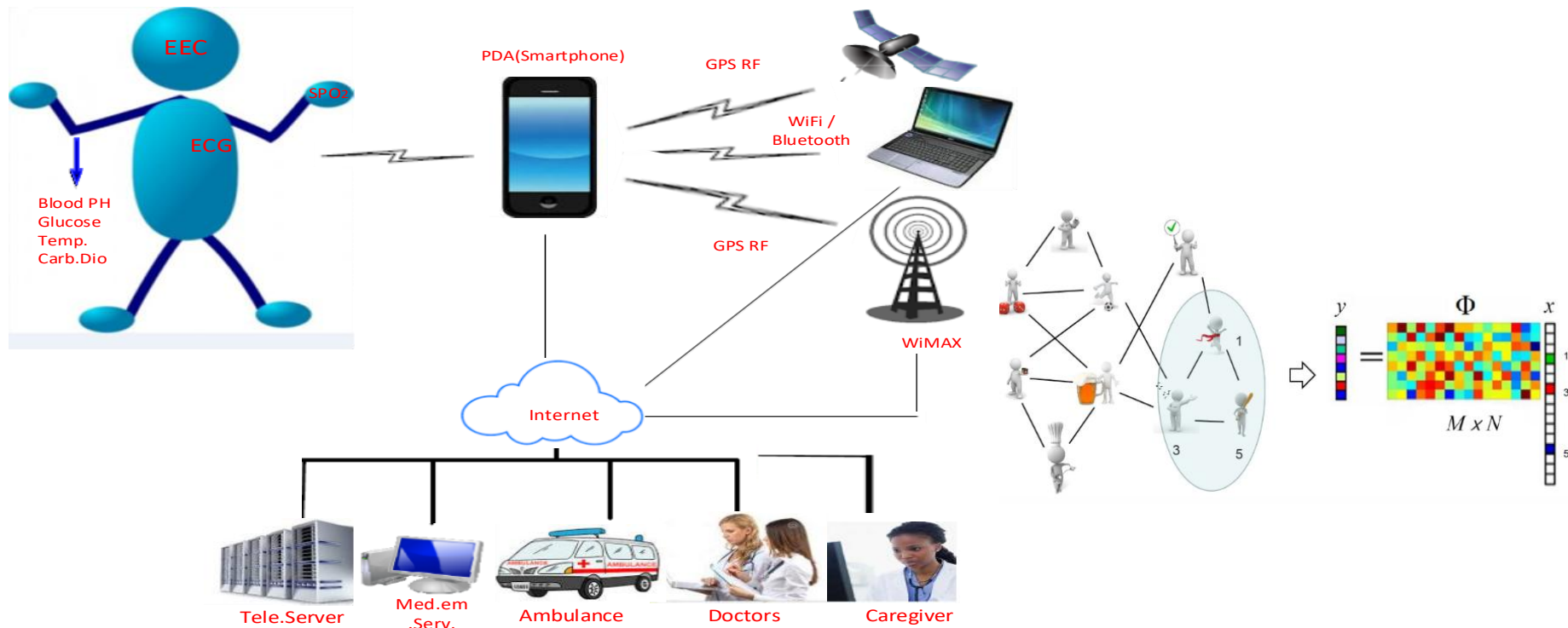
- **Wireless Body Area Network (WBAN) based on compressed sensing in MFOCUSS algorithm**



My Experience in JAVA

5

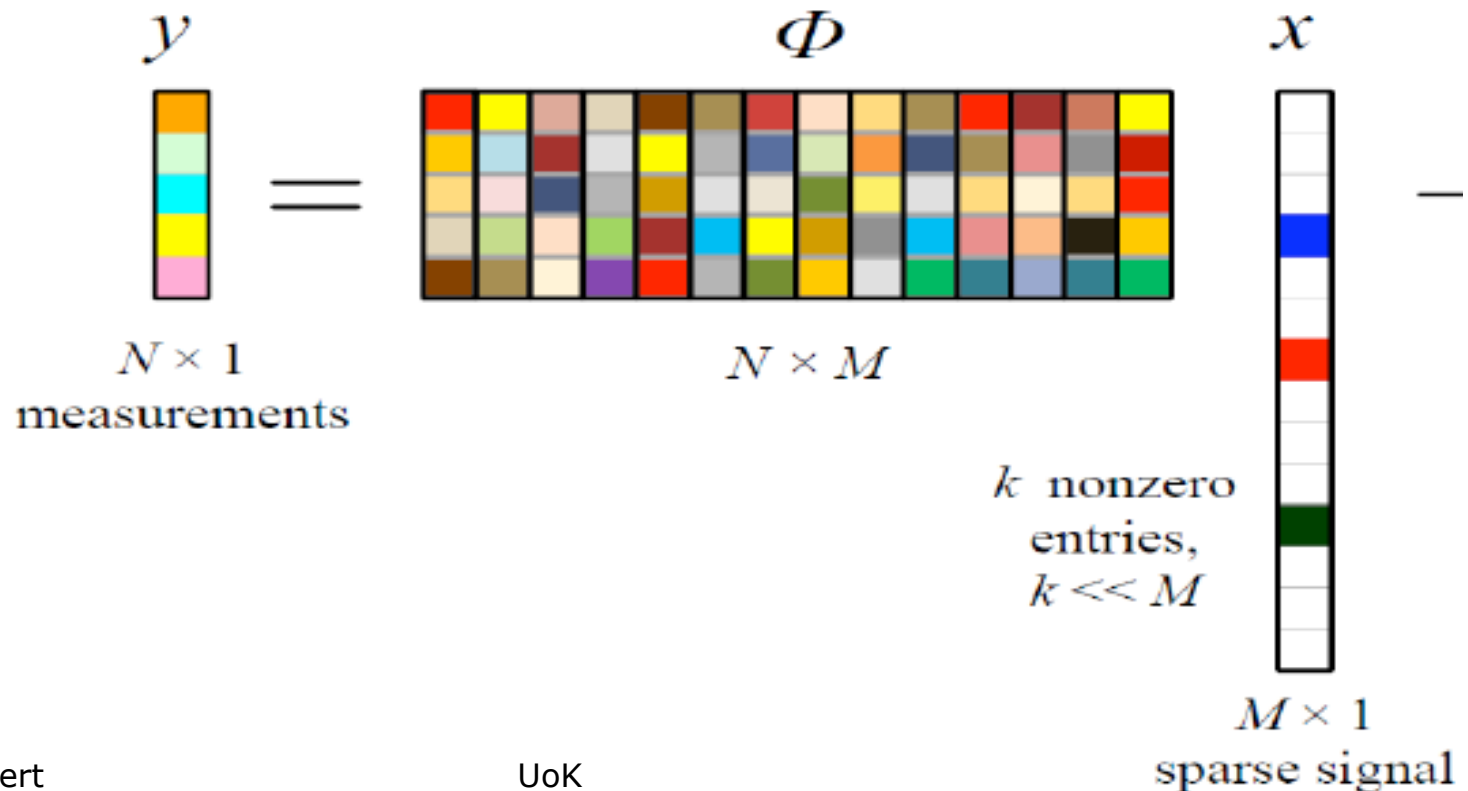
- The system can send the collected data to a medical center via a network coordinator so that doctors or caregivers can monitor the patient's health remotely.



My Experience in JAVA

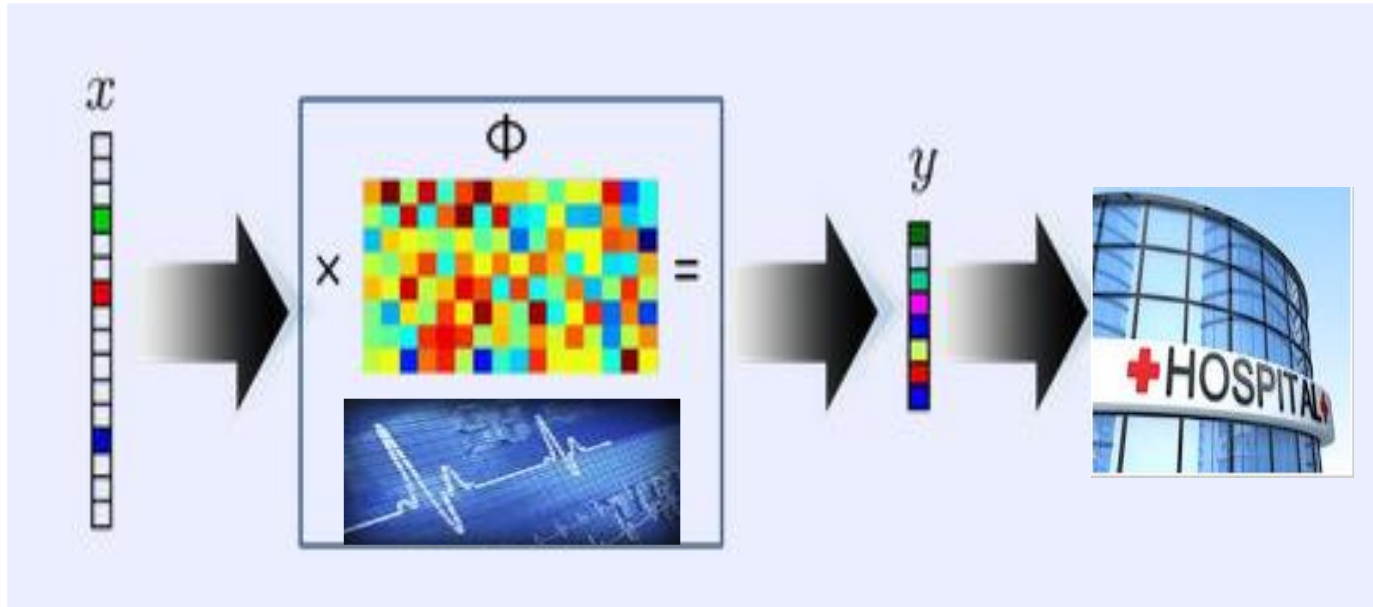
5

- The compressed data can decrease the number of bits and **delay/latency**; so, it **increases the lifetime** of wireless nodes and less power consumption is required to process data.



My Experience in JAVA

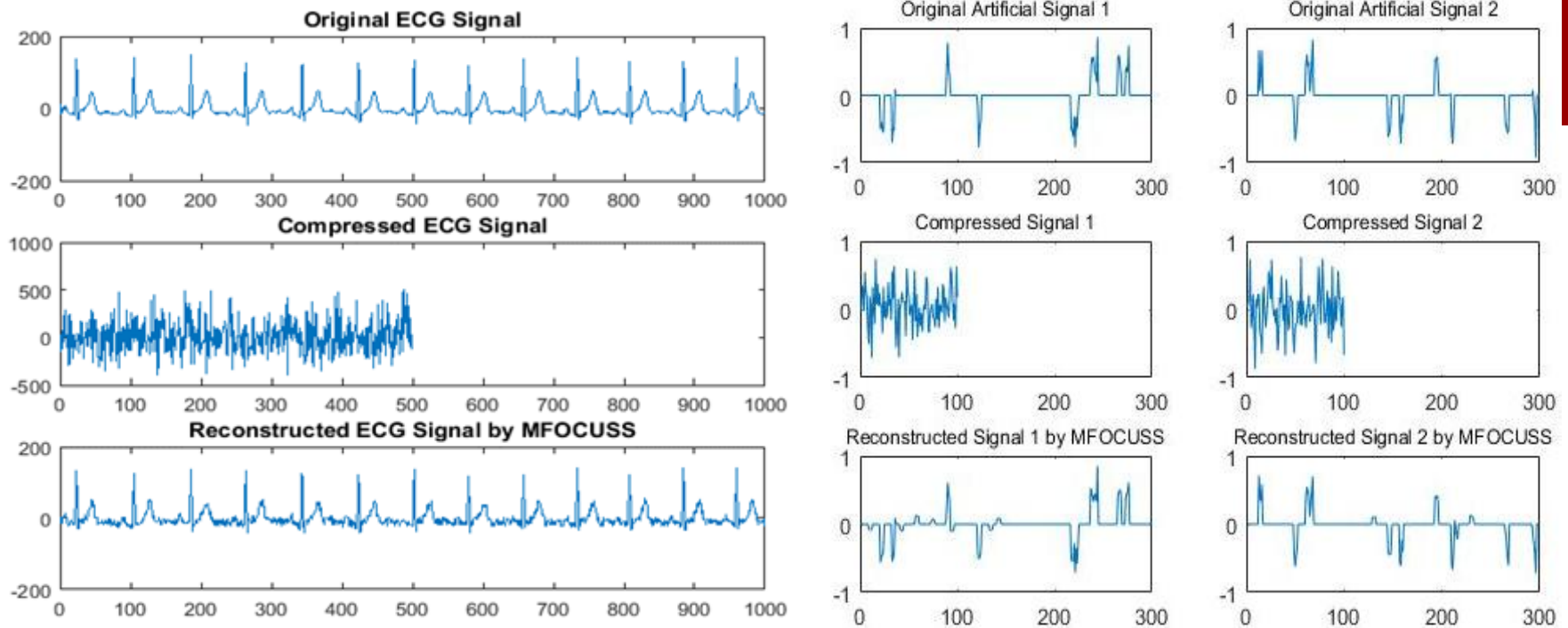
5



$$\begin{matrix} y \\ M \times 1 \end{matrix} = \begin{matrix} \Phi \\ M \times N \end{matrix} * \begin{matrix} x \\ N \times 1 \end{matrix}$$

My Experience in JAVA

5

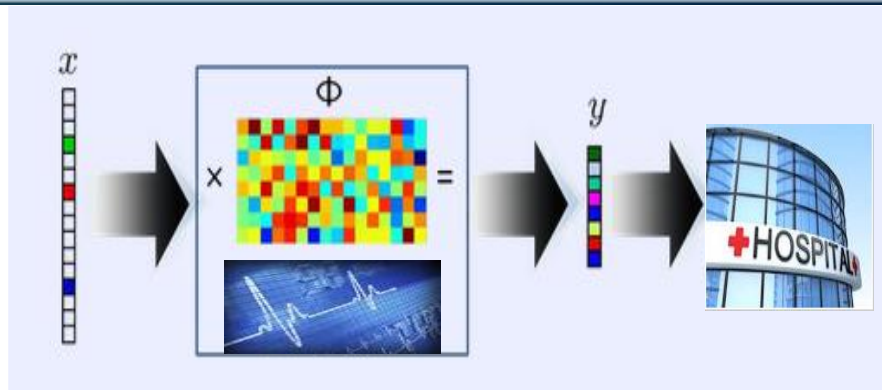
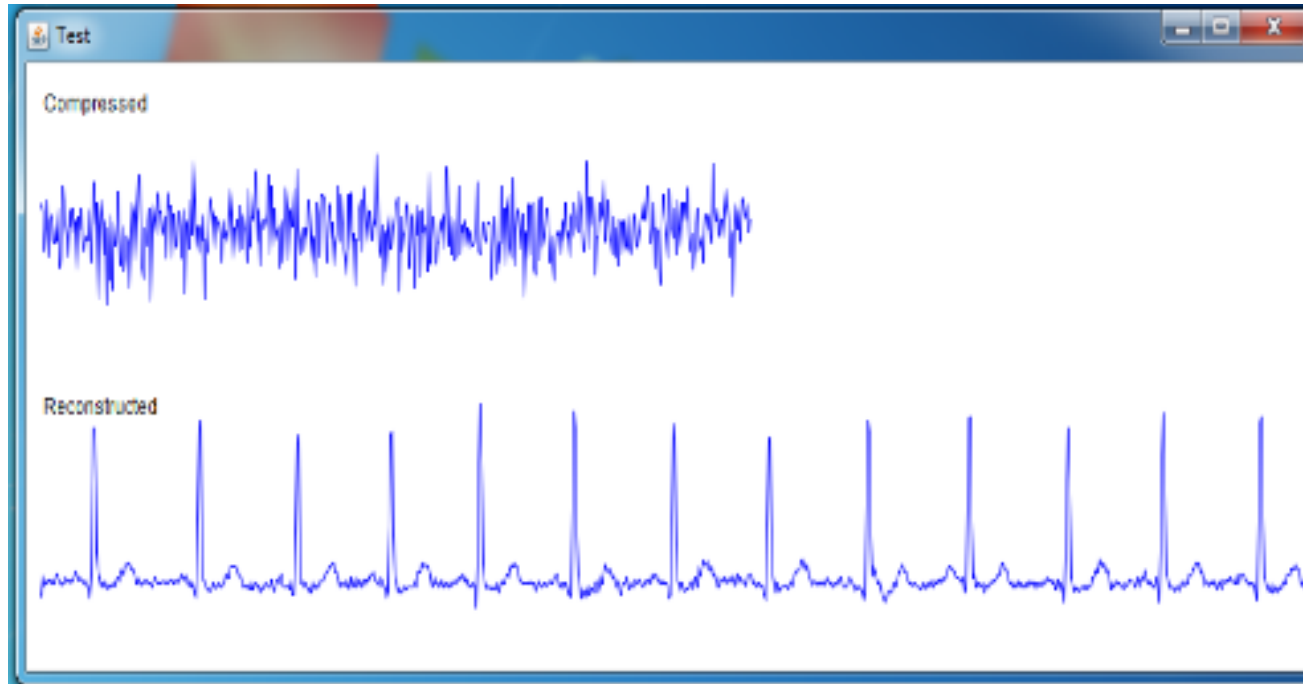


After compressing the Original ECG data (signal), the Reconstructed ECG data by MFOCUSS received on the other end is exactly the same as the Original ECG data as indicated by the above diagram.

My Experience in JAVA

5

- Reconstructed ECG data at the hospital server.



My Experience in JAVA

➤ Sample Java Code

```
Run Window Help
Mfocuss.java *fc.java operator.java do_while.java for_loop.java
1 package josbert_new_;
2
3 import java.util.ArrayList;
4 import java.util.Arrays;
5 import java.util.Collections;
6 import java.util.List;
7 import Jama.Matrix;
8 import Jama.SingularValueDecomposition;
9
10 public class fc {
11
12     /**
13      * @author JOSBERT
14      *
15      */
16
17     int N=6;
18     int M=6;
19     int L=1;
20     float lambda = 1e-10f;
21     double PRUNE_GAMMA = 1e-3f;
22     float MIN_GAMMA=1e-4f;
23     int MAX_ITERS = 800;
24     float EPSILON = 1e-8f;
25     double p = 0.8;
```

```
//=====
System.out.println(response.get(0, 0));
System.out.println(response.get(1, 0));
System.out.println(response.get(2, 0));
System.out.println(response.get(3, 0));
System.out.println(response.get(4, 0));
System.out.println(response.get(5, 0));
System.out.println("=====");
System.out.println("M-FOCUSS finished!");
System.out.println("@author JOSBERT!");
System.out.println("=====");
// Arrays.sort(keep_list);
```

```
}
```

```
}|
```


My Experience in JAVA

5

Patient smartphone with
MFOCUSS algorithm



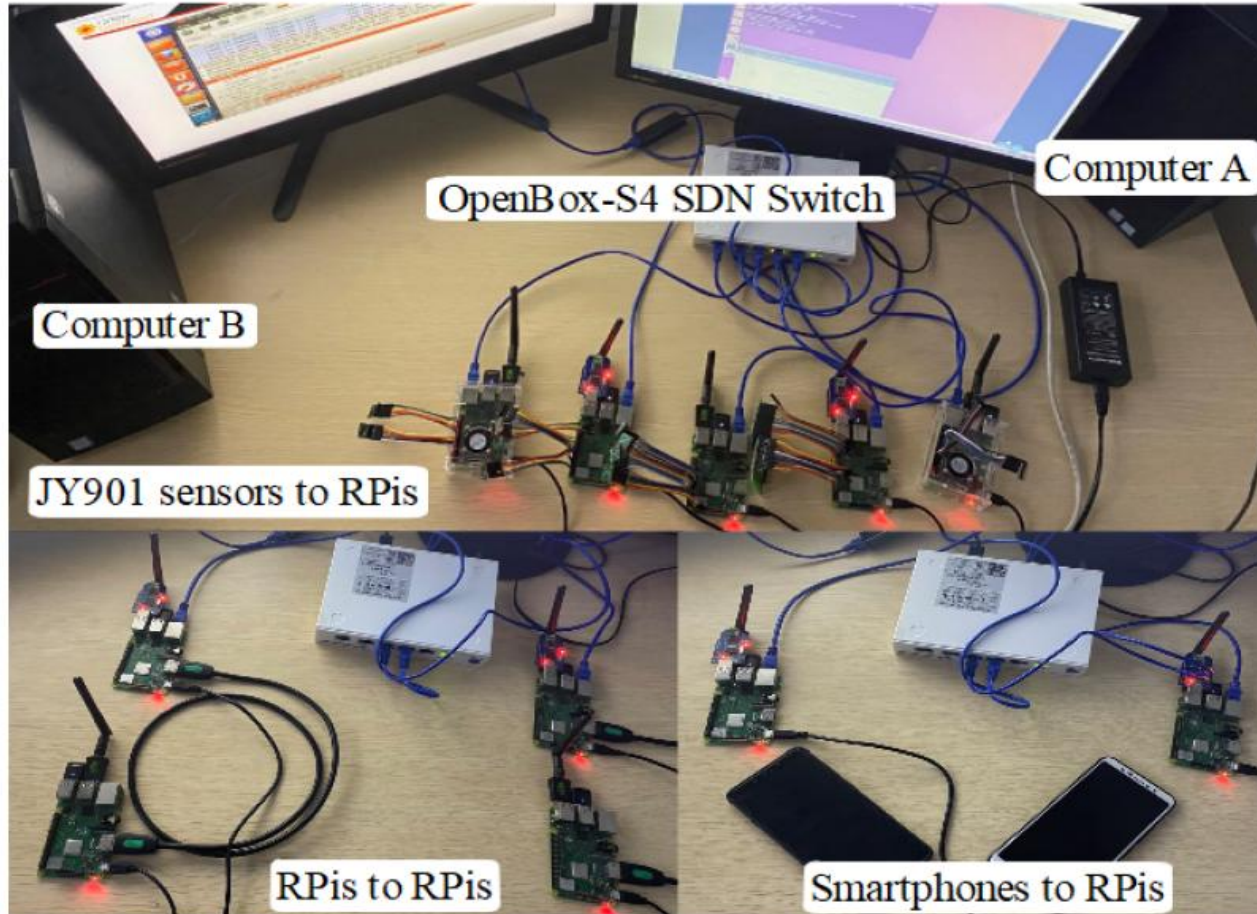
➤ Java Code Sample

```
System.out.println(response.get(23, 0));
System.out.println(response.get(24, 0));
System.out.println(response.get(25, 0));
System.out.println(response.get(26, 0));
System.out.println(response.get(27, 0));
System.out.println(response.get(28, 0));
System.out.println(response.get(29, 0));
.
.
.
System.out.println(response.get(997, 0));
System.out.println(response.get(998, 0));
System.out.println(response.get(999, 0));
System.out.println("=====");
System.out.println("M-FOCUSS finished!");
System.out.println("@author JOSBERT!");
System.out.println("=====");
```

My Experience in JAVA

5

- My second project based on Java programming.



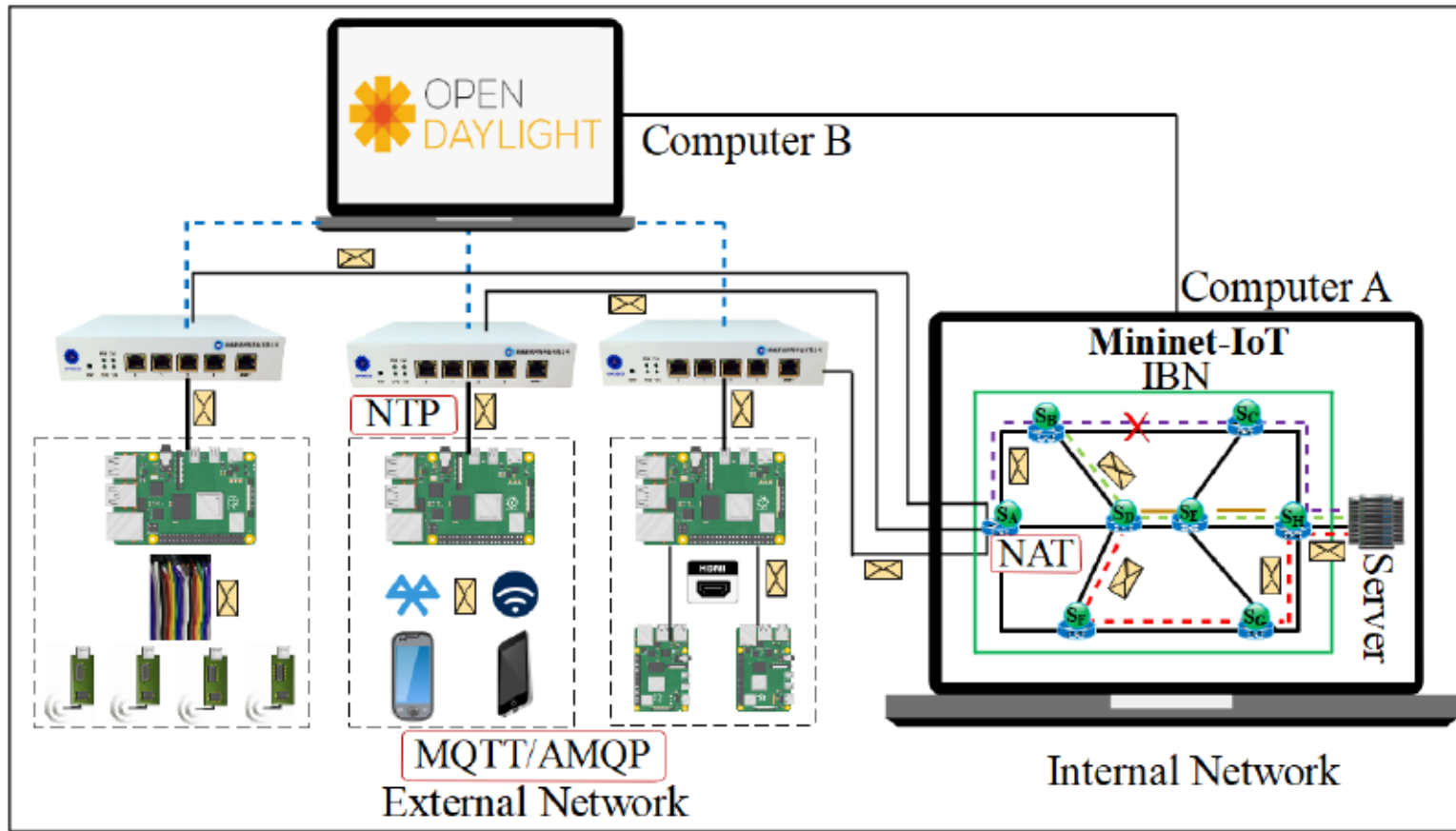
**SDN
Project**

SDN: Software-Defined Networking

My Experience in JAVA

5

- The second project based on Java programming.

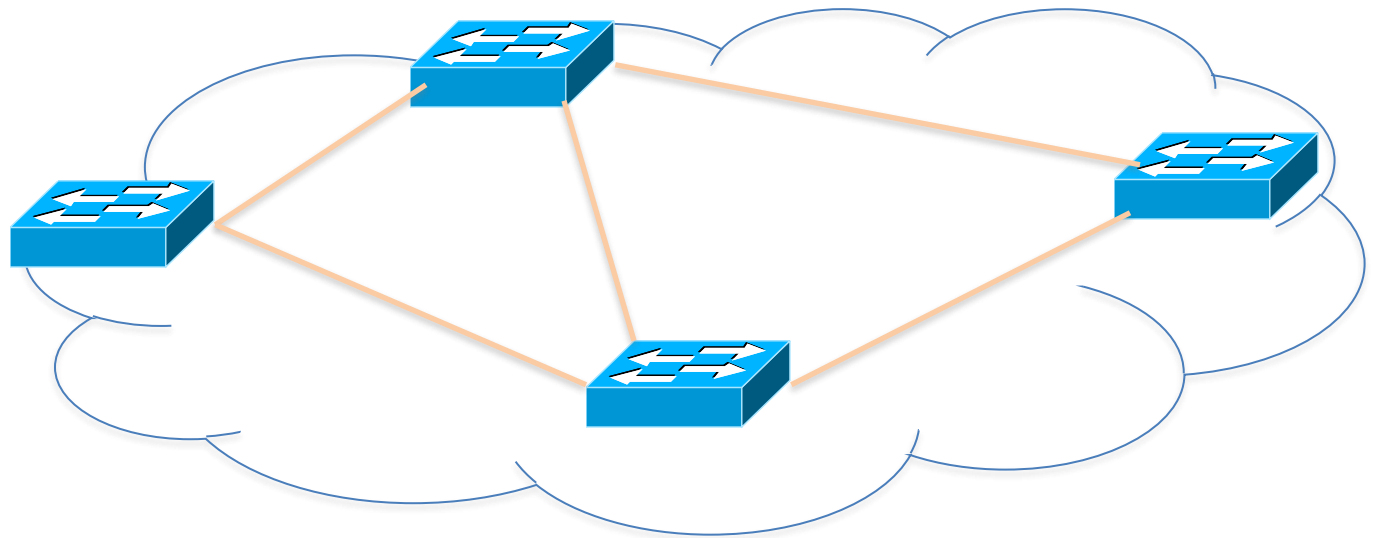


**SDN
Project**

What is the different between SDN and traditional network?

3

1.Introduction to Traditional Networking

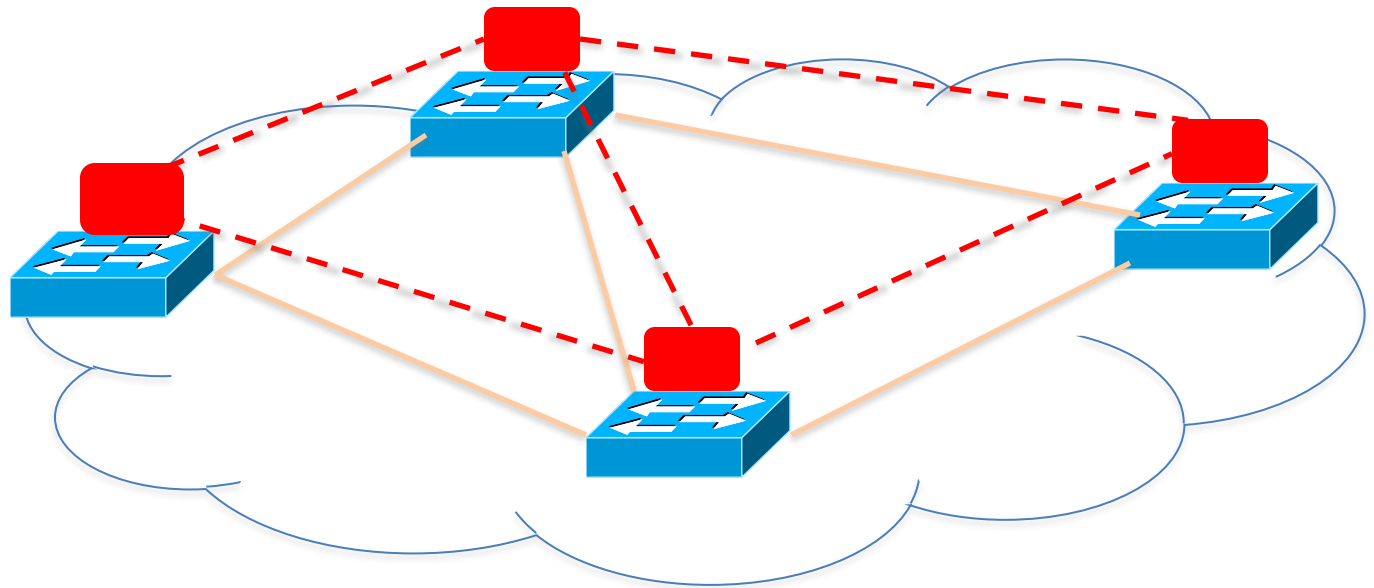


- Routing and performance were offered by vendor-specific hardware.
- Switches forwarding packets based on vendor-specific hardware.
- **Network configuration/reconfiguration and other updates across different devices required to be done manually.**

1. Traditional network model

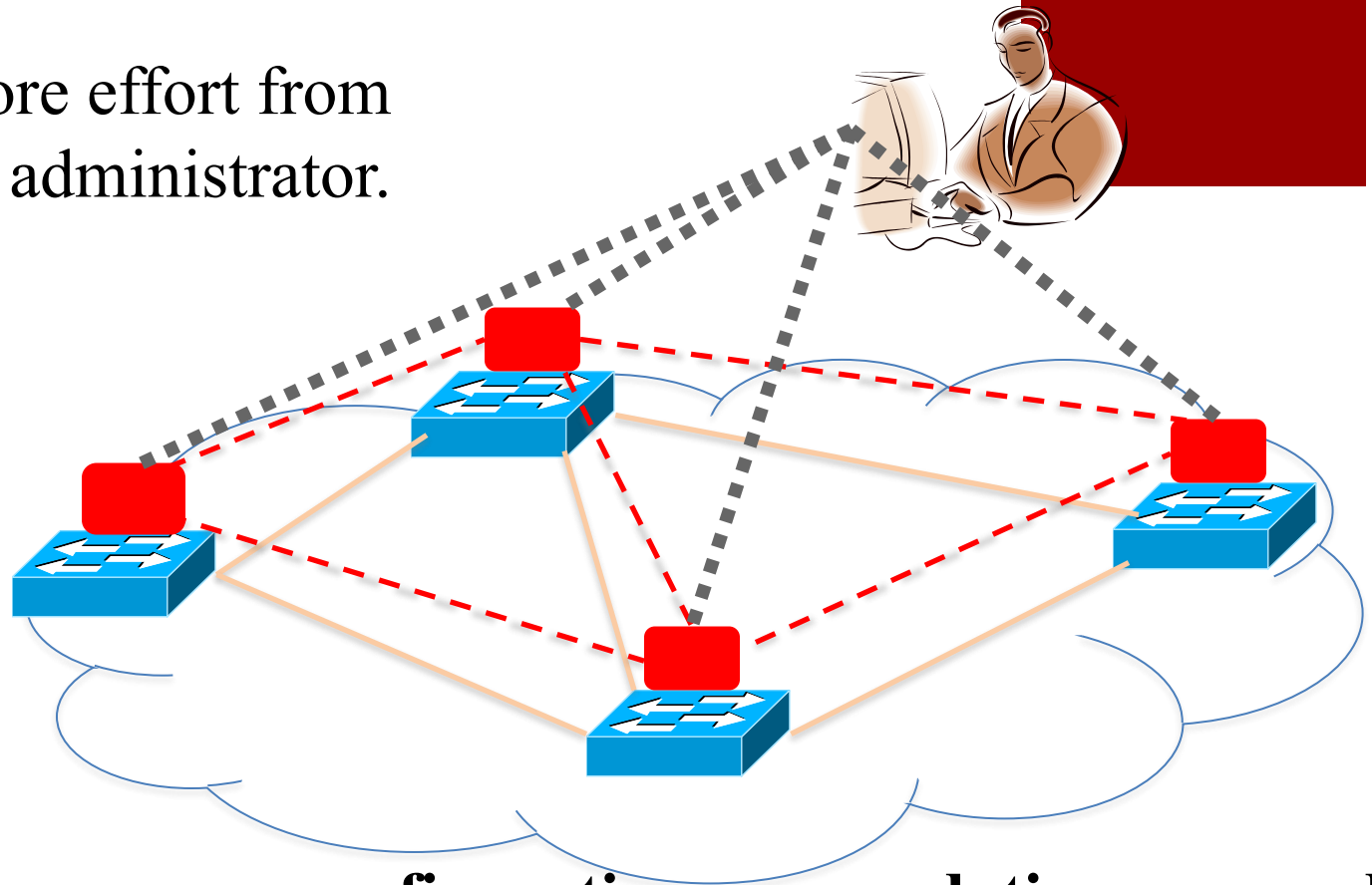
(continue...)

The switch is composed of a **hardware part** and an **intelligent part**.



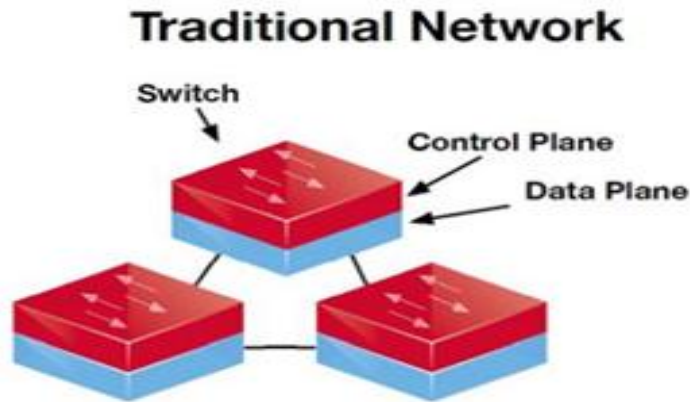
1. Traditional Network Model (continue....)

- Requires more effort from the network administrator.



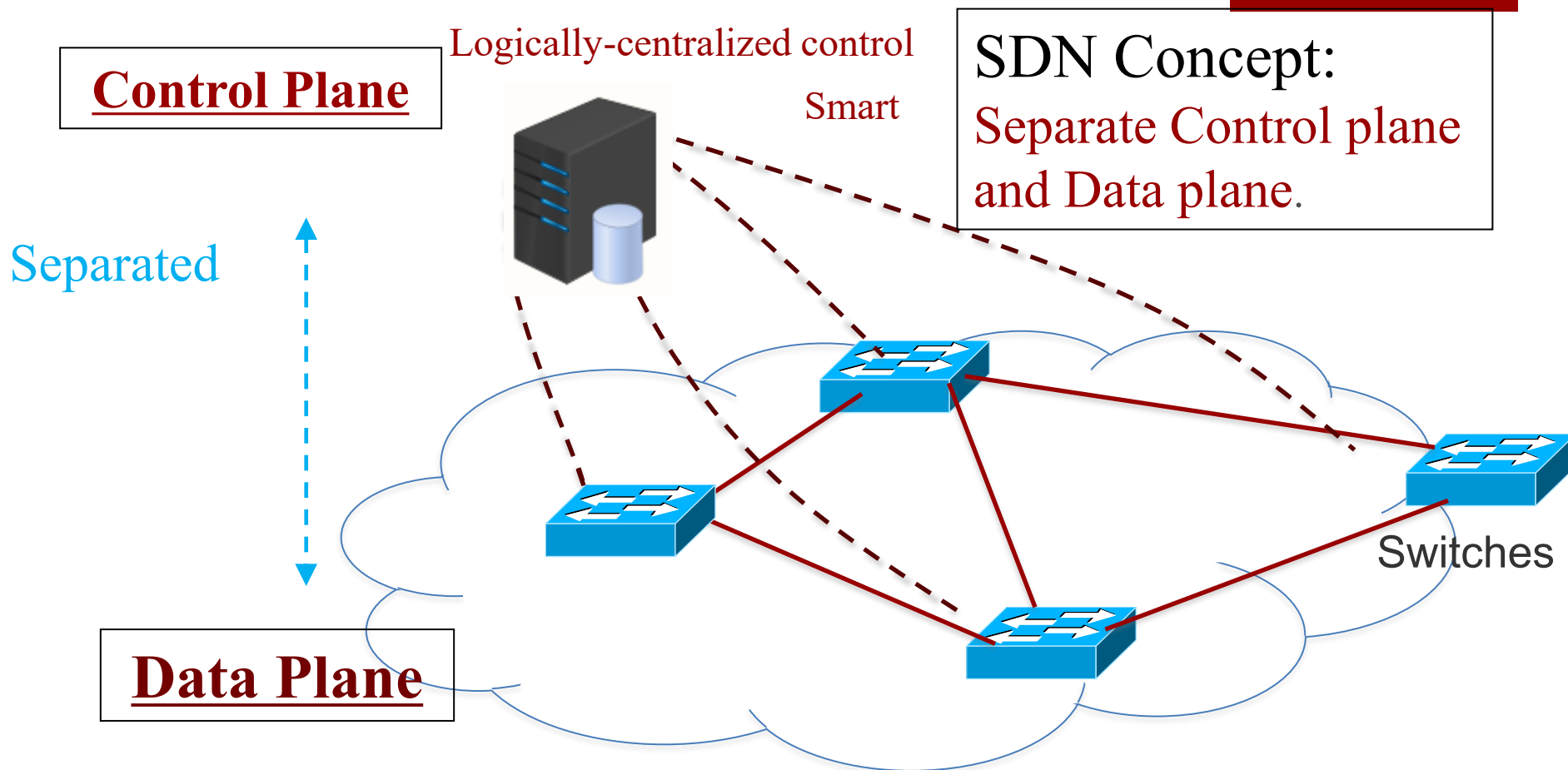
- **Configuration, reconfiguration, updating, and modification takes a long time because each box (switch) must be controlled and managed independently.**

1. Traditional Network Model (continue....)



- A traditional network is made up of **devices** with **integrated control plane** and **data-forwarding planes**.
- Each box needs to be configured and **managed independently**.
- This **slows down** the process and even a simple change to the network can sometimes take weeks or even worse, months to complete.
- It can take up this **much time** because these changes need to be made to each and **every device**.

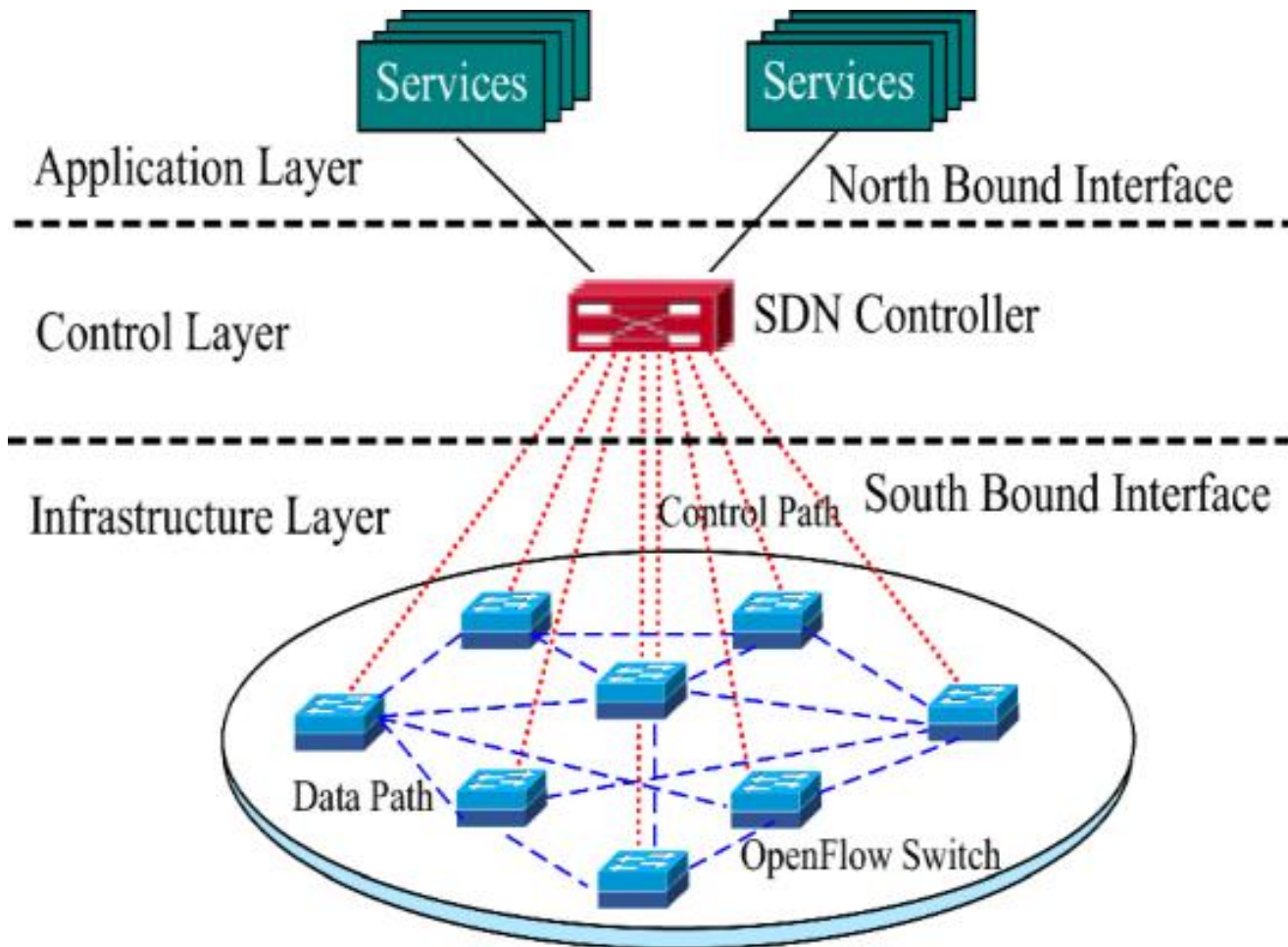
Imagine IF The Network is.....!!!



2. Software-Defined Networking (SDN)

21

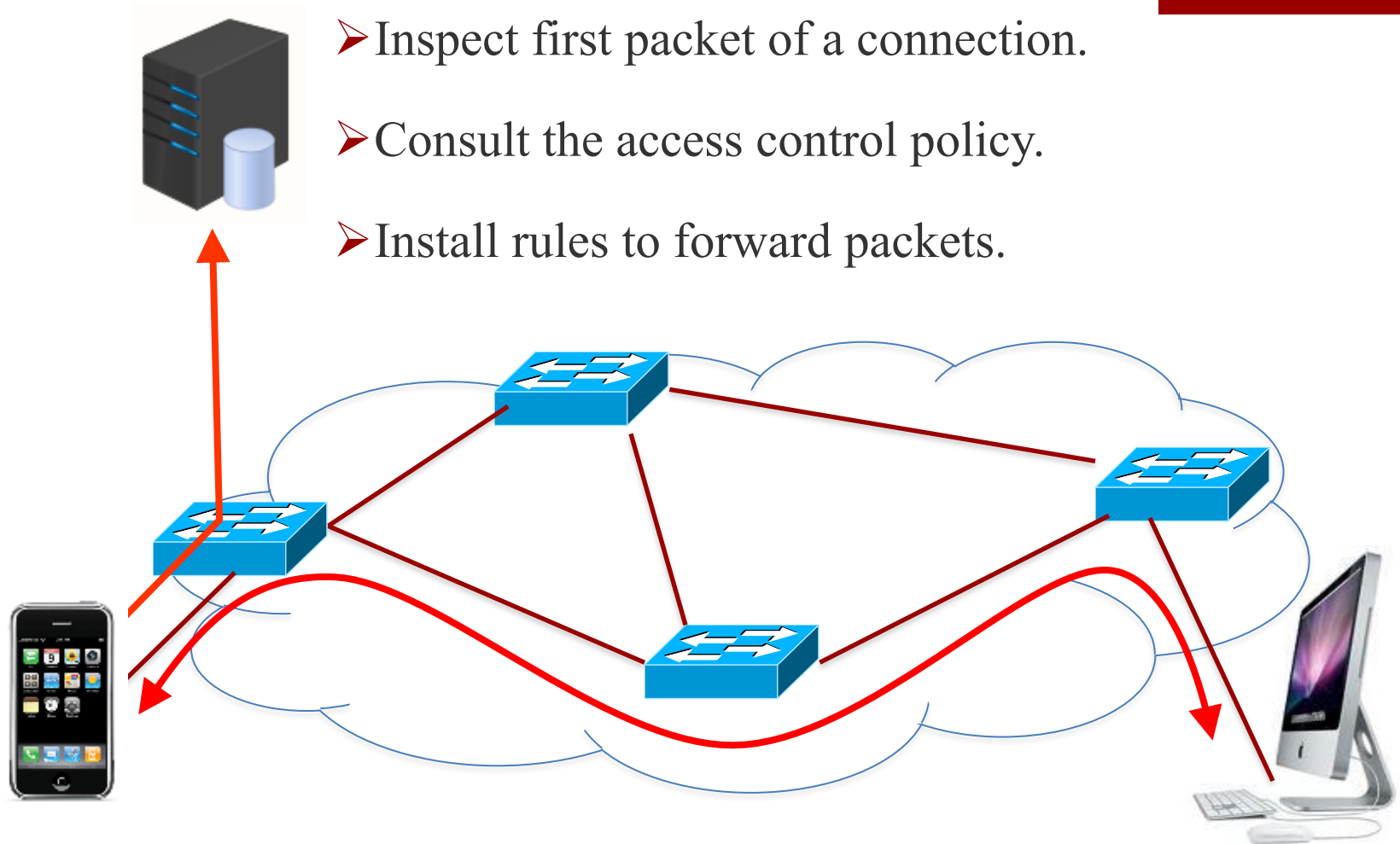
A. SDN Architecture



2. Software-Defined Networking (SDN) (continue....)

14

□ B. SDN Reactive Method

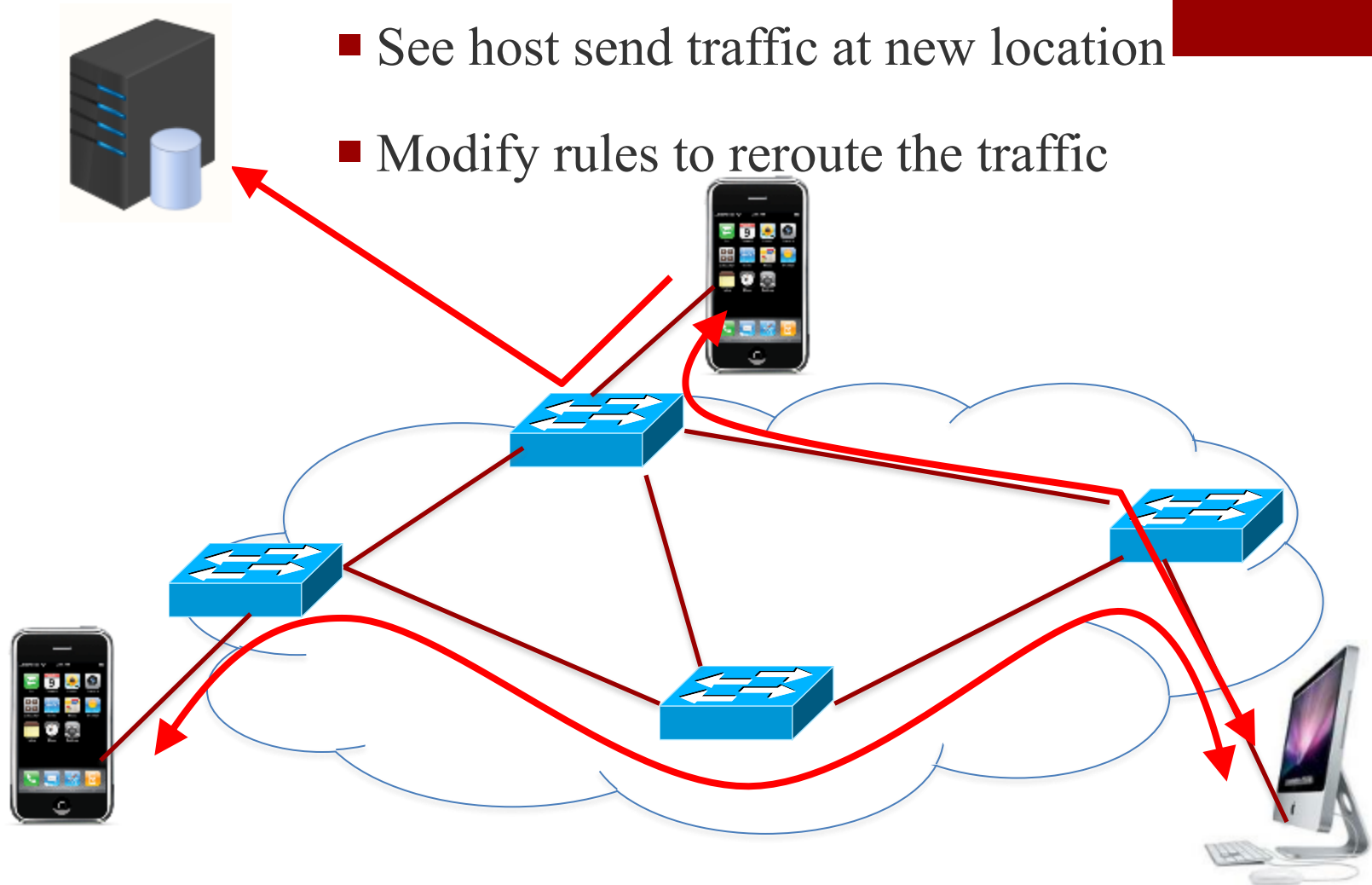


2. Software-Defined Networking (SDN)

(continue....)

15

B. SDN Reactive Method (continue...)



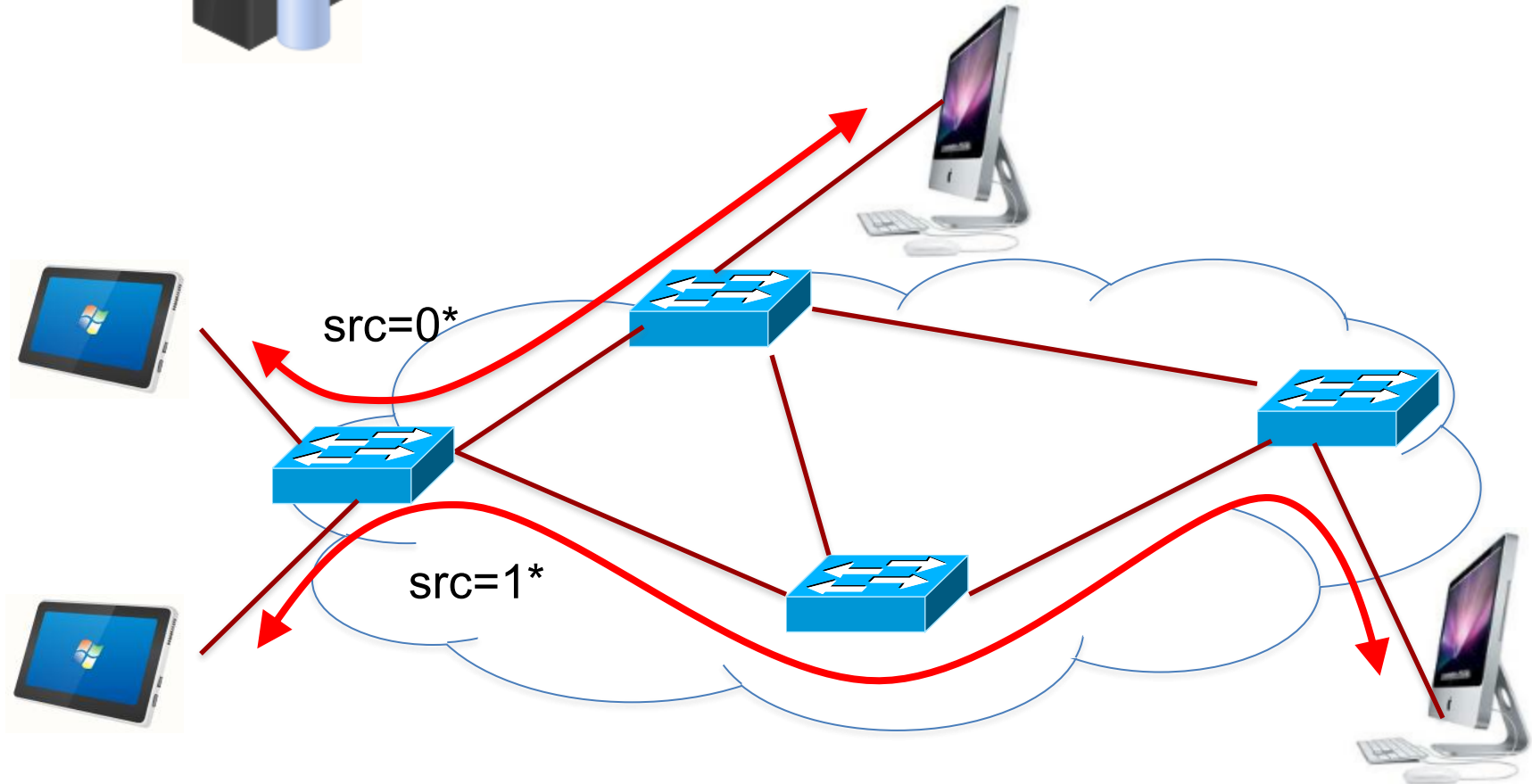
2. Software-Defined Networking (SDN)

(continue....)

C. SDN Proactive Method

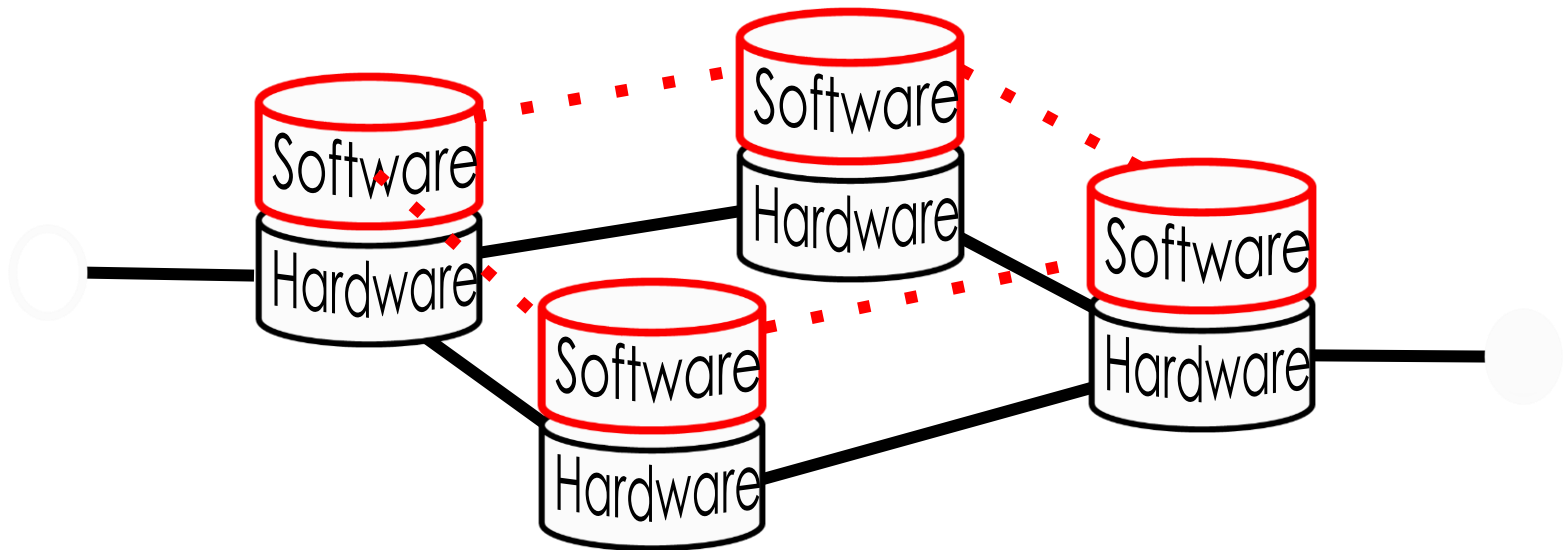


- Pre-install load-balancing policy.
- Compute shortest path before performance.



2. Software-Defined Networking (SDN) (continue....)

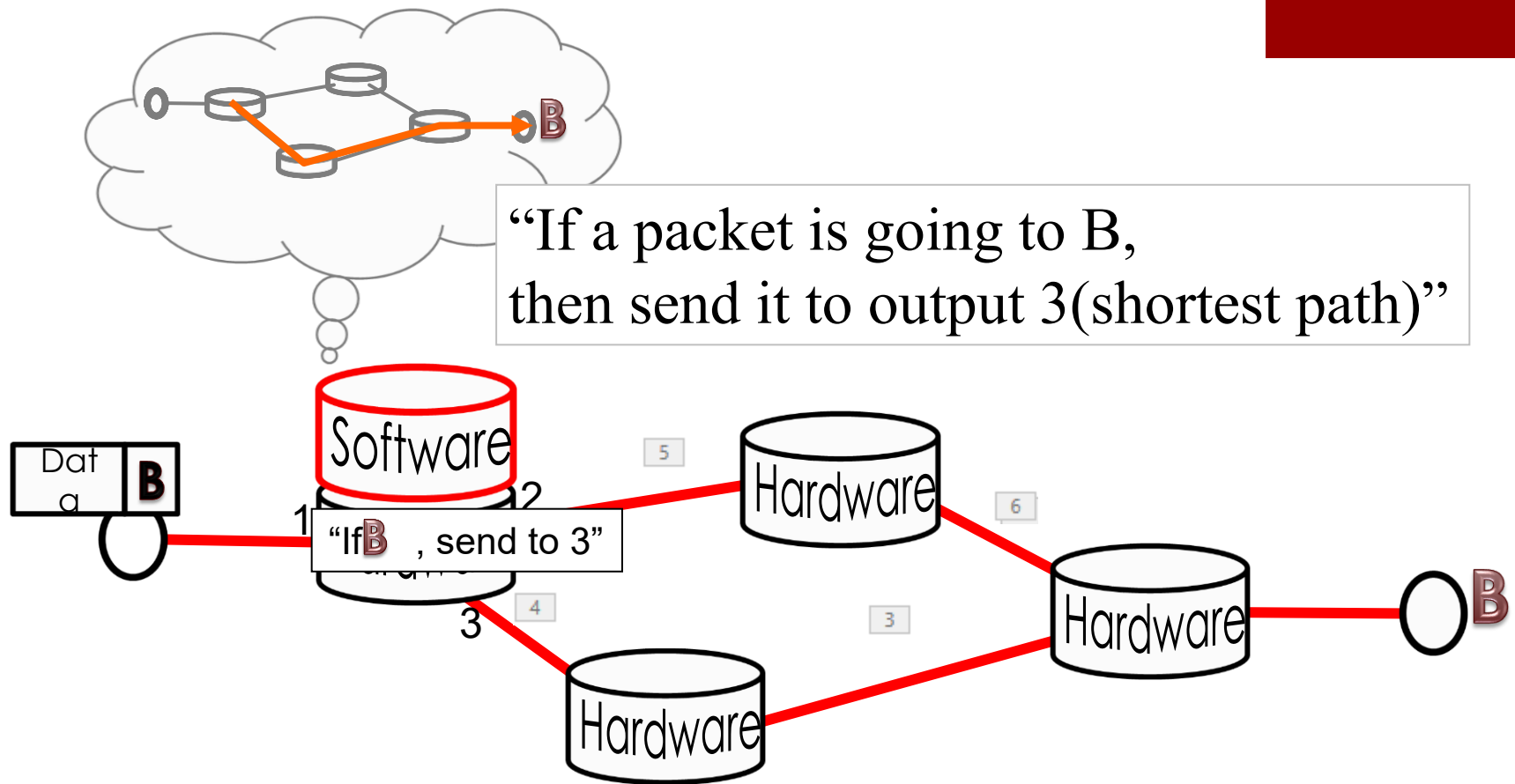
D. SDN Performance



2. Software-Defined Networking (SDN) (continue....)

26

□ D. SDN Performance (continue)

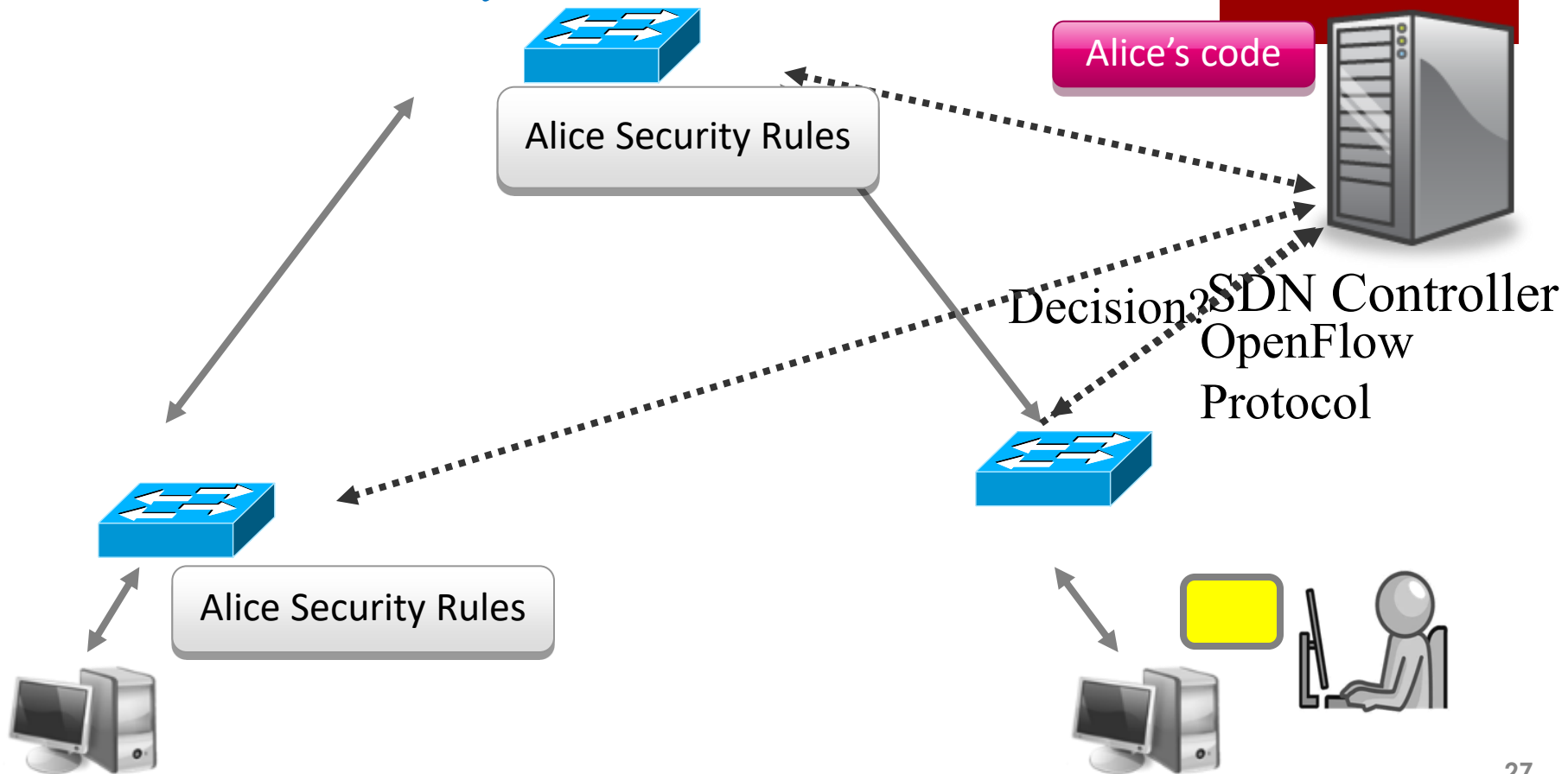


1. Figure out which routers and links are present.
2. Run Dijkstra’s algorithm to find shortest paths.

2. Software-Defined Networking (SDN)

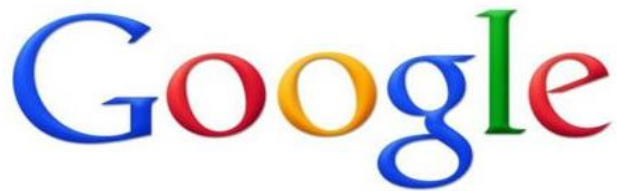
(continue....)

E. SDN Network Security



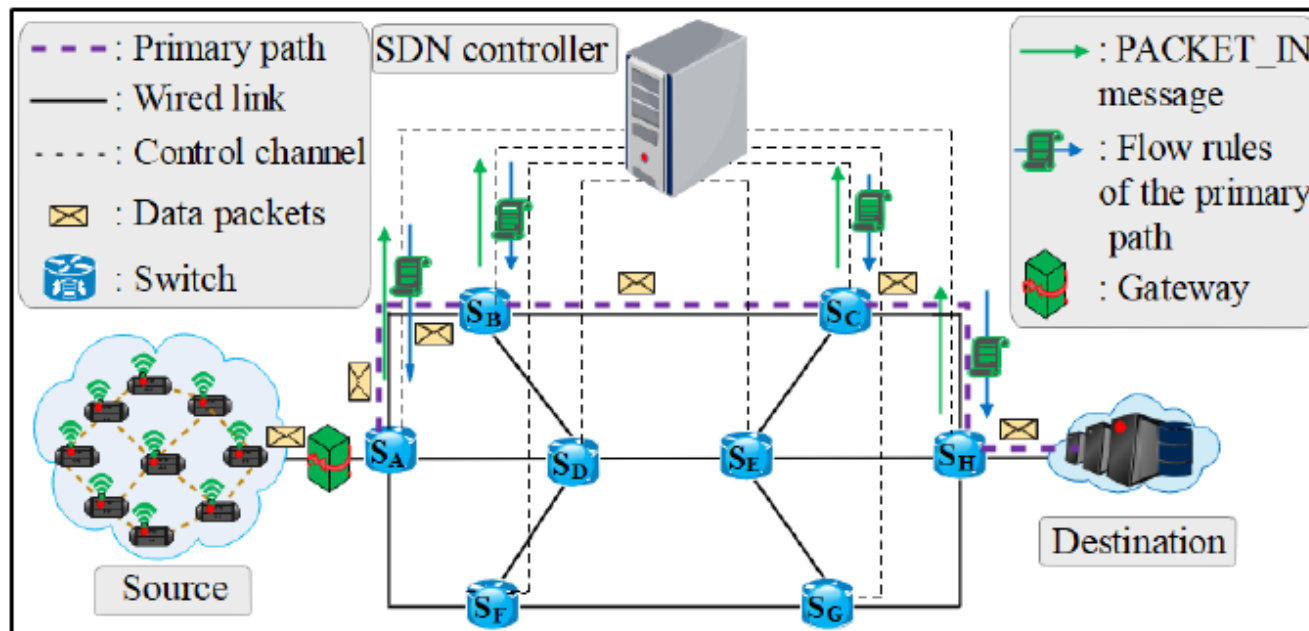
2. Software-Defined Networking (SDN) (continue...)

G. Some organizations and companies that adopt SDN technology



My SDN Innovation based on Java

- Local Restoration (LR) application (existing approach.)

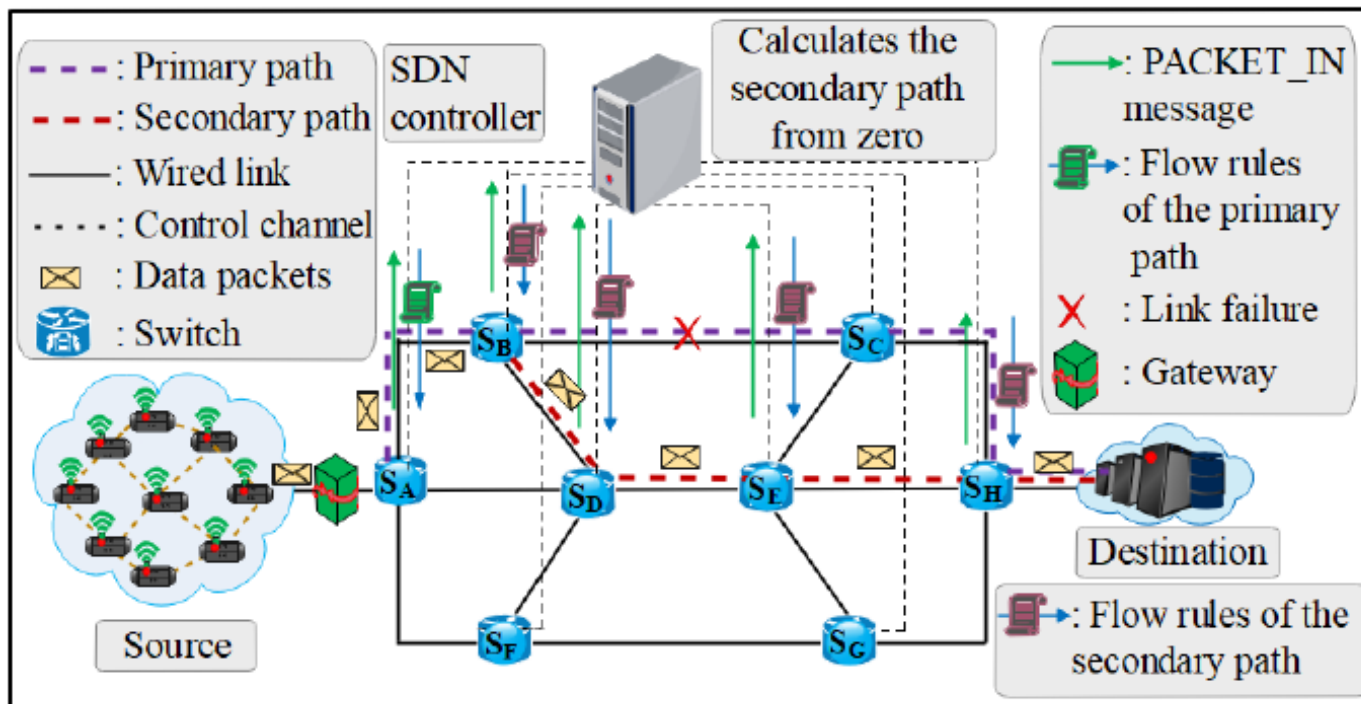


This approach increase the delay and round-trip time to the controller.

- Before an intermediate link S_B → S_C failure.

My SDN Innovation based on Java

- Local Restoration (LR) application (existing approach.)

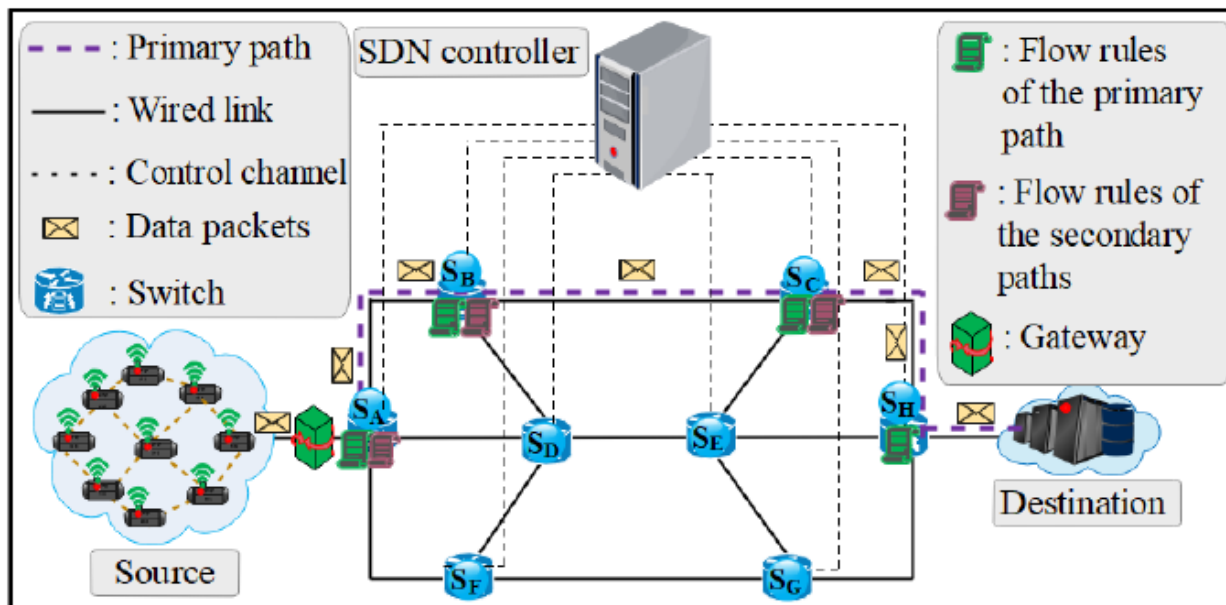


Increases the recovery time

- After an intermediate link $SB \rightarrow SC$ failure.

My SDN Innovation based on Java

➤ Path Protection (PP) application (existing approach.)

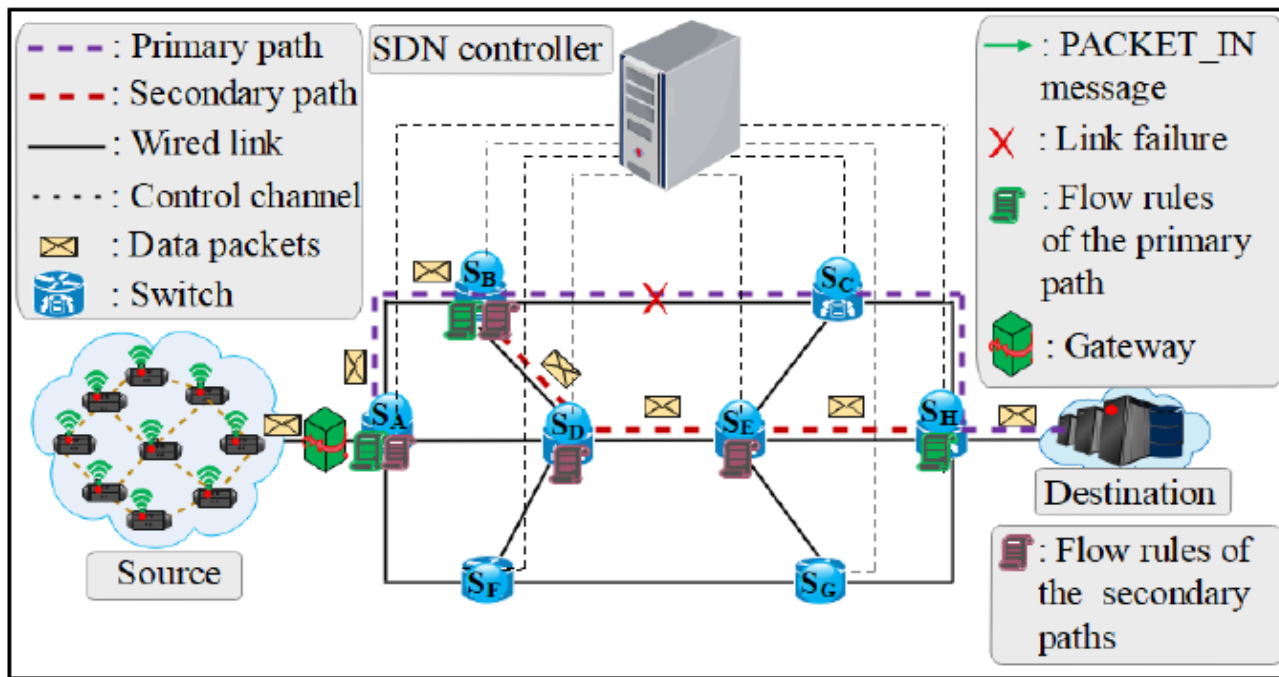


Utilizes more flow rules in the switch which in return increments the lookup time taken to discover an appropriate flow entry in the flow table.

➤ Before an intermediate link $S_B \rightarrow S_C$ failure.

My SDN Innovation based on Java

➤ Path Protection (PP) application (existing approach.)

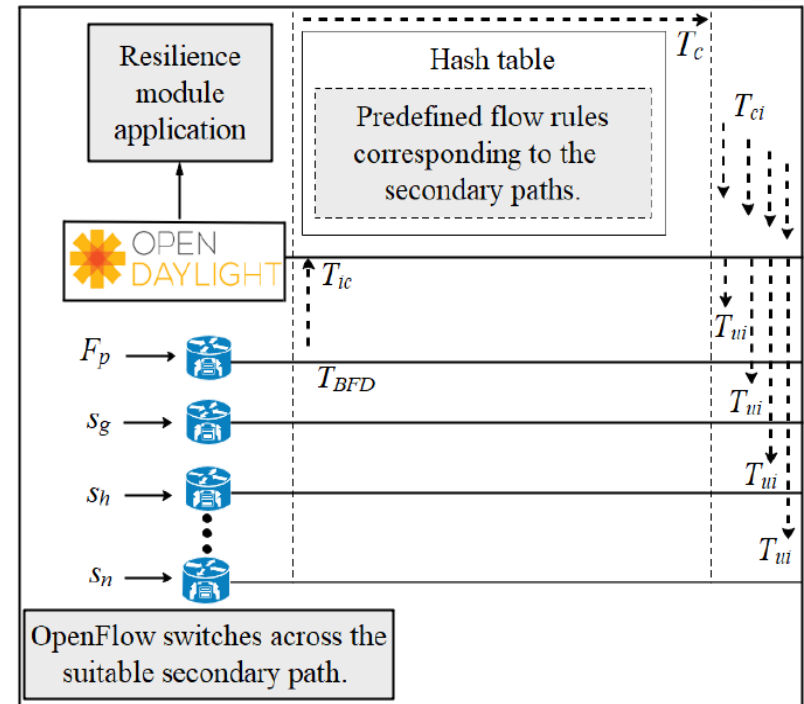
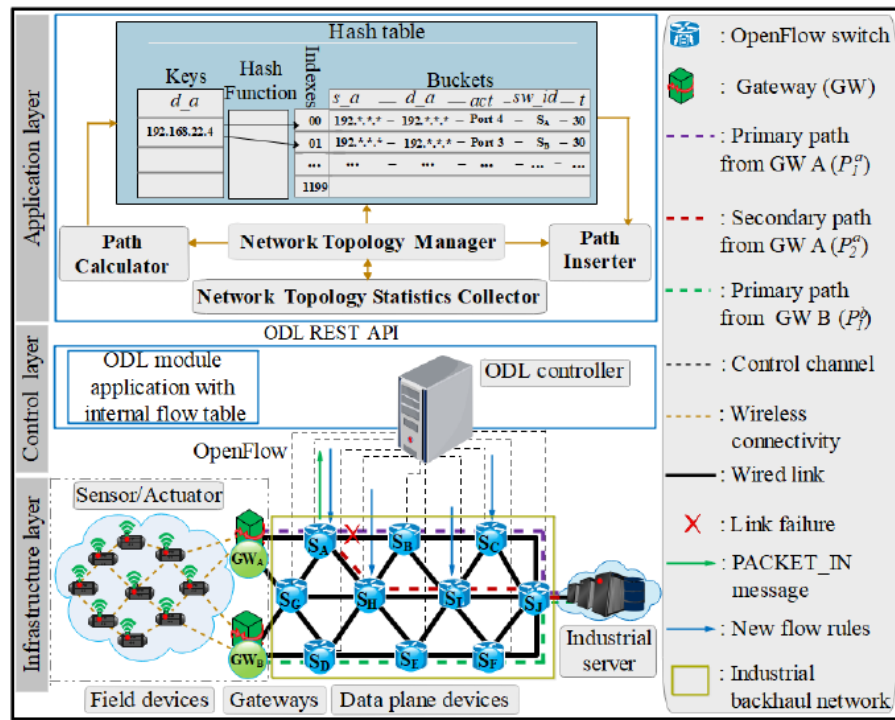


➤ After an intermediate link $S_B \rightarrow S_C$ failure.

My SDN Innovation based on Java

5

- Mixed Fast Resilience (MFR) application (my proposed approach.)



My SDN Innovation based on Java

- Mixed Fast Resilience (MFR) application (my proposed approach.)

In the MFR, the SDN controller establishes a new path after failure detection and this is based on flow rules stored in its memory through the **dynamic hash table structure as the internal flow table**.

At that time, it transmits the flow rules to all switches across the appropriate secondary path simultaneously from the failure point to the destination switch.

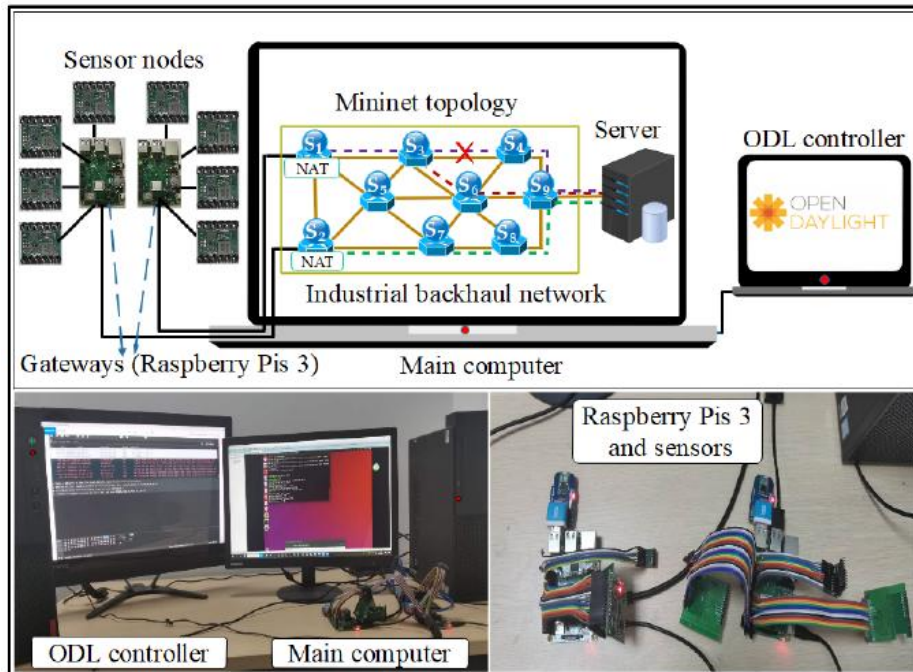
Moreover, these flow rules which correspond to secondary paths are **cached in the hash table** by considering the current **minimum path weight**.

My SDN Innovation based on Java

5

- Mixed Fast Resilience (MFR) application (my proposed approach)

Testbed and results from the simulation



Paper related to this approach is available online:
<https://www.mdpi.com/2078-2489/12/10/420>



Article

Industrial Networks Driven by SDN Technology for Dynamic Fast Resilience

Nteziriza Nkerabahizi Josbert ¹, Wang Ping ², Min Wei ^{2,*} and Yong Li ²

¹ Departement of Computer Science and Technology, Industrial IoT International S&T Cooperative Base, Chongqing University of Posts and Telecommunications, Chongqing 400065, China; josbertnn@gmail.com

² Departement of Automation, Industrial IoT International S&T Cooperative Base, Chongqing University of Posts and Telecommunications, Chongqing 400065, China; wangping@cqupt.edu.cn (W.P.); liyong@cqupt.edu.cn (Y.L.)

* Correspondence: weimin@cqupt.edu.cn; Tel.: +86-134-5233-3003

Abstract: Software-defined networking (SDN) provides the prospect of logically centralized management in industrial networks and simplified programming among devices. It also facilitates the reconfiguration of connectivity when there is a network element failure. This paper presents a new Industrial SDN (ISDN) resilience that addresses the gap between two types of resilience: the first is restoration while the second is protection. Using a restoration approach increases the recovery time proportionally to the number of affected flows contrarily to the protection approach which attains the fast recovery. Nevertheless, the protection approach utilizes more flow rules (flow entries) in the switch which in return increments the lookup time taken to discover an appropriate flow entry in

My SDN Innovation based on Java

5

An extended version of this approach is also available online:
<https://www.sciencedirect.com/science/article/abs/pii/S1569190X24000777>

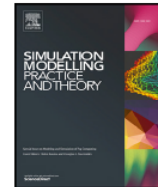
Simulation Modelling Practice and Theory 135 (2024) 102963



Contents lists available at ScienceDirect

Simulation Modelling Practice and Theory

journal homepage: www.elsevier.com/locate/simpat



Industrial IoT regulated by Software-Defined Networking platform for fast and dynamic fault tolerance application[☆]

Nteziriza Nkerabahizi Josbert, Min Wei^{*}, Ping Wang

Department of Automation, Key Lab for Industrial IoT International S&T Cooperative Base, Chongqing University of Posts and Telecommunications, Chongqing, PR China

ARTICLE INFO

Keywords:

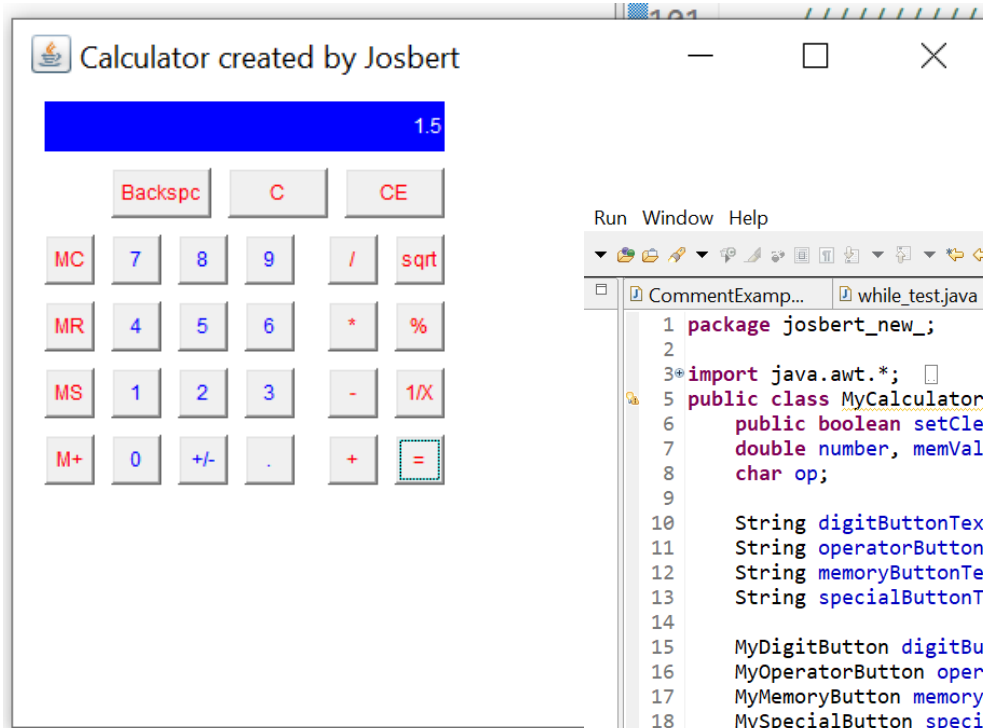
Software-Defined Networking
Industrial Internet of Things
Fault tolerance
Link failure
Quality-of-Service
Simulation and testbed

ABSTRACT

Software-Defined Networking (SDN) technology has emerged as a promising solution to guarantee high reliability in the Industrial Internet of Things (IIoT) ecosystem. Through SDN, both fault tolerance-based Route Protection (RP) and fault tolerance-based Route Restoration (RR) are available to provide traffic rerouting when a network failure occurs in IIoT. RR redefines routes dynamically based on the current network status. However, it increases significantly the recovery time, which is not suitable for the Quality-of-Service (QoS) requirements of IIoT. In contrary, RP ensures fast failover, but it cannot be updated when the network status changes until the timeout interval expires. To deal with these issues, we propose a Dynamic Route Protection (DRP) mechanism that recalculates and reinstalls new optimal link-disjoint routes in accordance with the change rather than awaiting the controller to retransmit new flow instructions. Moreover, DRP responds speedily to forward the data packets from the secondary route to the main route if it is repaired. To recover rapidly when the connection failure affects both a link on the main route and the link on the secondary route simultaneously, the DRP mechanism utilizes the strategy of caching the third route in the controller memory using the Dynamic Hash Table (DHT) structure. DRP considers the heterogeneous traffic flows such as either delay-sensitive or both delay-sensitive and loss-sensitive. Again, this paper introduces

Example of a well-known platform developed in Java programming

5

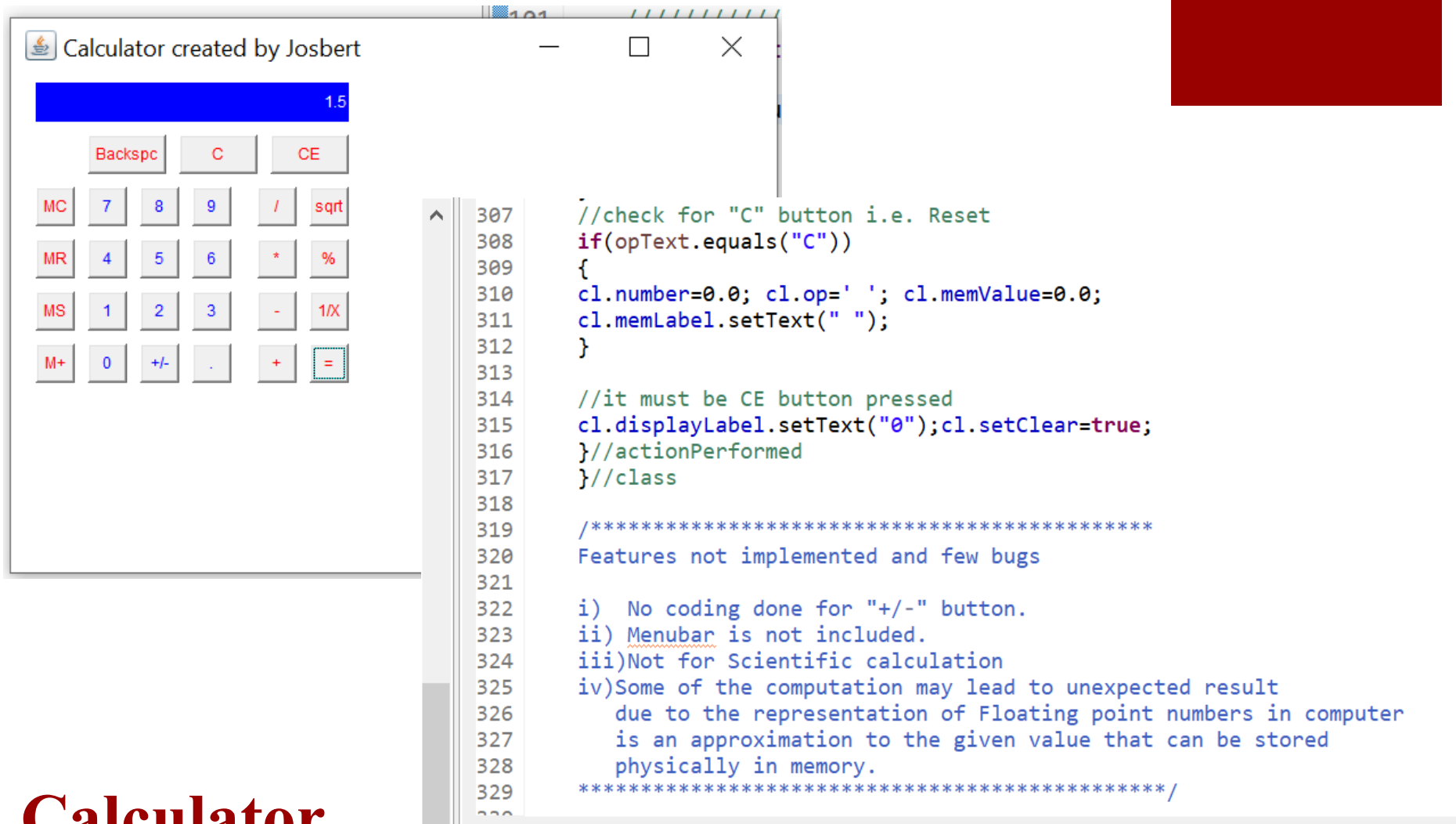


```
Run Window Help
CommentExamp... while_test.java shape_1.java Rectangle_1... main_2.java MyCalculator... »9+
1 package josbert_new_;
2
3 import java.awt.*;
4
5 public class MyCalculator extends Frame {
6     public boolean setClear=true;
7     double number, memValue;
8     char op;
9
10    String digitButtonText[] = {"7", "8", "9", "4", "5", "6", "1", "2", "3", "0", "+/-", "." };
11    String operatorButtonText[] = {"/", "sqrt", "**", "%", "-", "1/X", "+", "=" };
12    String memoryButtonText[] = {"MC", "MR", "MS", "M+" };
13    String specialButtonText[] = {"Backspc", "C", "CE" };
14
15    MyDigitButton digitButton[]=new MyDigitButton[digitButtonText.length];
16    MyOperatorButton operatorButton[]=new MyOperatorButton[operatorButtonText.length];
17    MyMemoryButton memoryButton[]=new MyMemoryButton[memoryButtonText.length];
18    MySpecialButton specialButton[]=new MySpecialButton[specialButtonText.length];
19
20    Label displayLabel=new Label("0",Label.RIGHT);
21    Label memLabel=new Label(" ",Label.RIGHT);
22
23    final int FRAME_WIDTH=325,FRAME_HEIGHT=325;
24    final int HEIGHT=30, WIDTH=30, H_SPACE=10,V_SPACE=10;
25    final int TOPX=30, TOPY=50;
26    ///////////////////////////////////////////////////
27    MyCalculator(String frameText)//constructor
28    {
29        super(frameText);
```

Calculator

Example of a well-known platform developed in Java programming

5



```
307 //check for "C" button i.e. Reset
308 if(opText.equals("C"))
309 {
310     cl.number=0.0; cl.op=' '; cl.memValue=0.0;
311     cl.memLabel.setText(" ");
312 }
313
314 //it must be CE button pressed
315 cl.displayLabel.setText("0");cl.setClear=true;
316 }//actionPerformed
317 }//class
318
319 /*****
320 Features not implemented and few bugs
321
322 i) No coding done for "+/-" button.
323 ii) Menubar is not included.
324 iii)Not for Scientific calculation
325 iv)Some of the computation may lead to unexpected result
326     due to the representation of Floating point numbers in computer
327     is an approximation to the given value that can be stored
328     physically in memory.
329 *****/
```

Calculator

1.1 Java AWT Tutorial

Java AWT (Abstract Window Toolkit) is *an API to develop GUI or window-based applications* in java

The java.awt **package** provides **classes** for AWT API such as **TextField, Label, TextArea, RadioButton, CheckBox, Choice, List** etc.

1.1 Java AWT Tutorial

Java AWT (Abstract Window Toolkit) is *an API to develop GUI or window-based applications* in java

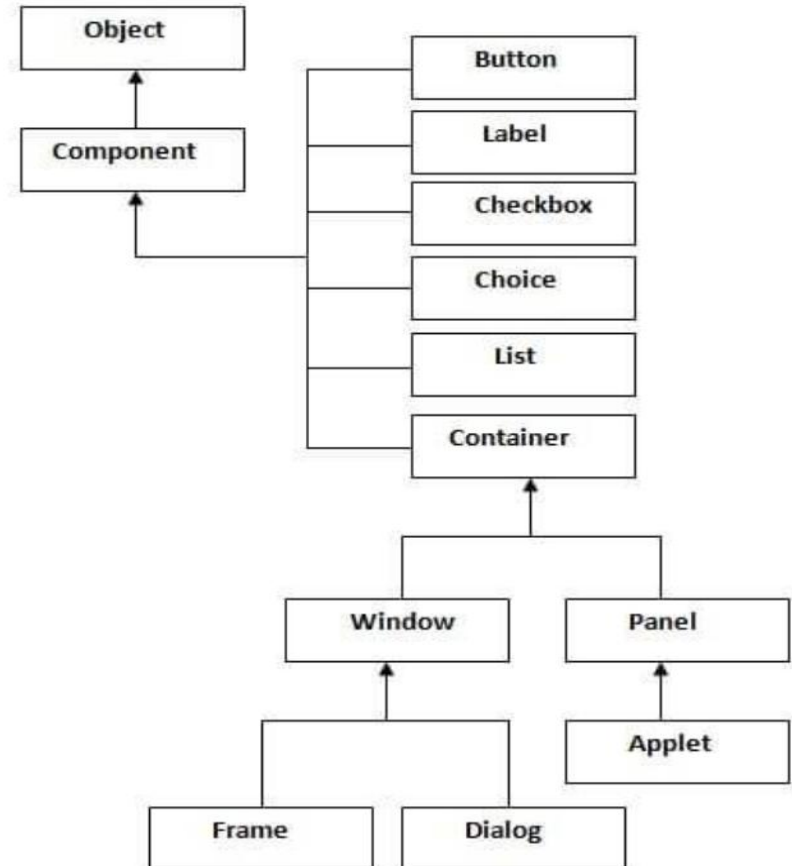
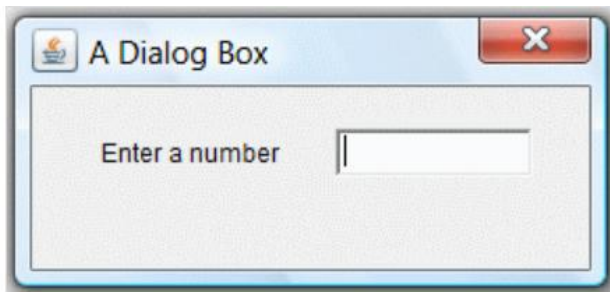
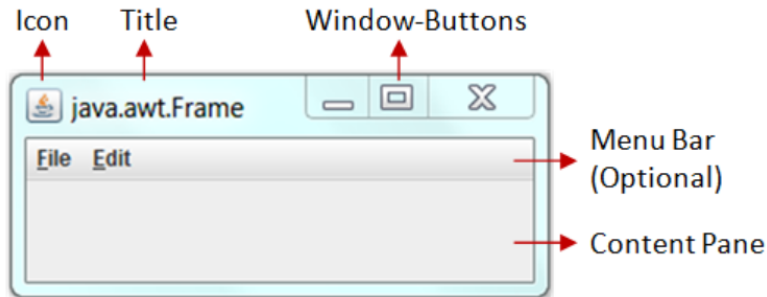
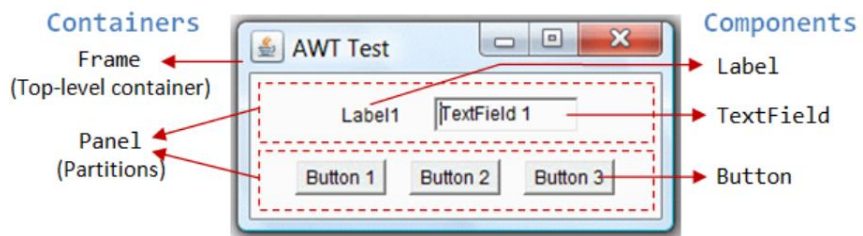
The java.awt **package** provides **classes** for AWT API such as **TextField, Label, TextArea, RadioButton, CheckBox, Choice, List** etc.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

42

1.1 Java AWT Tutorial

The hierarchy of Java AWT classes are given below.



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

43

1.1 Java AWT Tutorial

The hierarchy of Java AWT classes are given below.

Frame:

The Frame is the container that contain title bar and can have menu bars. It can have other components like button, textfield etc.

Dialogue:

The Dialog control represents a top-level window with a border and a title used to take some form of input from the user. It inherits the Window class. **Unlike Frame, it doesn't have maximize and minimize buttons.**

1.1 Java AWT Tutorial

Container:

The Container is a component in AWT that can contain other components like buttons, textfields, labels etc. The classes that extends Container class are known as container such as Window, Frame, Dialog, Panel, and Applet.

Window:

The window is the container that have no borders and menu bars. You must use frame, dialog or another window for creating a window.

Panel:

The Panel is the container that doesn't contain title bar and menu bars. It can have other components like button, textfield etc.

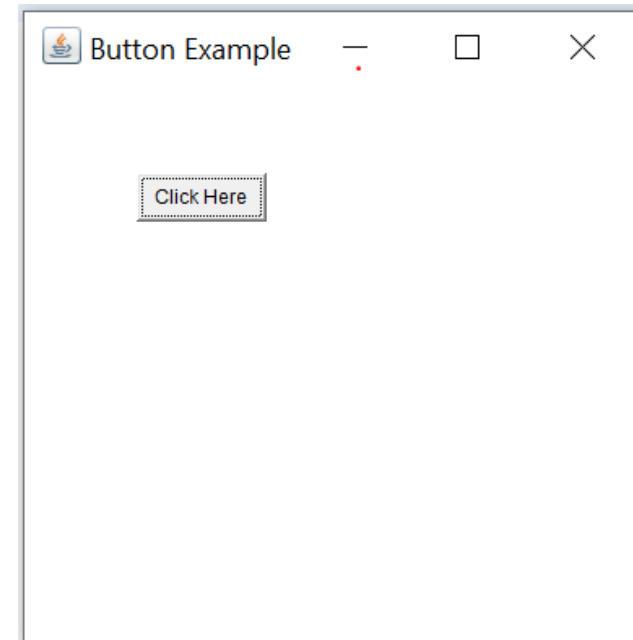
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

45

1.1 Java AWT Button Example

```
1 package josbert_new_;
2 import java.awt.*;
3 public class ButtonEx_cmts {
4
5     public static void main(String[] args) {
6         // Create a new Frame instance with the title "Button Example"
7         Frame f=new Frame("Button Example");
8
9         // Create a Button instance with the label "Click Here"
10        Button b=new Button("Click Here");
11
12        // Set the position and size of the button on the frame
13        b.setBounds(80,100,80,30);
14
15        // Add the button to the frame
16        f.add(b);
17
18        // Set the size of the frame
19        f.setSize(400,400);
20
21        // Set layout to null to manually position components
22        f.setLayout(null);
23
24        // Make the frame visible to display the button
25        f.setVisible(true);
26    }
27 }
```

Sample Output:



1.2 Java Swing Examples

There are two ways to create a frame:

- By creating the object of Frame class (association)
- By extending Frame class (inheritance)

We can write the code of swing inside the main(), constructor or any other method.

Simple Java Swing Example

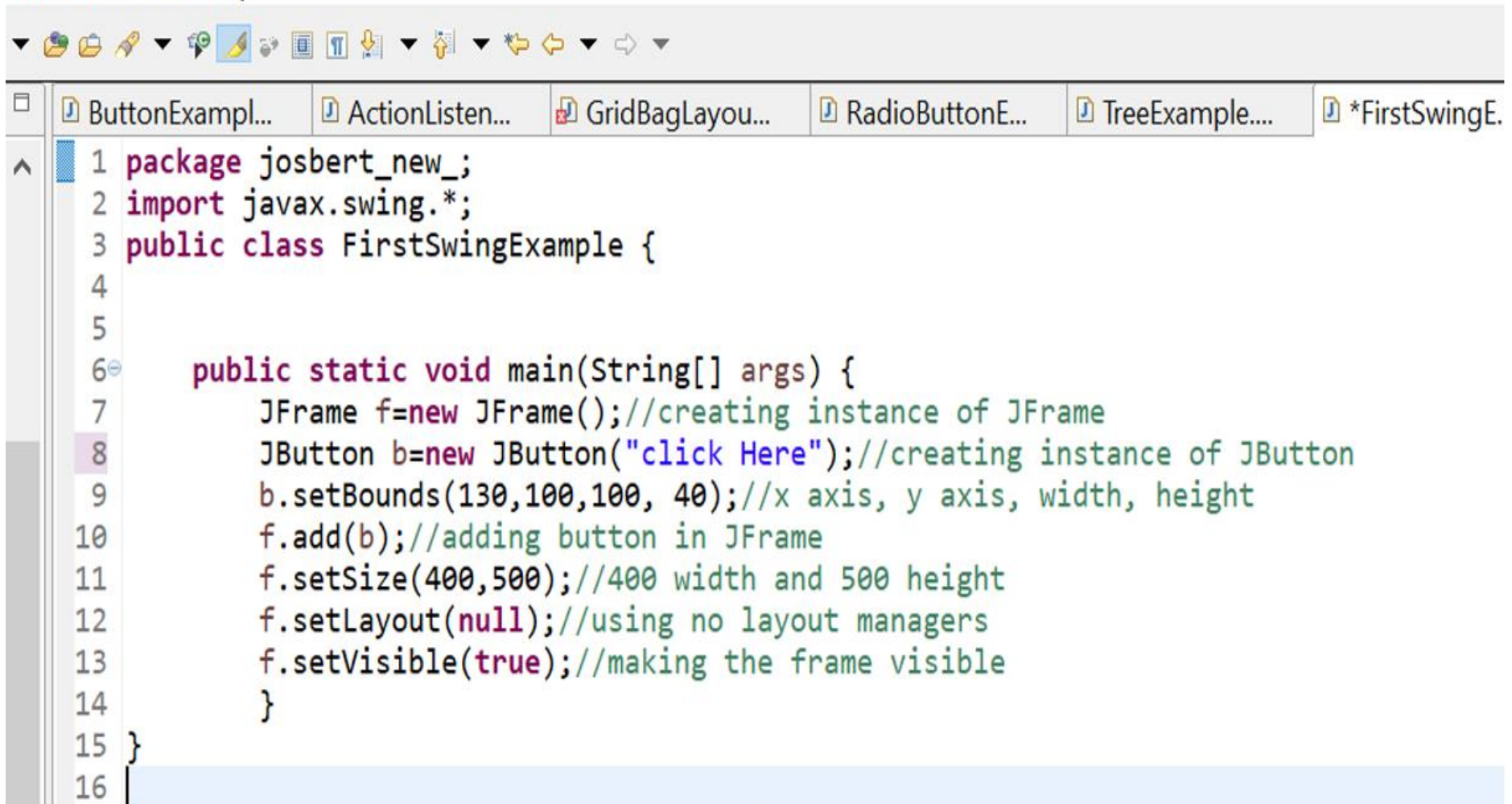
Let's see a simple swing example where we are creating one button and adding it on the JFrame object inside the main() method.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

47

1.2 Java Swing Examples

Run Window Help



The screenshot shows an IDE window with a menu bar (Run, Window, Help) and a toolbar. The tab bar contains several files: ButtonExempl..., ActionListener..., GridBagLayout..., RadioButtonE..., TreeExample..., and *FirstSwingE. The code editor displays the following Java code:

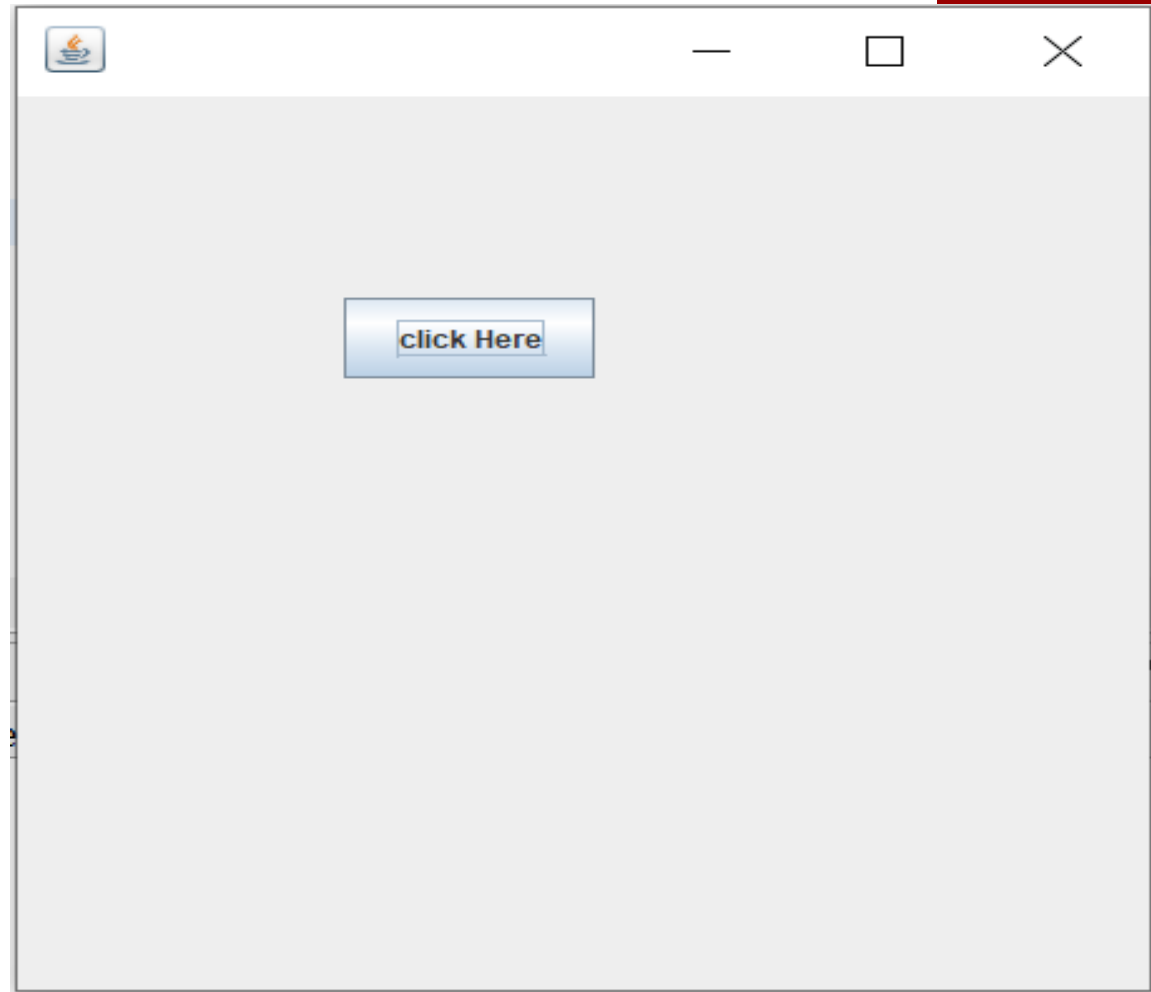
```
1 package josbert_new_;
2 import javax.swing.*;
3 public class FirstSwingExample {
4
5
6     public static void main(String[] args) {
7         JFrame f=new JFrame();//creating instance of JFrame
8         JButton b=new JButton("click Here");//creating instance of JButton
9         b.setBounds(130,100,100, 40);//x axis, y axis, width, height
10        f.add(b);//adding button in JFrame
11        f.setSize(400,500);//400 width and 500 height
12        f.setLayout(null);//using no layout managers
13        f.setVisible(true);//making the frame visible
14    }
15 }
16
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

48

17.2 Java Swing Examples

Sample Output



1.3 Java ActionListener Interface

The Java ActionListener is notified whenever you click on the button or menu item. It is notified against ActionEvent. The ActionListener interface is found in java.awt.event **package**. It has only one method: actionPerformed().

actionPerformed() method

The actionPerformed() method is invoked automatically whenever you click on the registered component.

1. public abstract void actionPerformed(ActionEvent e);

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

50

Sample Output:

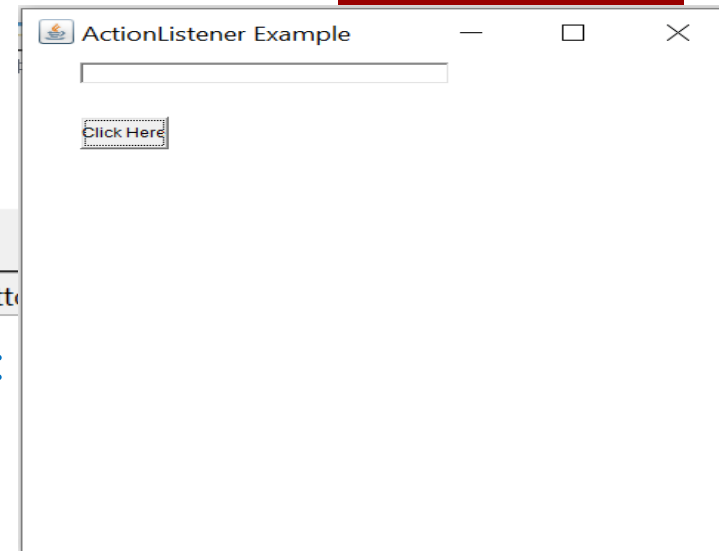
1.3 Java ActionListener Interface

Java ActionListener Example: On Button click

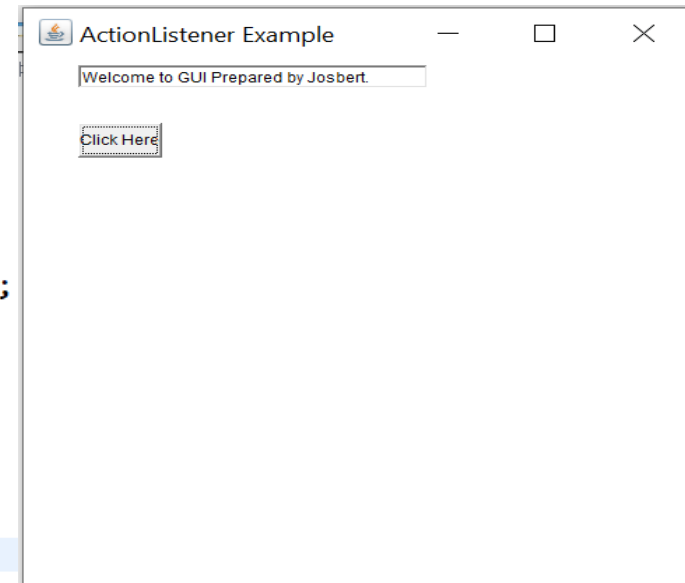
```
1 package josbert_new_;
2
3 import java.awt.*;
4 import java.awt.event.*;
5
6 public class ActionListenerExample {
7
8     public static void main(String[] args) {
9         Frame f=new Frame("ActionListener Example");
10        final TextField tf=new TextField();
11        tf.setBounds(50,50, 250,20);
12        Button b=new Button("Click Here");
13        b.setBounds(50,100,60,30);
14
15        b.addActionListener(new ActionListener(){
16            public void actionPerformed(ActionEvent e){
17                tf.setText("Welcome to GUI Prepared by Josbert.");
18            }
19        });
20        f.add(b);f.add(tf);
21        f.setSize(500,500);
22        f.setLayout(null);
23        f.setVisible(true);
24    }
25 }
```

Fall-2026-Dr. Josbert

Before click:



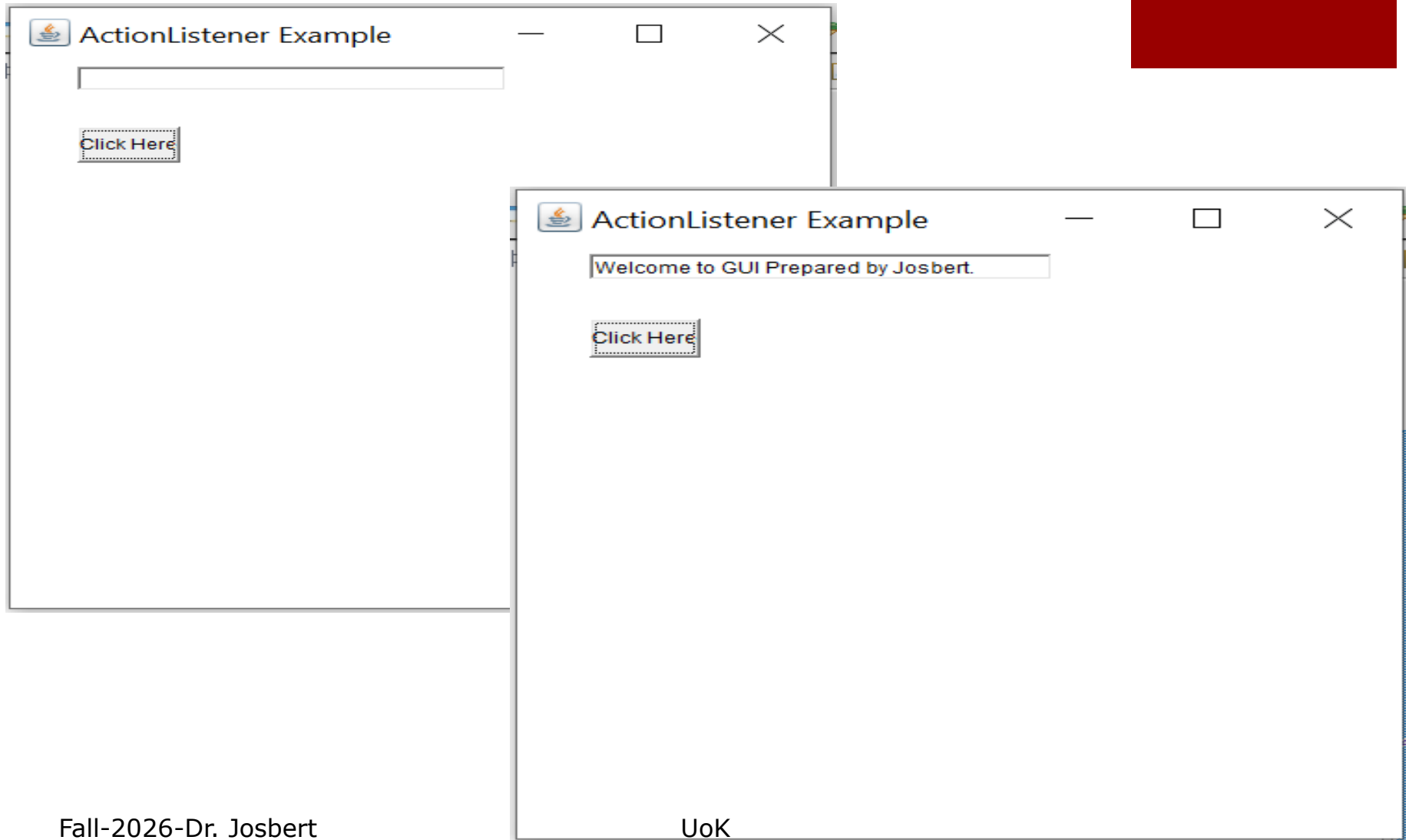
After click:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

51

1.3 Java ActionListener Interface



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

52

1.4 JApplet

As we prefer Swing to AWT. Now we can use JApplet that can have all the controls of swing. The JApplet class extends the Applet class.

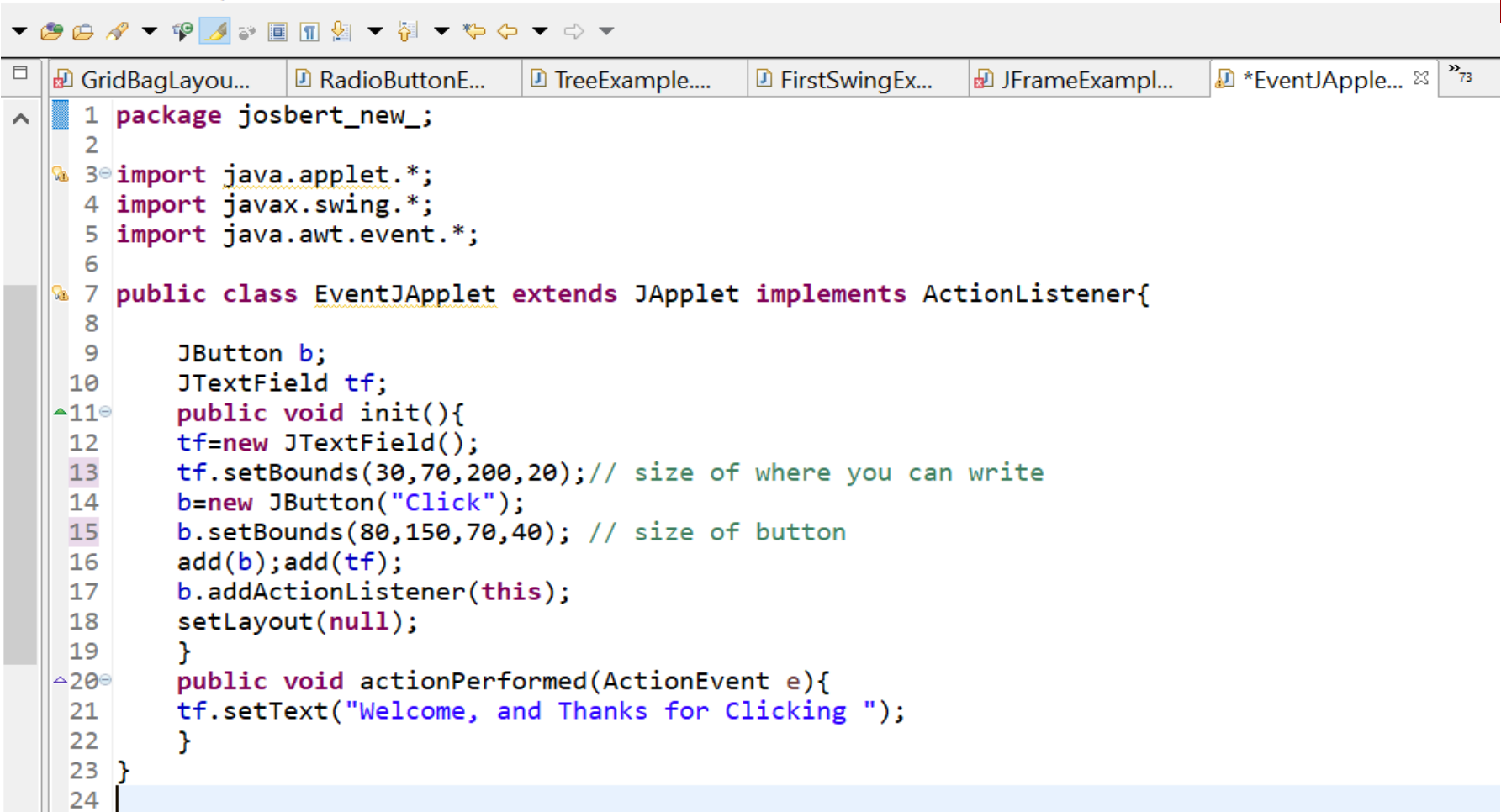
Example of EventHandling in JApplet:

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

53

1.4 JApplet Example of EventHandling in JApplet:

Run Window Help



The screenshot shows an IDE window with a menu bar (Run, Window, Help) and a toolbar. The title bar lists several files: GridBagLayout..., RadioButtonE..., TreeExample..., FirstSwingEx..., JFrameExempl..., and *EventJApple... (selected). The code editor displays the following Java code:

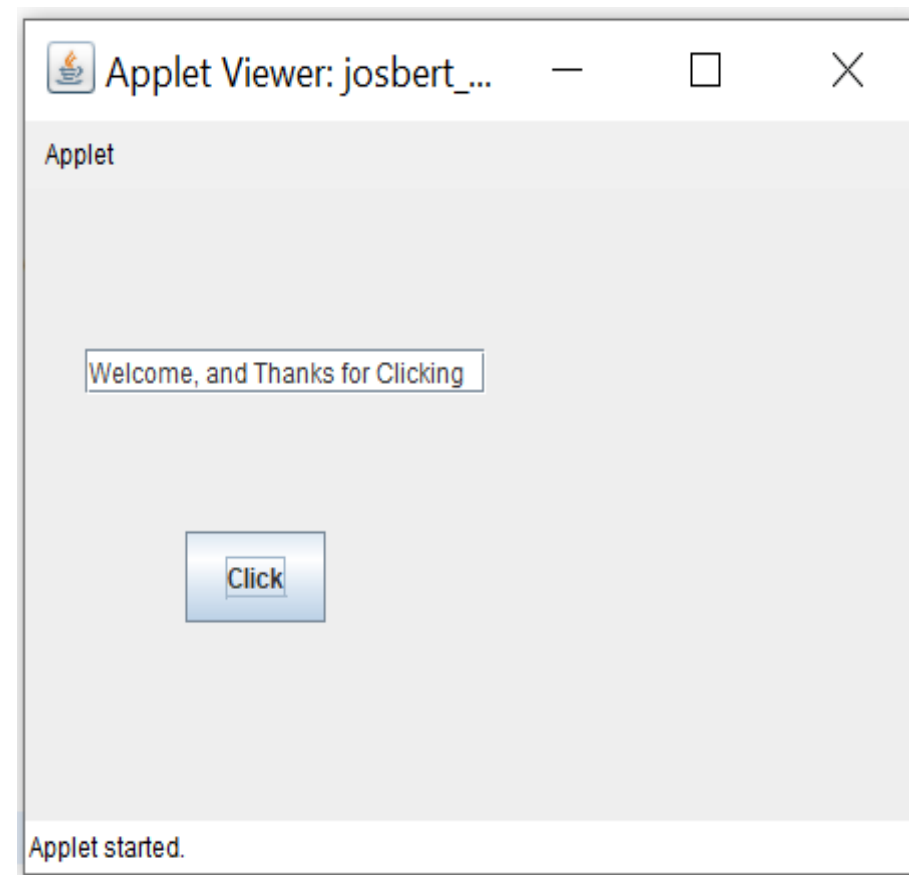
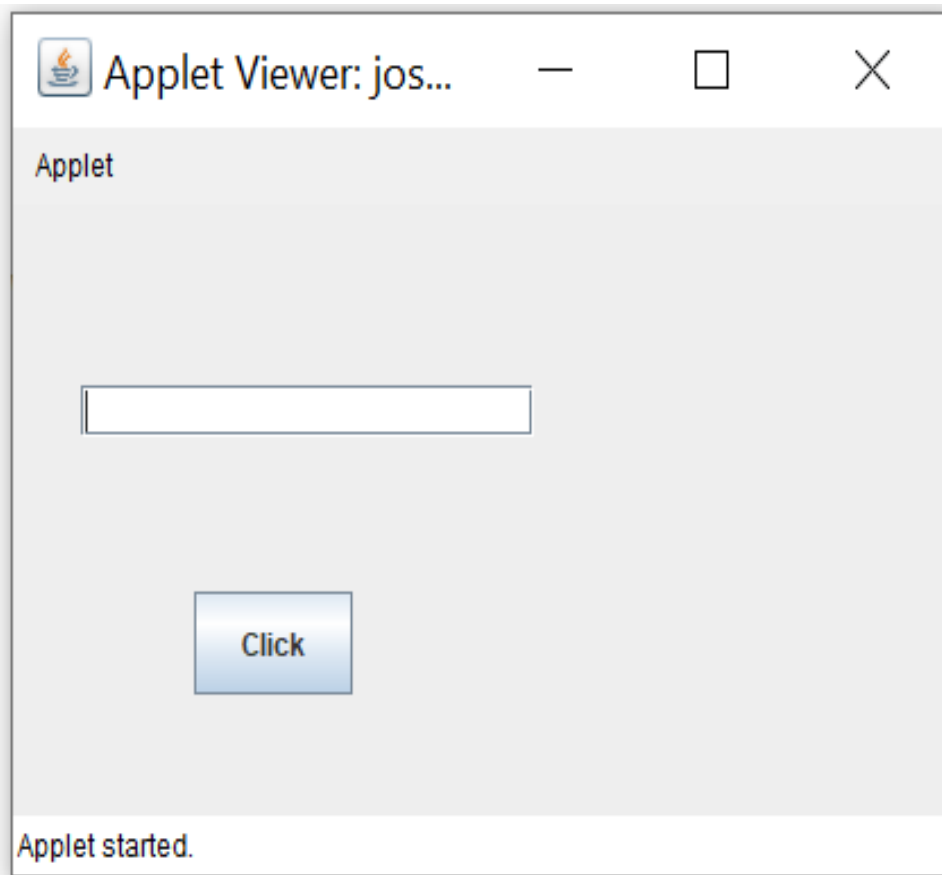
```
1 package josbert_new_;
2
3 import java.applet.*;
4 import javax.swing.*;
5 import java.awt.event.*;
6
7 public class EventJApplet extends JApplet implements ActionListener{
8
9     JButton b;
10    JTextField tf;
11    public void init(){
12        tf=new JTextField();
13        tf.setBounds(30,70,200,20); // size of where you can write
14        b=new JButton("Click");
15        b.setBounds(80,150,70,40); // size of button
16        add(b);add(tf);
17        b.addActionListener(this);
18        setLayout(null);
19    }
20    public void actionPerformed(ActionEvent e){
21        tf.setText("Welcome, and Thanks for Clicking ");
22    }
23 }
24
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

54

1.4 JApplet Example of EventHandling in JApplet:

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

55

17.4 JApplet Example of EventHandling in JApplet:

In the above example, we have created all the controls in `init()` method because it is invoked only once.

myapplet.html

1. <html>

2. <body>

3. <applet

code="EventJApplet.class"

width="300"

height="300">

</applet>

</body>

</html>

1.5 JDialog

The JDialog control represents a top-level window with a border and a title used to take some form of input from the user. It inherits the Dialog class. **Unlike JFrame, it doesn't have maximize and minimize buttons.**

JDialog class declaration

Let's see the declaration for javax.swing.JDialog class.

1. **public class** JDialog **extends** Dialog **implements**
WindowConstants, Accessible, RootPaneContainer

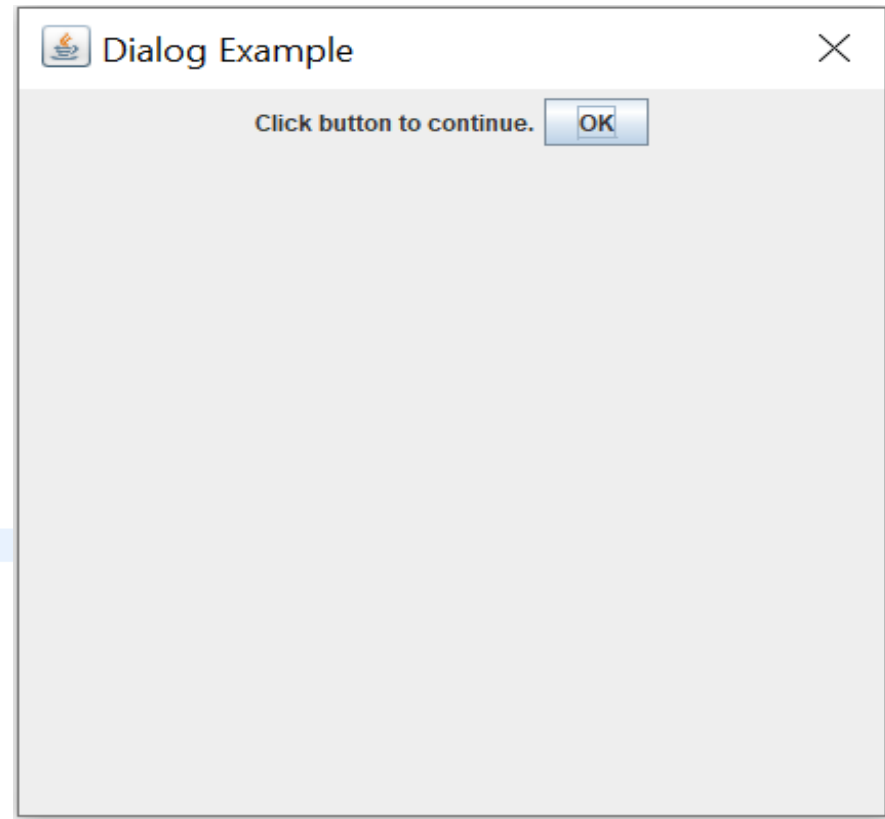
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

57

1.5 JDialog

JDialog Example

```
1 package josbert_new_;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.*;
6
7 public class DialogExample {
8
9     private static JDialog d;
10    DialogExample() {
11        JFrame f= new JFrame();
12        d = new JDialog(f , "Dialog Example", true);
13        d.setLayout( new FlowLayout() );
14        JButton b = new JButton ("OK");
15        b.addActionListener ( new ActionListener()
16        {
17            public void actionPerformed((ActionEvent e )
18            {
19                // when you click on OK, the frame will closed
20                DialogExample.d.setVisible(false);
21            }
22        });
23        d.add( new JLabel ("Click button to continue."));
24        d.add(b);
25        d.setSize(300,300);
26        d.setVisible(true);
27    }
28    public static void main(String args[])
29    {
30        new DialogExample();
31    }
32 }
```



1.6 JPanel

The JPanel is a simplest container class. It provides space in which an application can attach any other component. It inherits the JComponents class.

JPanel class declaration

1. **public class** JPanel **extends** JComponent **implements** Accessible

Java JPanel Example

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

59

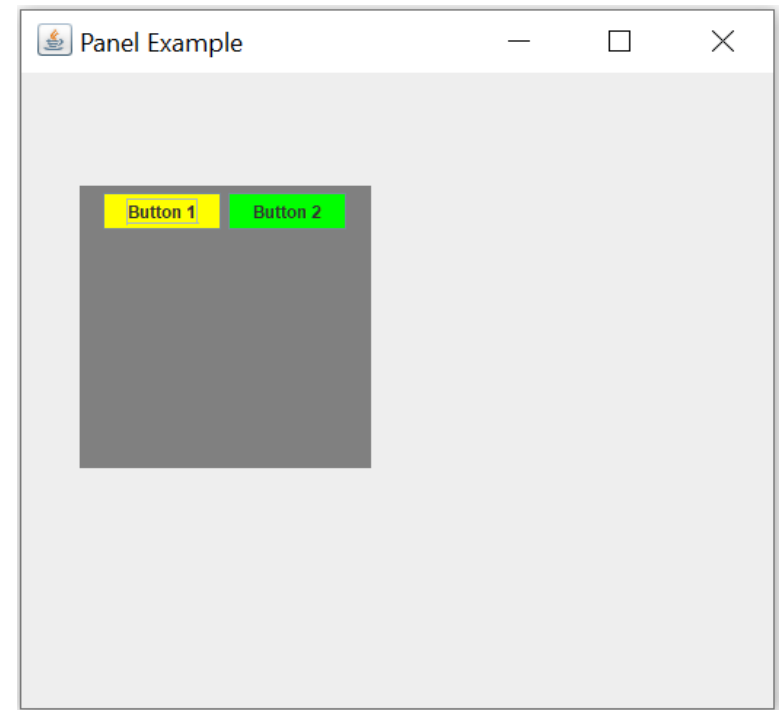
1.6 JPanel

Java JPanel Example

Run Window Help

```
1 package josbert_new_;
2 import java.awt.*;
3 import javax.swing.*;
4
5 public class PanelExample {
6
7     PanelExample()
8     {
9         JFrame f= new JFrame("Panel Example");
10        JPanel panel=new JPanel();
11        panel.setBounds(40,80,200,200);
12        panel.setBackground(Color.gray);
13        JButton b1=new JButton("Button 1");
14        b1.setBounds(50,100,80,30);
15        b1.setBackground(Color.yellow);
16        JButton b2=new JButton("Button 2");
17        b2.setBounds(100,100,80,30);
18        b2.setBackground(Color.green);
19        panel.add(b1); panel.add(b2);
20        f.add(panel);
21        f.setSize(400,400);
22        f.setLayout(null);
23        f.setVisible(true);
24    }
25    public static void main(String args[])
26    {
27        new PanelExample();
28    }
29 }
```

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

60

1.6 JPanel

Java JPanel Example

Let's add more comments to this program

```
1 package josbert_new_;
2 import java.awt.*; // Importing the AWT package for building the user interface
3 import javax.swing.*; // Importing the Swing package for advanced GUI components
4 public class PanelExample_cmts {
5
6     PanelExample_cmts()
7     {
8
9         // Creating a new JFrame with the title "Panel Example"
10        JFrame f = new JFrame("Panel Example");
11
12        // Creating a new JPanel to hold components
13        JPanel panel = new JPanel();
14
15        // Setting the position and size of the panel within the frame
16        panel.setBounds(40, 80, 200, 200); // x, y, width, height
17        panel.setBackground(Color.gray); // Setting the background color of the panel to gray
18
19        // Creating the first button (Button 1)
20        JButton b1 = new JButton("Button 1");
21        b1.setBounds(50, 100, 80, 30); // Setting position and size of Button 1
22        b1.setBackground(Color.yellow); // Setting the background color of Button 1 to yellow
23
24        // Creating the second button (Button 2)
25        JButton b2 = new JButton("Button 2");
26        b2.setBounds(100, 100, 80, 30); // Setting position and size of Button 2
27        b2.setBackground(Color.green); // Setting the background color of Button 2 to green
28
29        // Adding the buttons to the panel
30        panel.add(b1);
31        panel.add(b2);
32    }
33}
```

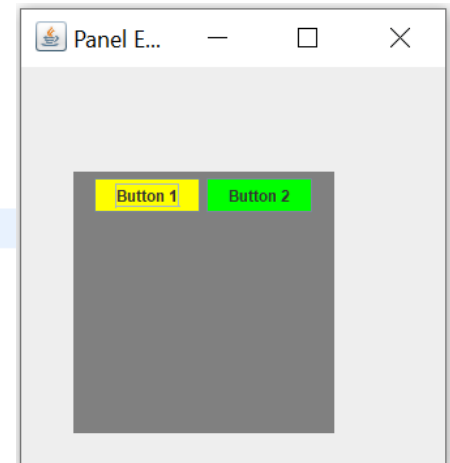

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

61

1.6 JPanel

Java JPanel Example

```
32
33     // Adding the panel to the JFrame
34     f.add(panel);
35
36     // Setting the size of the JFrame
37     f.setSize(400, 400); // width, height
38
39     // Setting the layout of the JFrame to null for absolute positioning
40     f.setLayout(null);
41
42     // Making the JFrame visible
43     f.setVisible(true);
44
45     // Setting default close operation to exit the application when the frame is closed
46     f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47 }
48
49 // Main method to launch the application
50 public static void main(String args[]) {
51     // Creating an instance of PanelExample, which sets up the GUI
52     new PanelExample_cmts|();
53 }
54 }
```



1.7 Overview of some Swing Components

Java JButton

The JButton class is used to create a labeled button that has platform independent implementation. The application result in some action when the button is pushed. It inherits AbstractButton class.

JButton class declaration

Let's see the declaration for javax.swing.JButton class.

1. **public class JButton extends AbstractButton implements Accessible.**

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

63

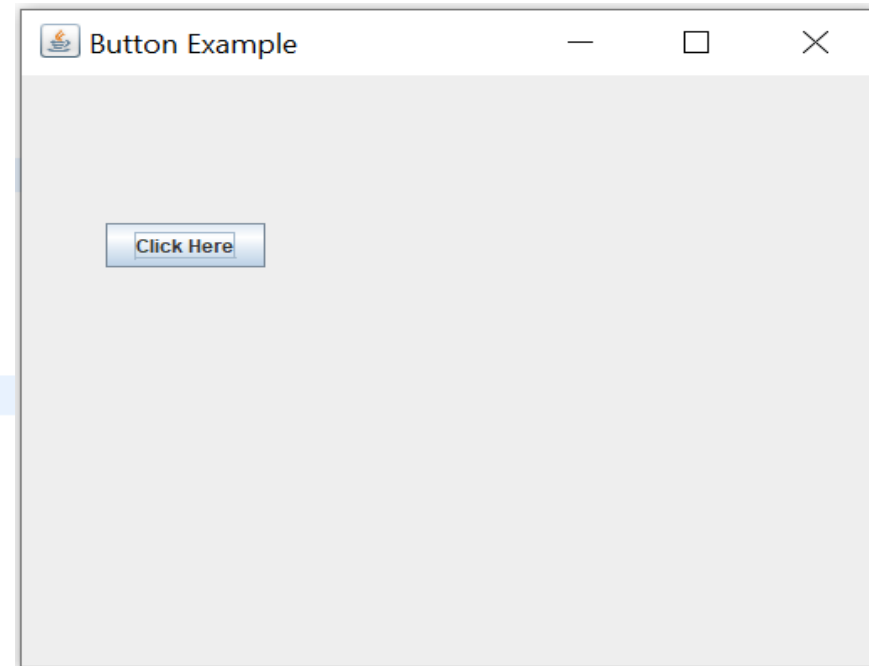
1.7 Overview of some Swing Components

Java JButton Example

Sample Output:

Run Window Help

```
FirstSwingEx... JFrameExampl... EventJApplet... DialogExampl... PanelExample... ButtonExampl... »76
1 package josbert_new_;
2 import javax.swing.*;
3 public class ButtonExample_ {
4     public static void main(String[] args) {
5         JFrame f=new JFrame("Button Example");
6         JButton b=new JButton("Click Here");
7         b.setBounds(50,100,95,30);
8         f.add(b);
9         f.setSize(400,400);
10        f.setLayout(null);
11        f.setVisible(true); }
12
13 }
14
```



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

64

1.7 Overview of some Swing Components Java JButton Example

AWT Button Example with ActionListener

In the following example, we are handling the button click events by implementing ActionListener Interface.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

AWT Button Example with ActionListener

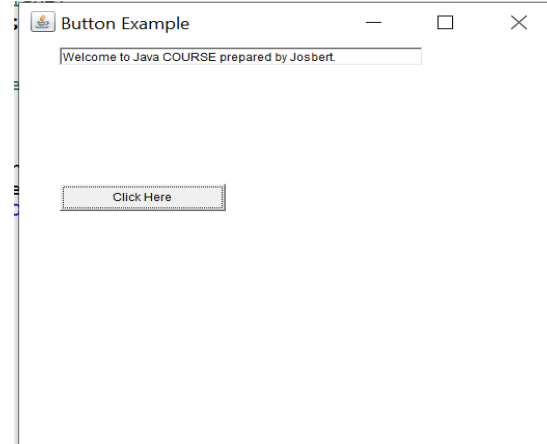
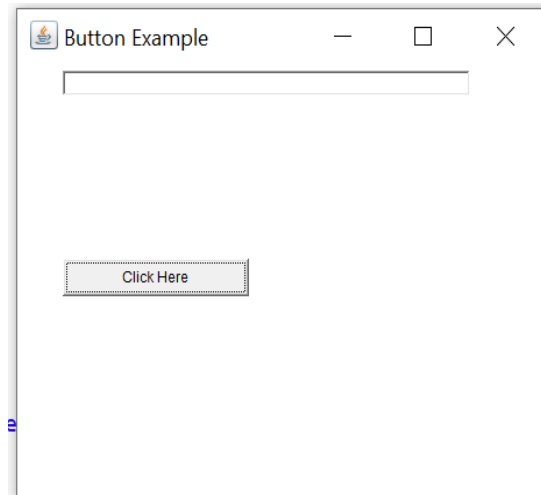
65

Run Window Help



TextAreaExam... CheckboxExam... ChoiceExampl... ChoiceExampl... ButtonExampl... *ButtonExamp... »99+

```
1 package josbert_new_;  
2 //importing necessary libraries  
3 import java.awt.*;  
4 import java.awt.event.*;  
5  
6 public class ButtonExample_4 {  
7  
8     public static void main(String[] args) {  
9         // create instance of frame with the label  
10        Frame f = new Frame("Button Example");  
11        final TextField tf=new TextField();  
12        tf.setBounds(50,50, 350,20);  
13        // create instance of button with label  
14        Button b=new Button("Click Here");  
15        // set the position for the button in frame  
16        b.setBounds(50,200,160,30);  
17        b.addActionListener(new ActionListener() {  
18            public void actionPerformed (ActionEvent e) {  
19                tf.setText("Welcome to Java COURSE prepared by Josbert.");  
20            }  
21        });  
22        // adding button the frame  
23        f.add(b);  
24        // adding textfield the frame  
25        f.add(tf);  
26        // setting size, layout and visibility  
27        f.setSize(400,400);  
28        f.setLayout(null);  
29        f.setVisible(true);  
30    }  
31  
32 }
```



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

66

1.7 Java JLabel

The object of JLabel class is a component for placing text in a container. It is used to display a single line of read only text. **The text can be changed by an application but a user cannot edit it directly. It inherits JComponent class.**

JLabel class declaration

Let's see the declaration for javax.swing.JLabel class.

```
1. public class JLabel extends JComponent implements  
SwingConstants, Accessible
```

Java JLabel Example

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

67

1.7 Java JLabel

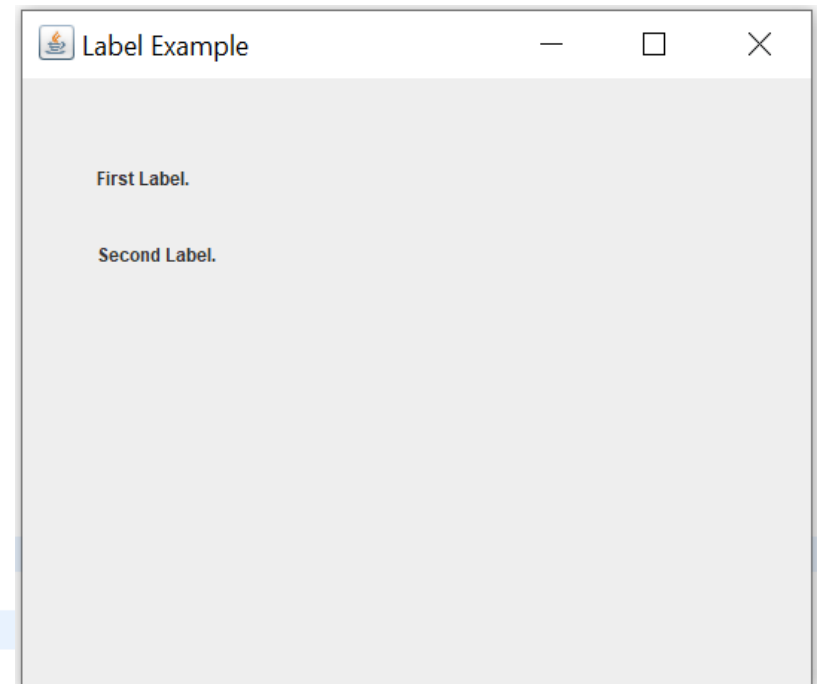
Run Window Help



JFrameExempl... EventJApplet... DialogExempl... PanelExample... ButtonExempl... LabelExample... »77

```
1 package josbert_new_;
2
3
4 import javax.swing.*;
5 public class LabelExample {
6
7     public static void main(String args[])
8     {
9         JFrame f= new JFrame("Label Example");
10        JLabel l1,l2;
11        l1=new JLabel("First Label.");
12        l1.setBounds(50,50, 100,30);
13        l2=new JLabel("Second Label.");
14        l2.setBounds(50,100, 100,30);
15        f.add(l1); f.add(l2);
16        f.setSize(300,300);
17        f.setLayout(null);
18        f.setVisible(true);
19    }
20 }
21
```

Sample Output:



1. 8 JTextField

The object of a JTextField class is a text component that allows the editing of a single line text. It inherits JTextComponent class.

JTextField class declaration

Let's see the declaration for javax.swing.JTextField class.

1. public class JTextField **extends** JTextComponent **implements** SwingConstants

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

69

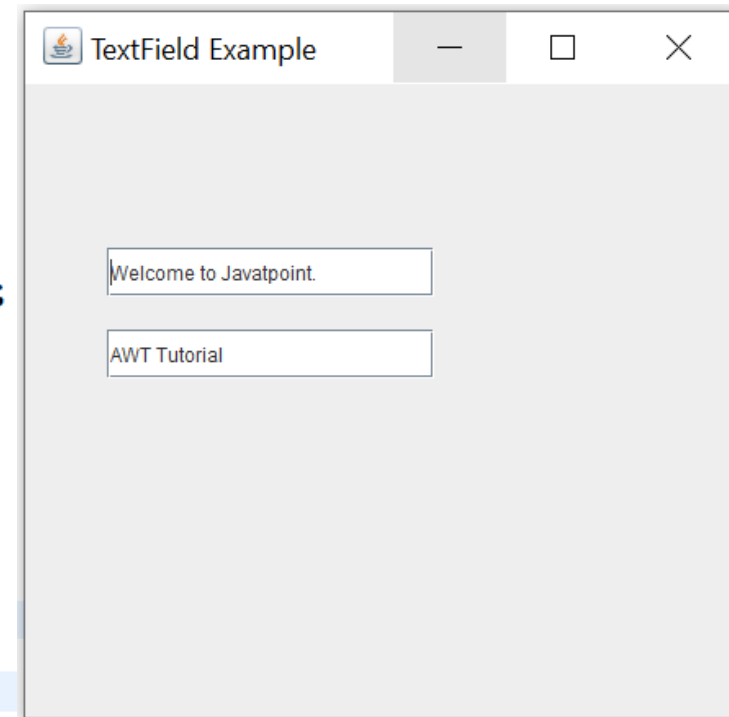
17.8 JTextField

Run Window Help



EventApplet... DialogExempl... PanelExample... ButtonExempl... LabelExample... extFiel

```
1 package josbert_new {
2     josbert_new/src/josbert_new/EventApplet.java
3 import javax.swing.*;
4 public class extFieldExample {
5
6
7     public static void main(String args[])
8     {
9         JFrame f= new JFrame("TextField Example");
10        JTextField t1,t2;
11        t1=new JTextField("Welcome to Javatpoint.");
12        t1.setBounds(50,100, 200,30);
13        t2=new JTextField("AWT Tutorial");
14        t2.setBounds(50,150, 200,30);
15        f.add(t1); f.add(t2);
16        f.setSize(400,400);
17        f.setLayout(null);
18        f.setVisible(true);
19    }
20 }
21
```



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING :Example of a simple calculator

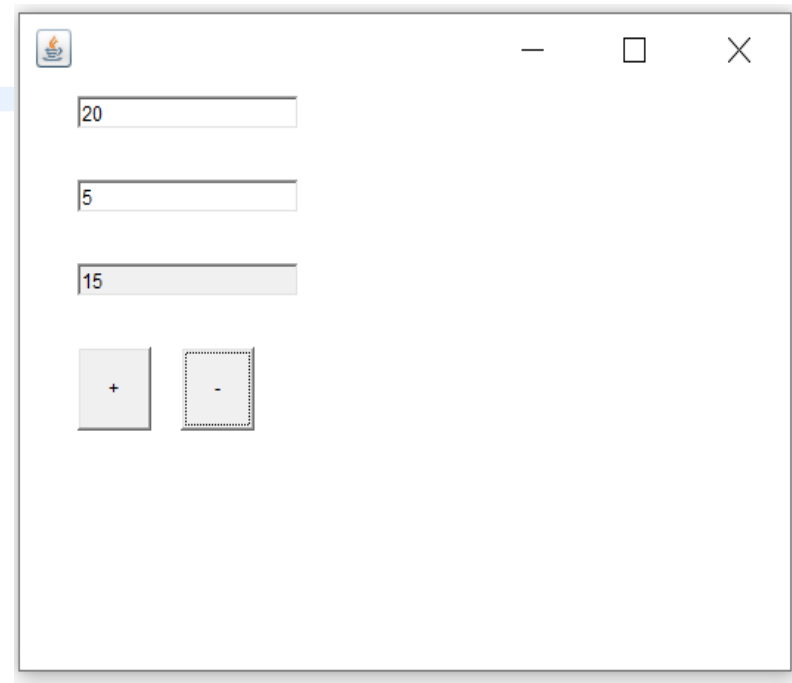
70

Java AWT TextField with ActionListener

```
Run Window Help
package josbert_new_;
import java.awt.*;
import java.awt.event.*;

public class TextFieldExample extends Frame implements ActionListener {
    TextField tf1,tf2,tf3;
    Button b1,b2;
    TextFieldExample(){
        tf1=new TextField();
        tf1.setBounds(50,50,150,20);
        tf2=new TextField();
        tf2.setBounds(50,100,150,20);
        tf3=new TextField();
        tf3.setBounds(50,150,150,20);
        tf3.setEditable(false); b1=new Button("+");
        b1.setBounds(50,200,50,50); b2=new Button("-");
        b2.setBounds(120,200,50,50); b1.addActionListener(this);
        b2.addActionListener(this); add(tf1);add(tf2);add(tf3);add(b1);add(b2);
        setSize(300,300); setLayout(null);
        setVisible(true);
    }
    public void actionPerformed(ActionEvent e) {
        String s1=tf1.getText();
        String s2=tf2.getText();
        int a=Integer.parseInt(s1);
        int b=Integer.parseInt(s2);
        int c=0;
        if(e.getSource()==b1){
            c=a+b;
        }else if(e.getSource()==b2){
            c=a-b; }
        String result=String.valueOf(c);
        tf3.setText(result); }
    public static void main(String[] args) {
        new TextFieldExample();
    }
}
```

Sample Output:



1.9 JTextArea

The object of a JTextArea class is a multi-line region that displays text. **It allows the editing of multiple line text.** It inherits JTextComponent class

JTextArea class declaration

Let's see the declaration for javax.swing.JTextArea class.

1. **public class** JTextArea **extends** JTextComponent

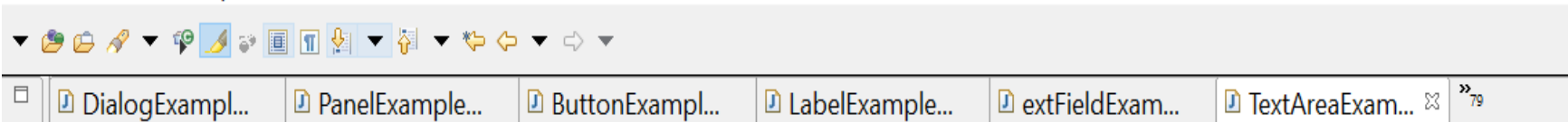
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

1.9 JTextArea

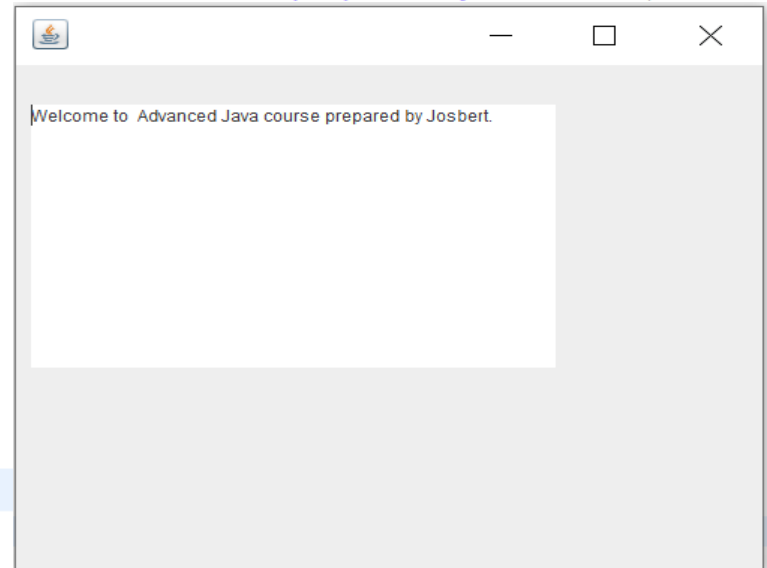
72

Java JTextArea Example

Run Window Help



```
1 package josbert_new_;
2
3 import javax.swing.*;
4 public class TextAreaExample {
5
6
7     TextAreaExample(){
8         JFrame f= new JFrame();
9         JTextArea area=new JTextArea("Welcome to Advanced Java course prepared by Josbert.");
10        area.setBounds(10,30, 350,200);
11        f.add(area);
12        f.setSize(300,300);
13        f.setLayout(null);
14        f.setVisible(true);
15    }
16    public static void main(String args[])
17    {
18        new TextAreaExample();
19    }
20 }
21
```



Sample Output:

1. 9 AWT Checkbox

The Checkbox class is used to create a checkbox. It is used to turn an option **on (true) or off (false)**. Clicking on a Checkbox changes its state from "on" to "off" or from "off" to "on".

Java AWT Checkbox Example

In the following example we are creating two checkboxes using the `Checkbox(String label)` constructor and adding them into the Frame using `add()` method.

1.9 AWT Checkbox

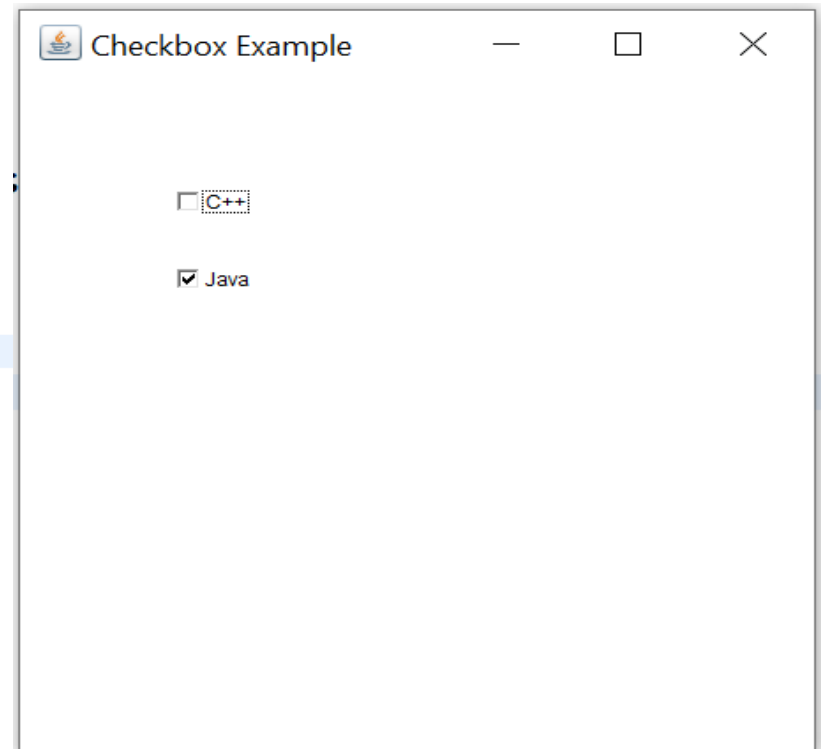
Run Window Help



AEvent.java KeyListenerE... AdapterExamp... TextFieldExa... TextAreaExam... CheckboxExam... » 96

```
1 package josbert_new_;
2
3 import java.awt.*;
4 public class CheckboxExample1 {
5
6     // constructor to initialize
7     CheckboxExample1() {
8         //creating the frame with the title
9         Frame f = new Frame("Checkbox Example");
10        //creating the checkboxes
11        Checkbox checkbox1 = new Checkbox("C++");
12        checkbox1.setBounds(100, 100, 50, 50);
13        Checkbox checkbox2 = new Checkbox("Java", true);
14        //setting location of checkbox in frame
15        checkbox2.setBounds(100, 150, 50, 50);
16        //adding checkboxes to frame
17        f.add(checkbox1);
18        f.add(checkbox2);
19
20        //setting size, layout and visibility of frame
21        f.setSize(400,400);
22        f.setLayout(null);
23        f.setVisible(true);
24    }
25
26    //main method
27    public static void main (String args[])
28    {
29        new CheckboxExample1();
30    }
31 }
```

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

75

11. 7 AWT Checkbox AWT Checkbox Example with ItemListener

Run Window Help



```
1 package josbert_new_;
2
3 //importing necessary packages
4 import java.awt.*;
5 import java.awt.event.*;
6
7 public class CheckboxExample2 {
8
9     // constructor to initialize
10     CheckboxExample2() {
11 //creating the frame
12         Frame f = new Frame ("CheckBox Example");
13 //creating the label
14         final Label label = new Label();
15 //setting the alignment, size of label
16         label.setAlignment(Label.CENTER);
17         label.setSize(400,100);
18 //creating the checkboxes
19         Checkbox checkbox1 = new Checkbox("C++");
20         checkbox1.setBounds(100, 100, 50, 50);
21         Checkbox checkbox2 = new Checkbox("Java");
22         checkbox2.setBounds(100, 150, 50, 50);
23 //adding the checkbox to frame
24 f.add(checkbox1);
25 f.add(checkbox2);
26 f.add(label);
27
```

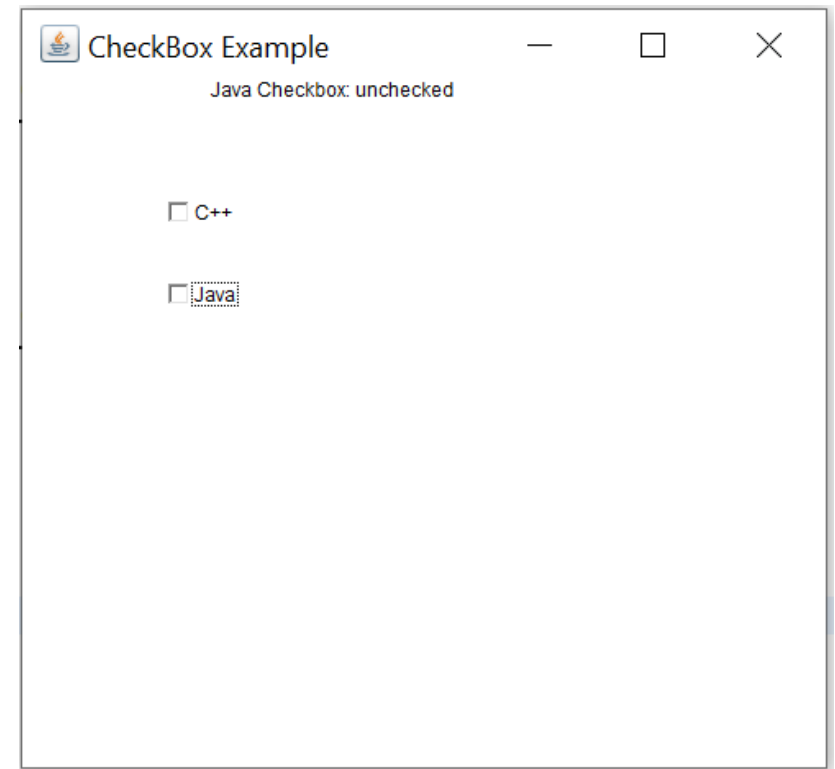
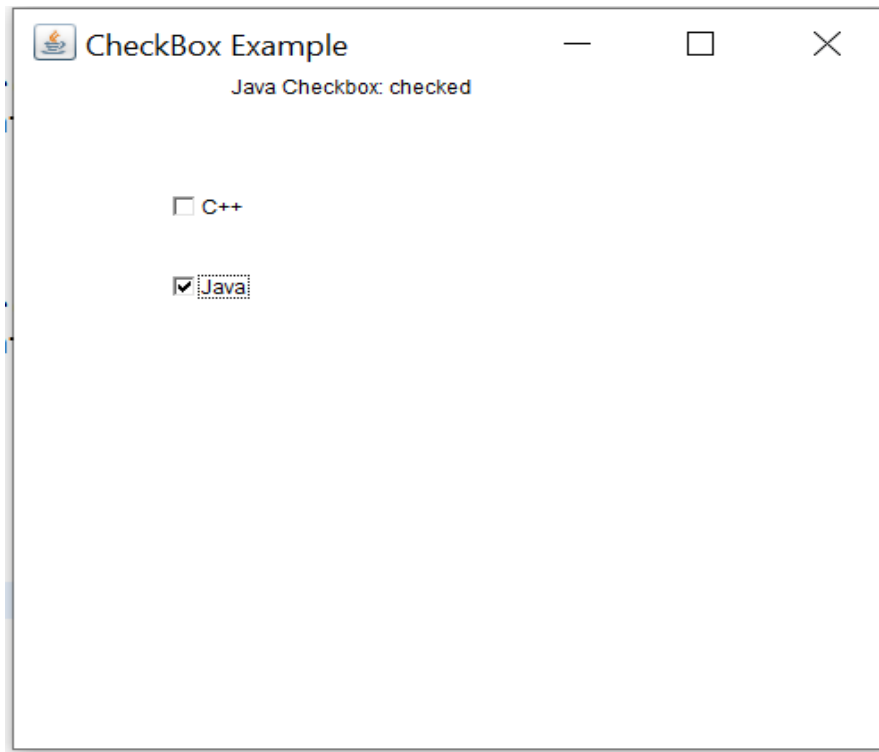
```
28 //adding event to the checkboxes
29     checkbox1.addItemListener(new ItemListener() {
30         public void itemStateChanged(ItemEvent e) {
31             label.setText("C++ Checkbox: "
32                 + (e.getStateChange()==1?"checked":"unchecked"));
33         }
34     });
35     checkbox2.addItemListener(new ItemListener() {
36         public void itemStateChanged(ItemEvent e) {
37             label.setText("Java Checkbox: "
38                 + (e.getStateChange()==1?"checked":"unchecked"));
39         }
40     });
41 //setting size, layout and visibility of frame
42     f.setSize(400,400);
43     f.setLayout(null);
44     f.setVisible(true);
45 }
46 //main method
47 public static void main(String args[])
48 {
49     new CheckboxExample2();
50 }
51
52 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

76

1.9 AWT Checkbox AWT Checkbox Example with ItemListener

Sample Output:



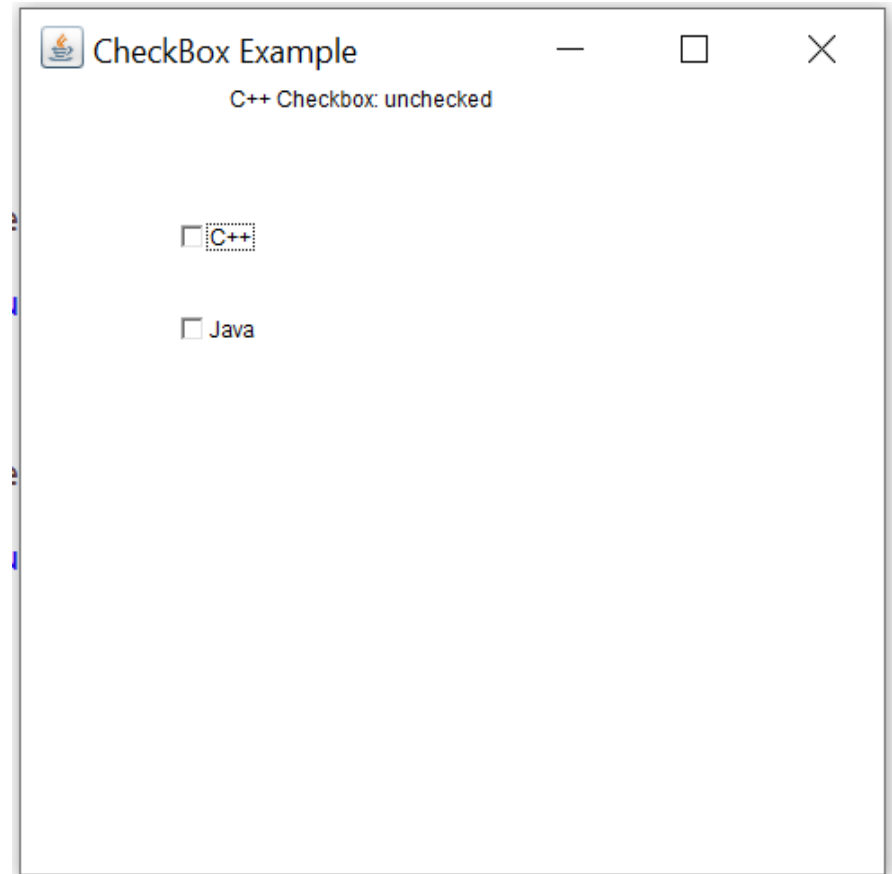
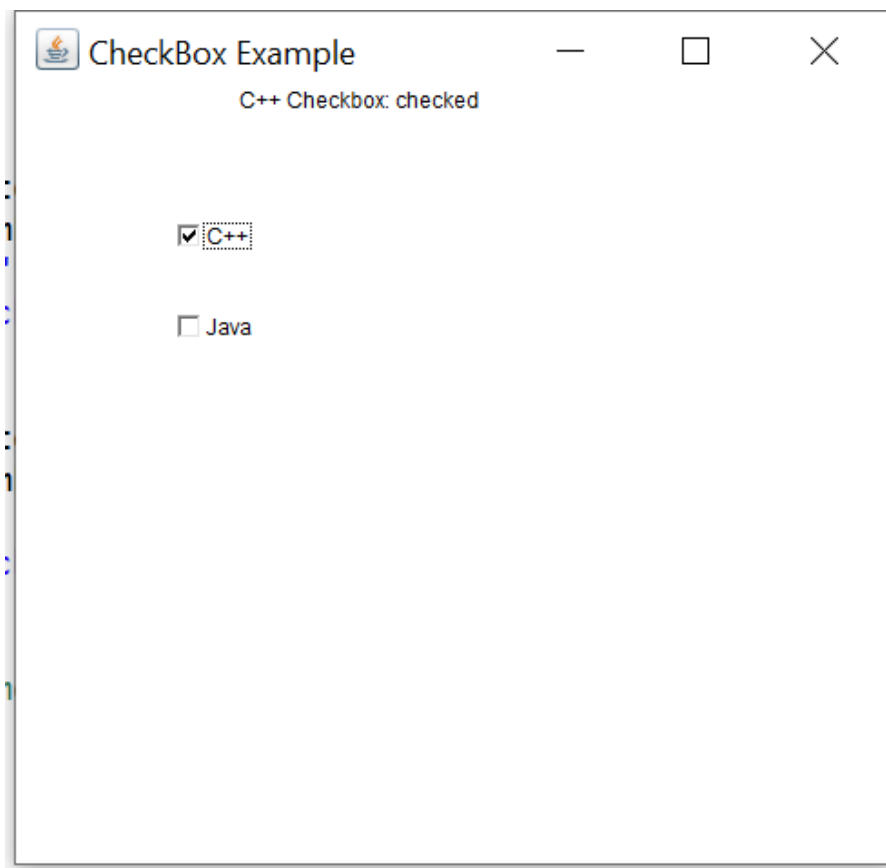
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

77

1.9 AWT Checkbox

AWT Checkbox Example with ItemListener

Sample Output:



1. 10 Java AWT List

The object of List class represents a list of text items. With the help of the List class, user can choose either one item or multiple items. It inherits the Component class.

AWT List class Declaration

1. public class List **extends** Component **implements** ItemSelectable, Accessible

In the following example, we are creating a List component with 5 rows and adding it into the Frame.

1. 10 Java AWT List

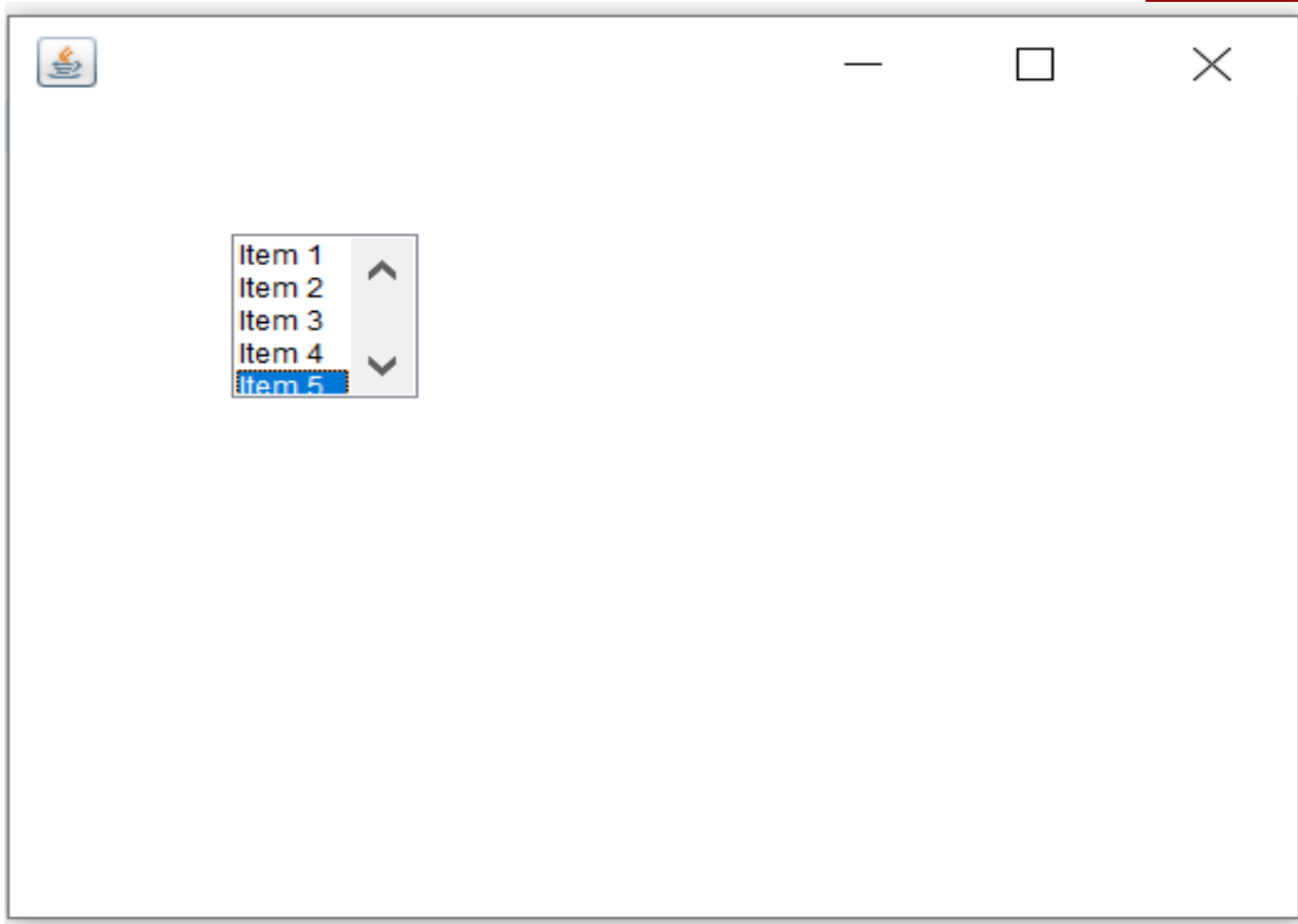
```
ComboBoxExam... A.java B.java A6.java TestSleepMet...
1 package josbert_new_;
2
3 //importing awt class
4 import java.awt.*;
5
6 public class ListExample1 {
7
8     // class constructor
9     ListExample1() {
10         // creating the frame
11         Frame f = new Frame();
12         // creating the list of 5 rows
13         List l1 = new List(5);
14
15         // setting the position of list component
16         l1.setBounds(100, 100, 75, 75);
17
18         // adding list items into the list
19         l1.add("Item 1");
20         l1.add("Item 2");
21         l1.add("Item 3");
22         l1.add("Item 4");
23         l1.add("Item 5");
24
25         // adding the list to frame
26         f.add(l1);
27
28         // setting size, layout and visibility of frame
29         f.setSize(400, 400);
30         f.setLayout(null);
31         f.setVisible(true);
32     }
33
34 //main method
35 public static void main(String args[])
36 {
37     new ListExample1();
38 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

80

1. 10 Java AWT List

Sample Output:



1. 10 Java AWT List

Java AWT List Example with ActionListener

In the following example, we are creating two List components, a Button and a Label and adding them into the frame. Here, we are generating an event on the button using the **addActionListener (ActionListener l)** method. On clicking the button, it displays the selected programming language and the framework.

1. 10 Java AWT List Java AWT List Example with ActionListener

```
1 package josbert_new_;
2
3
4 //importing awt and event class
5 import java.awt.*;
6 import java.awt.event.*;
7
8 public class ListExample2 {
9
10     // class constructor
11     ListExample2() {
12         // creating the frame
13         Frame f = new Frame();
14         // creating the label which is final
15         final Label label = new Label();
16
17         // setting alignment and size of label
18         label.setAlignment(Label.CENTER);
19         label.setSize(500, 100);
20
21         // creating a button
22         Button b = new Button("Show");
23
24         // setting location of button
25         b.setBounds(200, 150, 80, 30);
26
27         // creating the 2 list objects of 4 rows
28         // adding items to the list using add()
29         // setting location of list components
30         final List l1 = new List(4, false);
31         l1.setBounds(100, 100, 70, 70);
32         l1.add("C");
```

1. 10 Java AWT List Java AWT List Example with ActionListener

```
33     l1.add("C++");
34     l1.add("Java");
35     l1.add("PHP");
36
37
38     final List l2=new List(4, true);
39     l2.setBounds(100, 200, 70, 70);
40     l2.add("Turbo C++");
41     l2.add("Spring");
42     l2.add("Hibernate");
43     l2.add("CodeIgniter");
44
45     // adding List, Label and Button to the frame
46     f.add(l1);
47     f.add(l2);
48     f.add(label);
49     f.add(b);
50
51     // setting size, layout and visibility of frame
52     f.setSize(450,450);
53     f.setLayout(null);
54     f.setVisible(true);
55
```

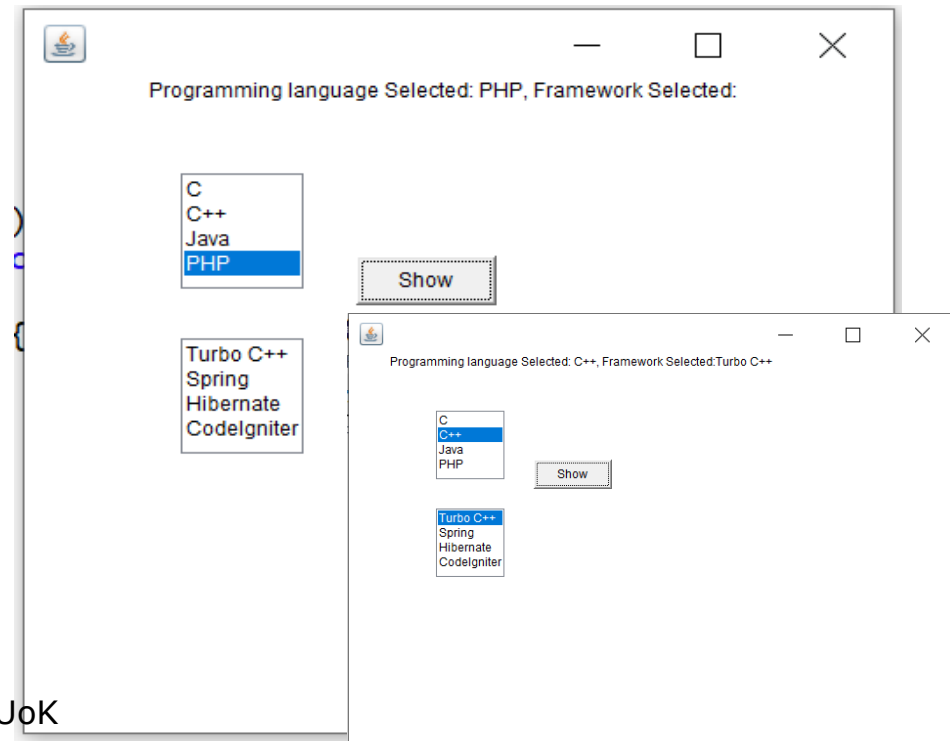
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

84

1. 10 Java AWT List Java AWT List Example with ActionListener

```
56 // generating event on the button
57 b.addActionListener(new ActionListener() {
58     public void actionPerformed(ActionEvent e) {
59         String data = "Programming language Selected: "+l1.getSelectedItem();
60         data += ", Framework Selected:";
61         for(String frame:l2.getSelectedItems()) {
62             data += frame + " ";
63         }
64         label.setText(data);
65     }
66 });
67
68 }
69
70 // main method
71 public static void main(String args[])
72 {
73     new ListExample2();
74 }
75 }
```

Sample Output:



1. 11 AWT Choice

The object of Choice class is used to show popup menu of choices. Choice selected by user is shown on the top of a menu. It inherits Component class.

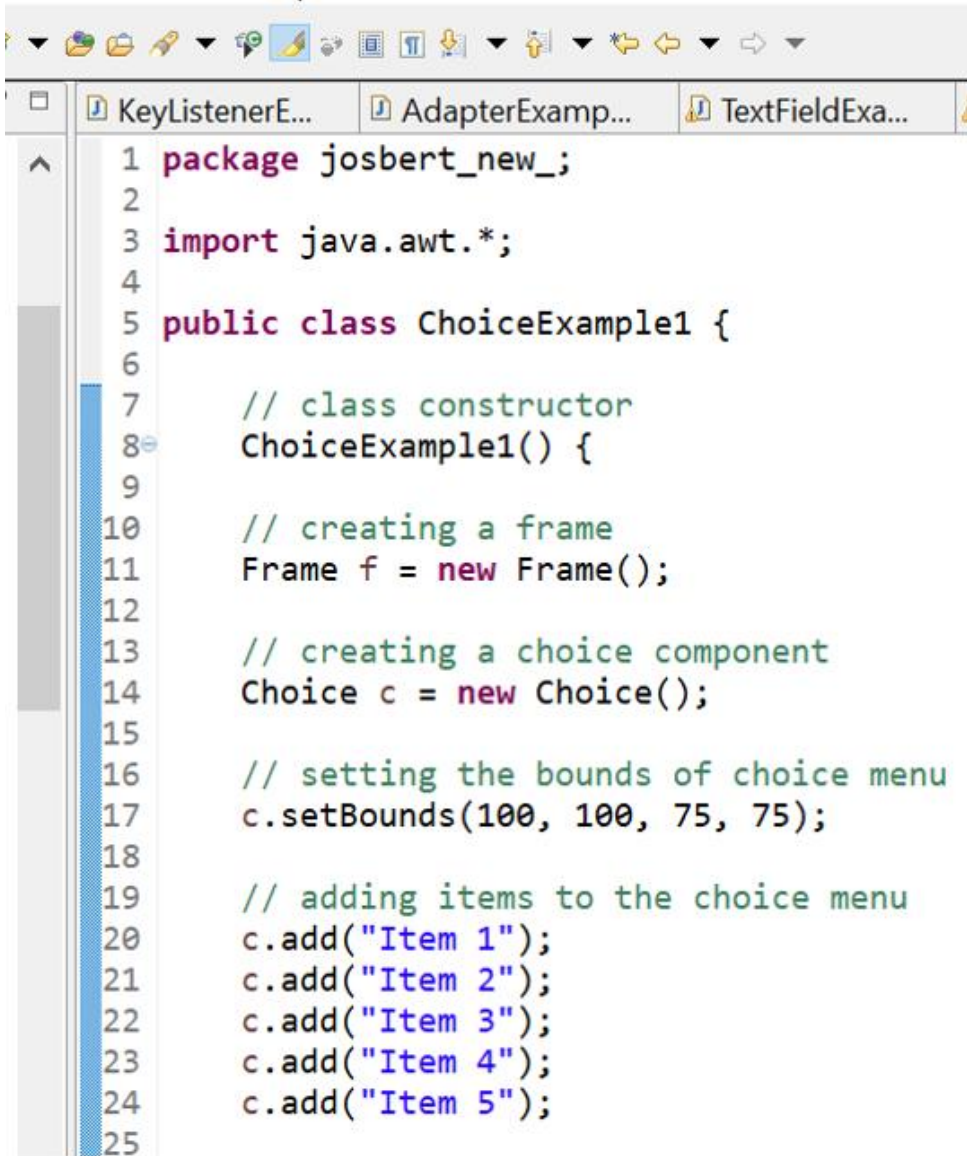
AWT Choice Example

In the following example, we are creating a choice menu using Choice() constructor. **Then we add 5 items to the menu using add() method and Then add the choice menu into the Frame.**

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

86

1. 11 AWT Choice



```
1 package josbert_new_;
2
3 import java.awt.*;
4
5 public class ChoiceExample1 {
6
7     // class constructor
8     ChoiceExample1() {
9
10        // creating a frame
11        Frame f = new Frame();
12
13        // creating a choice component
14        Choice c = new Choice();
15
16        // setting the bounds of choice menu
17        c.setBounds(100, 100, 75, 75);
18
19        // adding items to the choice menu
20        c.add("Item 1");
21        c.add("Item 2");
22        c.add("Item 3");
23        c.add("Item 4");
24        c.add("Item 5");
25    }
```

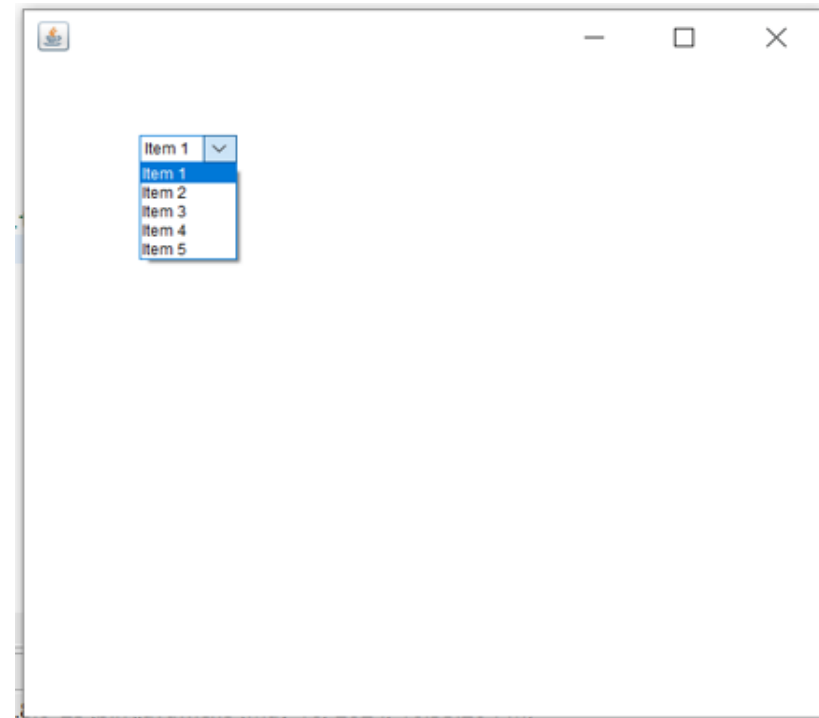
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

87

1. 11 AWT Choice

```
26 // adding choice menu to frame
27 f.add(c);
28
29 // setting size, layout and visibility of frame
30 f.setSize(400, 400);
31 f.setLayout(null);
32 f.setVisible(true);
33 }
34
35 //main method
36 public static void main(String args[])
37 {
38     new ChoiceExample1();
39 }
40
41 }
```

Sample Output:



1. 11 AWT Choice

AWT Choice Example with ActionListener

In the following example, we are creating a choice menu with 5 items. Along with that we are creating a button and a label. Here, we are adding an event to the button component using **`addActionListener(ActionListener a)`** method i.e. the selected item from the choice menu is displayed on the label when the button is clicked.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

89

1. 11 AWT Choice AWT Choice Example with ActionListener

```
1 package josbert_new_;
2 import java.awt.*;
3 import java.awt.event.*;
4 public class ChoiceExample2 {
5     // class constructor
6     ChoiceExample2() {
7
8         // creating a frame
9         Frame f = new Frame();
10
11         // creating a final object of Label class
12         final Label label = new Label();
13
14         // setting alignment and size of label component
15         label.setAlignment(Label.CENTER);
16         label.setSize(400, 100);
17
18         // creating a button
19         Button b = new Button("Show");
20
21         // setting the bounds of button
22         b.setBounds(200, 100, 50, 20);
23
24         // creating final object of Choice class
25         final Choice c = new Choice();
26
27         // setting bounds of choice menu
28         c.setBounds(100, 100, 75, 75);
29
30         // adding 5 items to choice menu
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

90

1. 11 AWT Choice AWT Choice Example with ActionListener

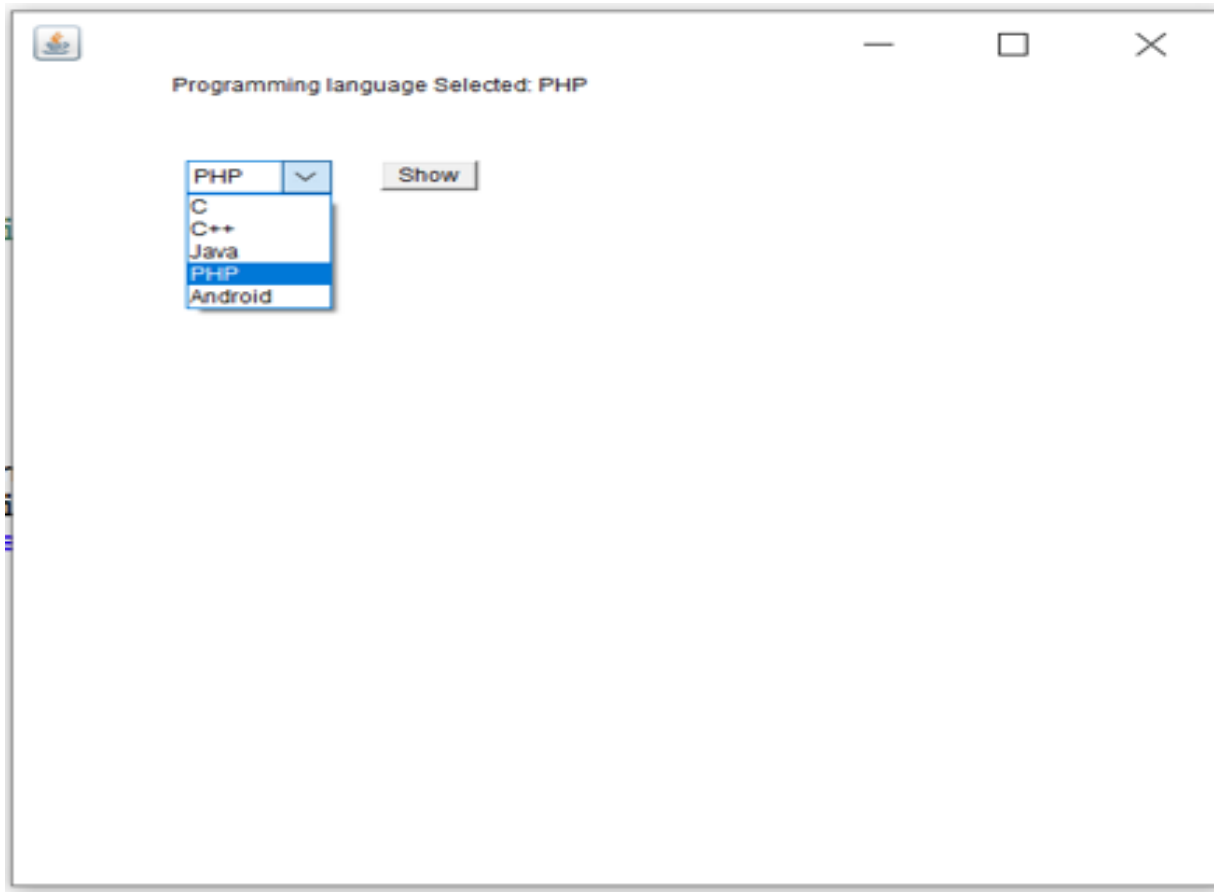
```
31     c.add("C");
32     c.add("C++");
33     c.add("Java");
34     c.add("PHP");
35     c.add("Android");
36
37     // adding above components into the frame
38     f.add(c);
39     f.add(label);
40     f.add(b);
41
42     // setting size, layout and visibility of frame
43     f.setSize(400, 400);
44     f.setLayout(null);
45     f.setVisible(true);
46
47     // adding event to the button
48     // which displays the selected item from the list when button is clicked
49     b.addActionListener(new ActionListener() {
50         public void actionPerformed(ActionEvent e) {
51             String data = "Programming language Selected: " + c.getItem(c.getSelectedIndex());
52             label.setText(data);
53         }
54     });
55 }
56 //main method
57 public static void main(String args[])
58 {
59     new ChoiceExample2();
60 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

91

1. 11 AWT Choice AWT Choice Example with ActionListener

Sample Output:



1. 11 AWT Choice AWT Choice Example with ActionListener

Another example

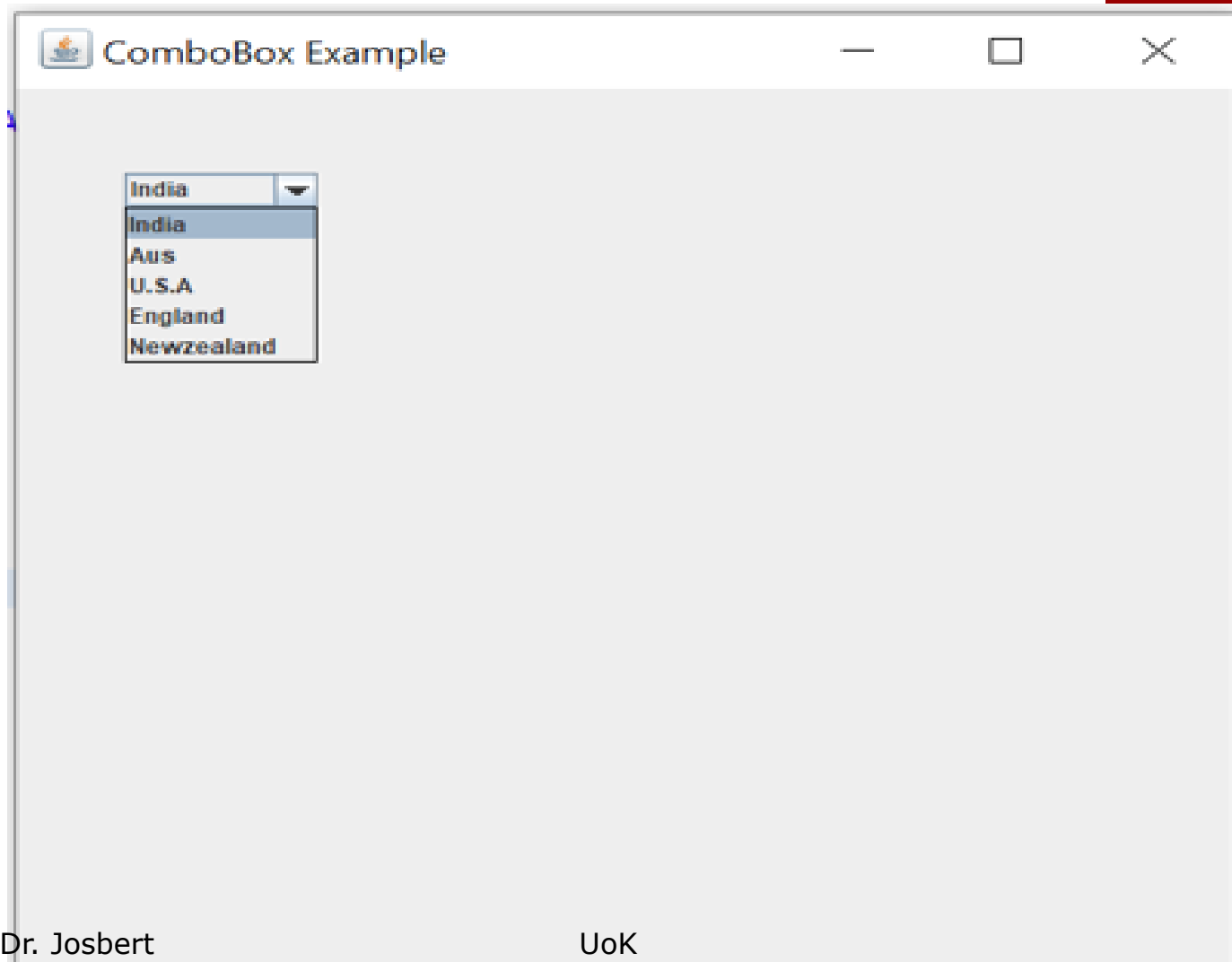
```
ChoiceExempl... ChoiceExempl... ButtonExempl... ButtonExempl... ButtonExempl...
1 package josbert_new_;
2
3
4 import javax.swing.*;
5 public class ComboBoxExample {
6
7     JFrame f;
8     ComboBoxExample(){
9         f=new JFrame("ComboBox Example");
10        String country[]={"India","Aus","U.S.A","England","Newzealand"};
11        JComboBox cb=new JComboBox(country);
12        cb.setBounds(50, 50,90,20);
13        f.add(cb);
14        f.setLayout(null);
15        f.setSize(400,500);
16        f.setVisible(true);
17    }
18    public static void main(String[] args) {
19        new ComboBoxExample();
20    }
21 }
22
```


UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

93

1. 11 AWT Choice AWT Choice Example with ActionListener

Sample Output:



1. 12 Layout Management

Java LayoutManagers

The LayoutManagers are used to arrange components in a particular manner. LayoutManager is an interface that is implemented by all the classes of layout managers.

BorderLayout

The BorderLayout provides five constants for each region:

1. **public static final int NORTH**
2. **public static final int SOUTH**
3. **public static final int EAST**
4. **public static final int WEST**
5. **public static final int CENTER**

1. 12 Layout Management

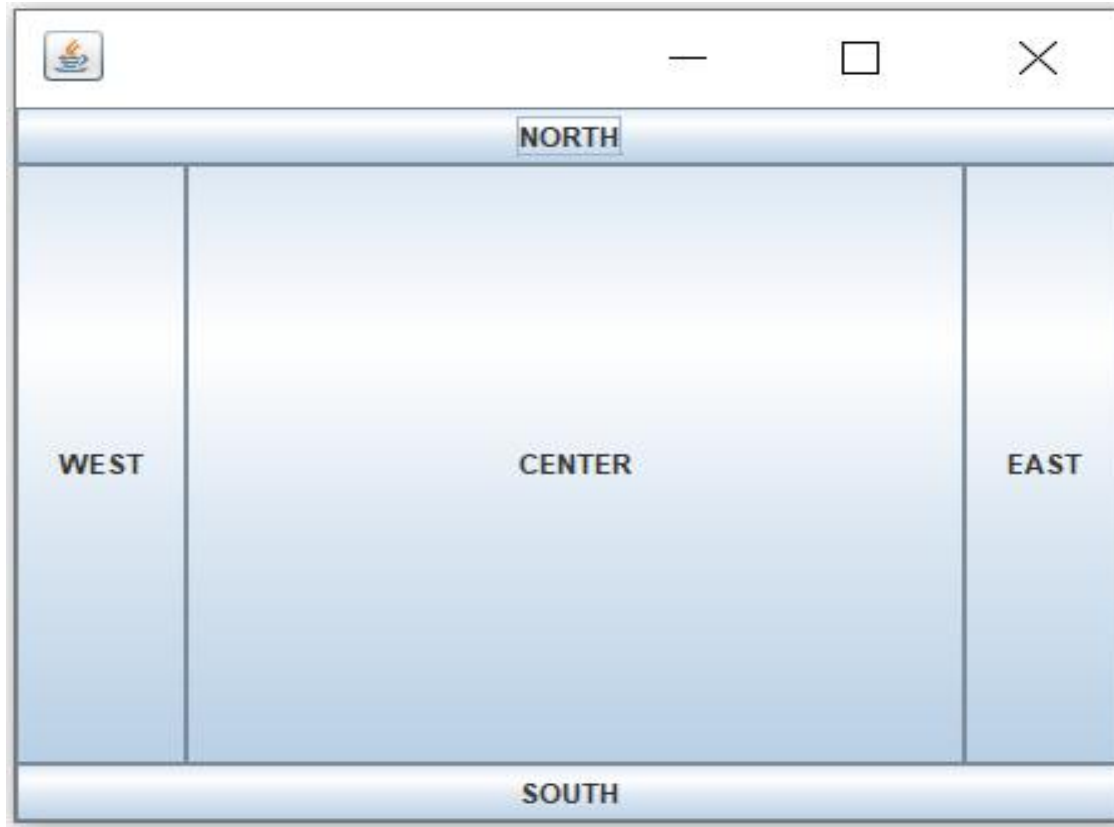
```
LabelExample... extFieldExam... TextAreaExam... Example.java Border.java Border_1.java 82
1 package josbert_new_;
2
3 import java.awt.*;
4 import javax.swing.*;
5 public class Border_1 {
6
7     JFrame f;
8     Border_1()
9     {
10         f = new JFrame();
11
12         // creating buttons
13         JButton b1 = new JButton("NORTH");; // the button will be labeled as NORTH
14         JButton b2 = new JButton("SOUTH");; // the button will be labeled as SOUTH
15         JButton b3 = new JButton("EAST");; // the button will be labeled as EAST
16         JButton b4 = new JButton("WEST");; // the button will be labeled as WEST
17         JButton b5 = new JButton("CENTER");; // the button will be labeled as CENTER
18
19         f.add(b1, BorderLayout.NORTH); // b1 will be placed in the North Direction
20         f.add(b2, BorderLayout.SOUTH); // b2 will be placed in the South Direction
21         f.add(b3, BorderLayout.EAST); // b2 will be placed in the East Direction
22         f.add(b4, BorderLayout.WEST); // b2 will be placed in the West Direction
23         f.add(b5, BorderLayout.CENTER); // b2 will be placed in the Center
24
25         f.setSize(300, 300);
26         f.setVisible(true);
27     }
28     public static void main(String[] args) {
29         new Border_1();
30     }
31 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

96

1. 12 Layout Management

Sample Output:



1. 12 Layout Management

The following example inserts horizontal and vertical **gaps** between buttons using the parameterized constructor `BorderLayout(int hgap, int gap)`.

1. 12 Layout Management

```
1 package josbert_new_;  
2 //import statement  
3 import java.awt.*;  
4 import javax.swing.*;  
5 public class BorderLayoutExample {  
6  
7     JFrame jframe;  
8     // constructor  
9     BorderLayoutExample()  
10    {  
11        // creating a Frame  
12        jframe = new JFrame();  
13        // create buttons  
14        JButton btn1 = new JButton("NORTH");  
15        JButton btn2 = new JButton("SOUTH");  
16        JButton btn3 = new JButton("EAST");  
17        JButton btn4 = new JButton("WEST");  
18        JButton btn5 = new JButton("CENTER");  
19        // creating an object of the BorderLayout class using  
20        // the parameterized constructor where the horizontal gap is 20  
21        // and vertical gap is 15. The gap will be evident when buttons are placed  
22        // in the frame  
23        jframe.setLayout(new BorderLayout(20, 15));  
24        jframe.add(btn1, BorderLayout.NORTH);  
25        jframe.add(btn2, BorderLayout.SOUTH);  
26        jframe.add(btn3, BorderLayout.EAST);  
27        jframe.add(btn4, BorderLayout.WEST);  
28        jframe.add(btn5, BorderLayout.CENTER);  
29        jframe.setSize(300,300);  
30        jframe.setVisible(true);  
31    }  
32    // main method  
33    public static void main(String argsv[])  
34    {  
35        new BorderLayoutExample();  
36    }  
37  
38 }
```

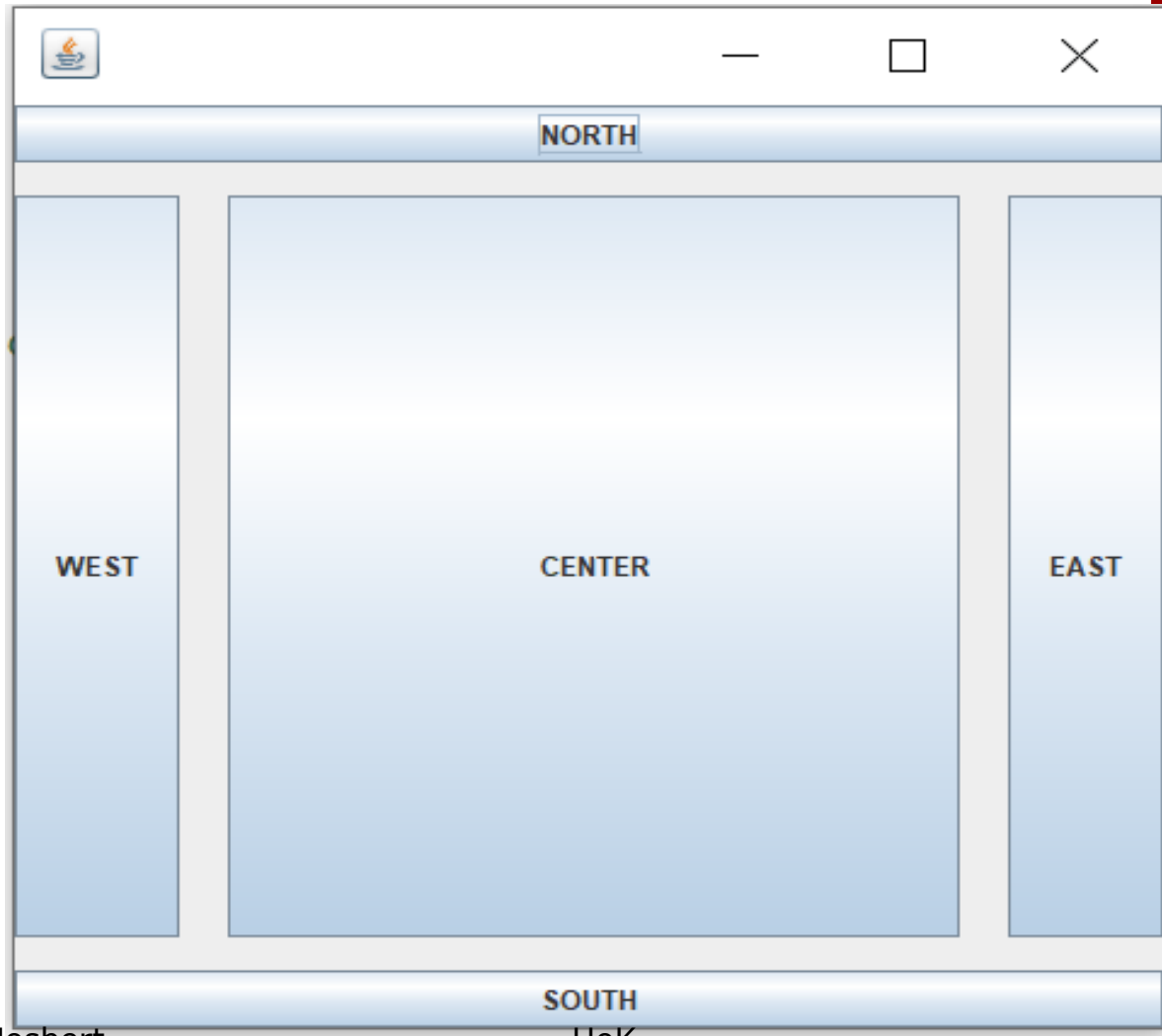
Fall-2026-Dr. Josbert

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

99

1. 12 Layout Management

Sample Output:



TASK

10
0

HOSPITAL PATIENT BILLING FORM

Patient Name:

Base Treatment Cost:

Ward Charge (15%):

Lab & Pharmacy (5%):

Gross Bill:

Insurance (5%):

VAT:

Other Deductions:

Net Payable:

Calculate

Clear

Exit

1. 12 Java GridBagLayout

The Java GridLayout class is used to arrange the components in a rectangular grid. One component is displayed in each rectangle.

Constructors of GridLayout class

1. **GridLayout()**: creates a grid layout with one column per component in a row.
2. **GridLayout(int rows, int columns)**: creates a grid layout with the given rows and columns but **no gaps** between the components.
3. **GridLayout(int rows, int columns, int hgap, int vgap)**: creates a grid layout with the given rows and columns along with given **horizontal and vertical gaps**.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

10
2

1. 12 Java GridBagLayout

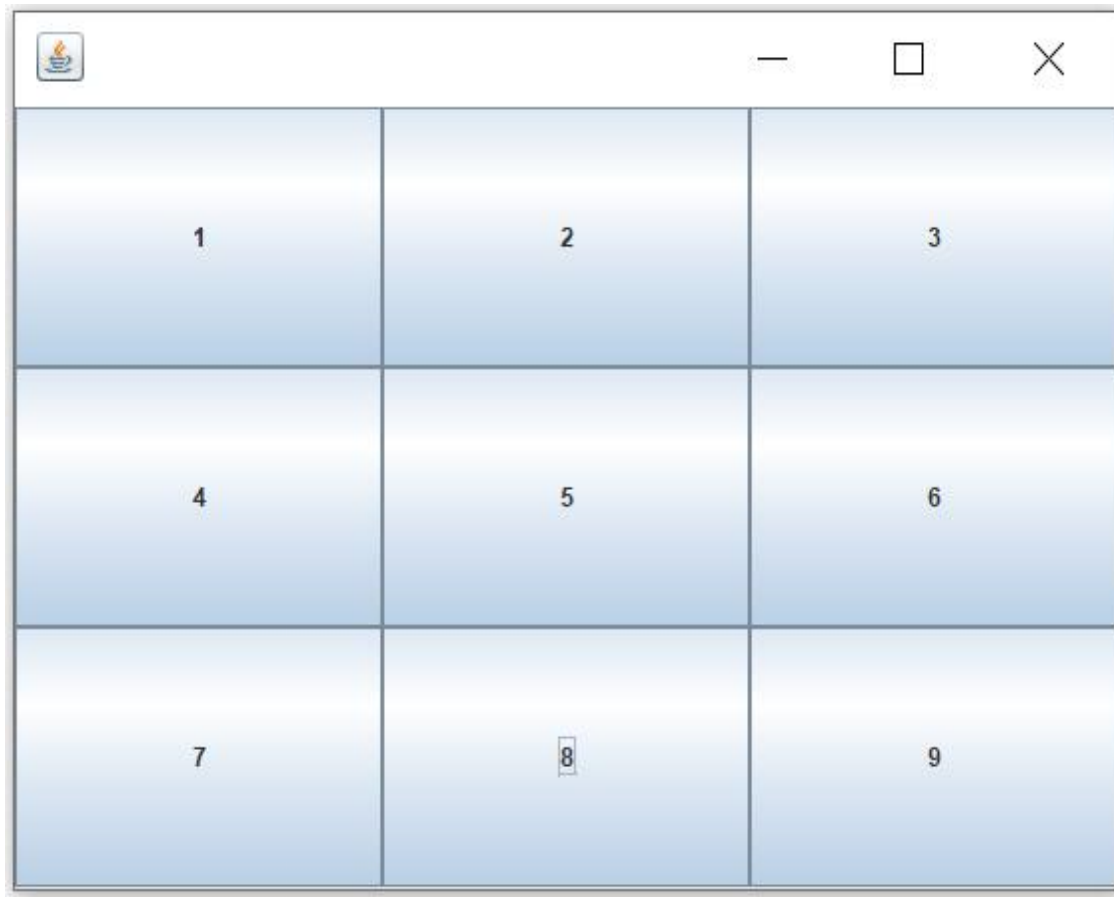
Example of GridLayout class: Using GridLayout(int rows, int columns) Constructor

```
Border.java  Border_1.java  BorderLayout...  MyGridLayout...
1 package josbert_new_;
2 import java.awt.*;
3 import javax.swing.*;
4 public class MyGridLayout_1 {
5     JFrame f;
6     MyGridLayout_1(){
7         f=new JFrame();
8         JButton b1=new JButton("1");
9         JButton b2=new JButton("2");
10        JButton b3=new JButton("3");
11        JButton b4=new JButton("4");
12        JButton b5=new JButton("5");
13        JButton b6=new JButton("6");
14        JButton b7=new JButton("7");
15        JButton b8=new JButton("8");
16        JButton b9=new JButton("9");
17        // adding buttons to the frame
18        f.add(b1); f.add(b2); f.add(b3);
19        f.add(b4); f.add(b5); f.add(b6);
20        f.add(b7); f.add(b8); f.add(b9);
21
22        // setting grid layout of 3 rows and 3 columns
23        f.setLayout(new GridLayout(3,3));
24        f.setSize(300,300);
25        f.setVisible(true);
26    }
27    public static void main(String[] args) {
28        new MyGridLayout_1();
29    }
30
31 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

1. 12 Java GridBagLayout

Sample Output:



10
3

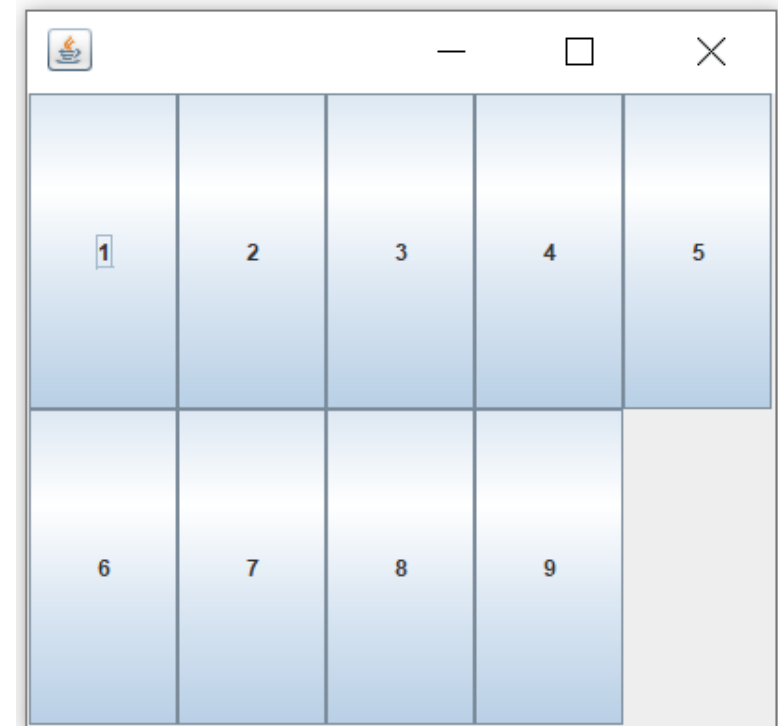
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

1. 12 Java GridBagLayout

10
4

Sample Output:

```
1 package josbert_new_;
2 import java.awt.*;
3 import javax.swing.*;
4 public class MyGridLayout_1 {
5     JFrame f;
6     MyGridLayout_1(){
7         f=new JFrame();
8         JButton b1=new JButton("1");
9         JButton b2=new JButton("2");
10        JButton b3=new JButton("3");
11        JButton b4=new JButton("4");
12        JButton b5=new JButton("5");
13        JButton b6=new JButton("6");
14        JButton b7=new JButton("7");
15        JButton b8=new JButton("8");
16        JButton b9=new JButton("9");
17        // adding buttons to the frame
18        f.add(b1); f.add(b2); f.add(b3);
19        f.add(b4); f.add(b5); f.add(b6);
20        f.add(b7); f.add(b8); f.add(b9);
21
22        // setting grid layout of 2 rows and 4 columns
23        f.setLayout(new GridLayout(2,4));
24        f.setSize(300,300);
25        f.setVisible(true);
26    }
27    public static void main(String[] args) {
28        new MyGridLayout_1();
29    }
30
31 }
32
```



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

10
5

1. 12 Java GridBagLayout

Example of GridLayout class: Using GridLayout(int rows, int columns, int hgap, int vgap) Constructor

The following example inserts horizontal and vertical **gaps** between buttons using the parameterized constructor GridLayout(int rows, int columns, int hgap, int vgap).

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

10
6

1. 12 Java GridBagLayout

Example of GridLayout class: Using GridLayout(int rows, int columns, int hgap, int vgap) Constructor

```
Border_1.java  BorderLayout...  MyGridLayout...  GridLayoutEx...  MyGridLayout...

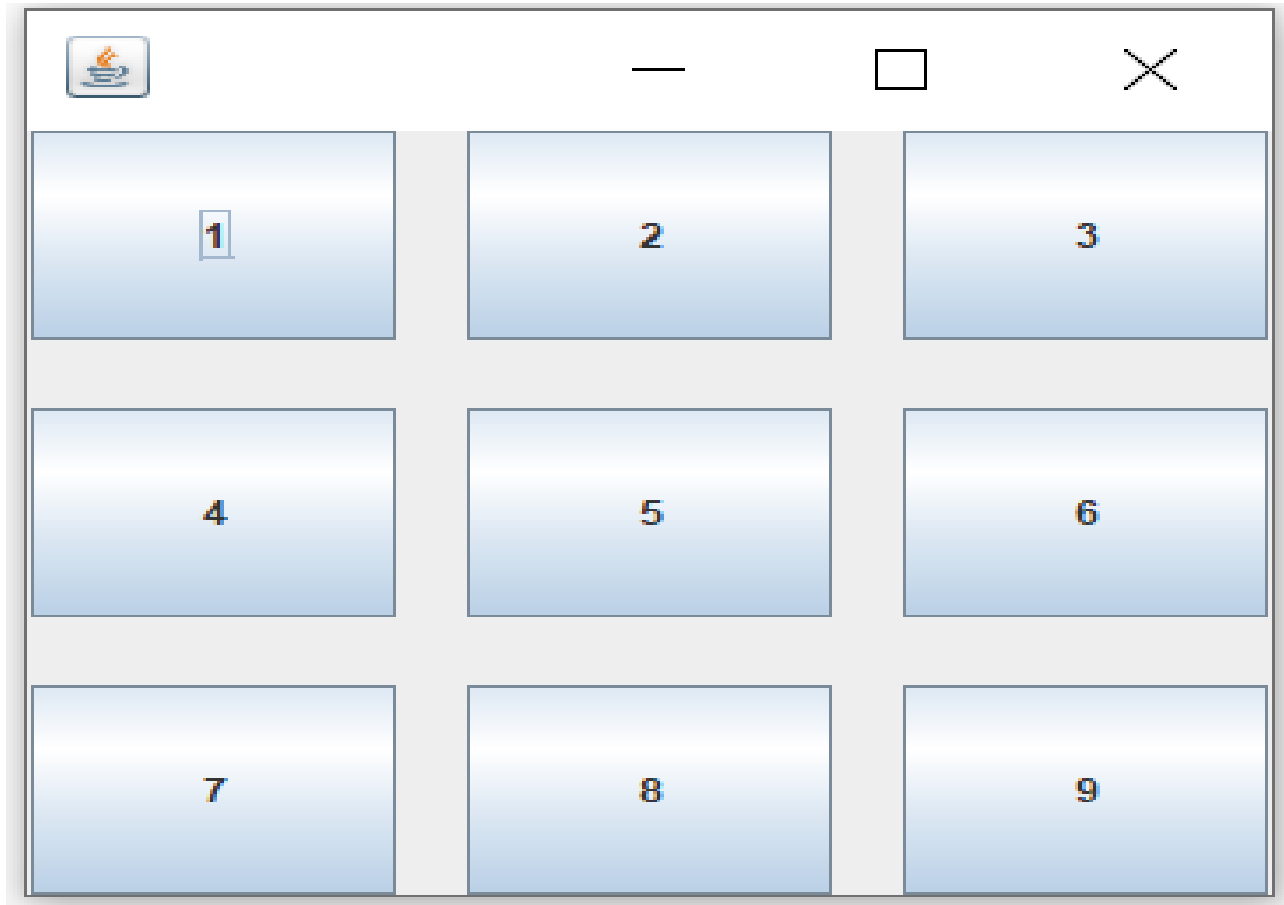
1 package josbert_new_;
2
3 import java.awt.*;
4 import javax.swing.*;
5
6 public class GridLayoutExample1 {
7     Frame frameObj;
8
9     // constructor
10    GridLayoutExample1()
11    {
12        frameObj = new JFrame();
13
14        // creating 9 buttons
15        JButton btn1 = new JButton("1");
16        JButton btn2 = new JButton("2");
17        JButton btn3 = new JButton("3");
18        JButton btn4 = new JButton("4");
19        JButton btn5 = new JButton("5");
20        JButton btn6 = new JButton("6");
21        JButton btn7 = new JButton("7");
22        JButton btn8 = new JButton("8");
23        JButton btn9 = new JButton("9");
24
25        // adding buttons to the frame
26        // since, we are using the parameterless constructor, therefore;
27        // the number of columns is equal to the number of buttons we
28        // are adding to the frame. The row count remains one.
29
30        frameObj.add(btn1); frameObj.add(btn2); frameObj.add(btn3);
31        frameObj.add(btn4); frameObj.add(btn5); frameObj.add(btn6);
32        frameObj.add(btn7); frameObj.add(btn8); frameObj.add(btn9);
33        // setting the grid layout
34        // a 3 * 3 grid is created with the horizontal gap 20
35        // and vertical gap 25
36        frameObj.setLayout(new GridLayout(3, 3, 20, 25));
37        frameObj.setSize(300, 300);
38        frameObj.setVisible(true);
39    }
40    // main method
41    public static void main(String argsv[])
42    {
43        new GridLayoutExample1();
44    }
45
46 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

10
7

1. 12 Java GridBagLayout

Sample Output:



1. 13 Java FlowLayout

The Java FlowLayout class is used to arrange the components in a line, one after another (in a flow). It is the default layout of the applet or panel.

Fields of FlowLayout class

1. **public static final int LEFT**
2. **public static final int RIGHT**
3. **public static final int CENTER**
4. **public static final int LEADING**
5. **public static final int TRAILING**

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

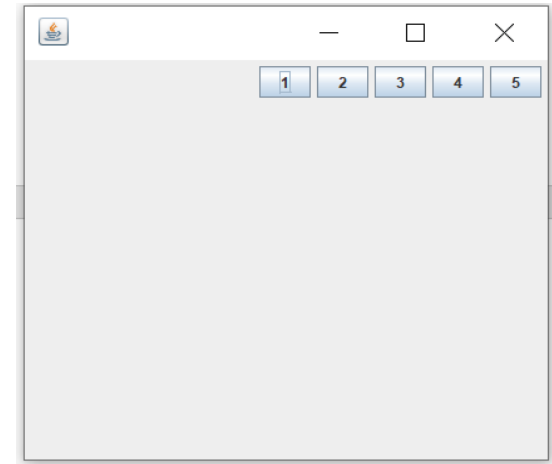
10
9

1. 13 Java FlowLayout

Example of FlowLayout class: Using FlowLayout(int align) constructor

```
1 package josbert_new_;
2
3 import java.awt.*;
4 import javax.swing.*;
5
6 public class MyFlowLayout {
7
8     JFrame f;
9     MyFlowLayout(){
10         f=new JFrame();
11
12         JButton b1=new JButton("1");
13         JButton b2=new JButton("2");
14         JButton b3=new JButton("3");
15         JButton b4=new JButton("4");
16         JButton b5=new JButton("5");
17
18         // adding buttons to the frame
19         f.add(b1); f.add(b2); f.add(b3); f.add(b4); f.add(b5);
20
21         // setting flow layout of right alignment
22         f.setLayout(new FlowLayout(FlowLayout. RIGHT));
23         // Test another like LEFT
24
25         f.setSize(300,300);
26         f.setVisible(true);
27     }
28     public static void main(String[] args) {
29         new MyFlowLayout();
30     }
31 }
```

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

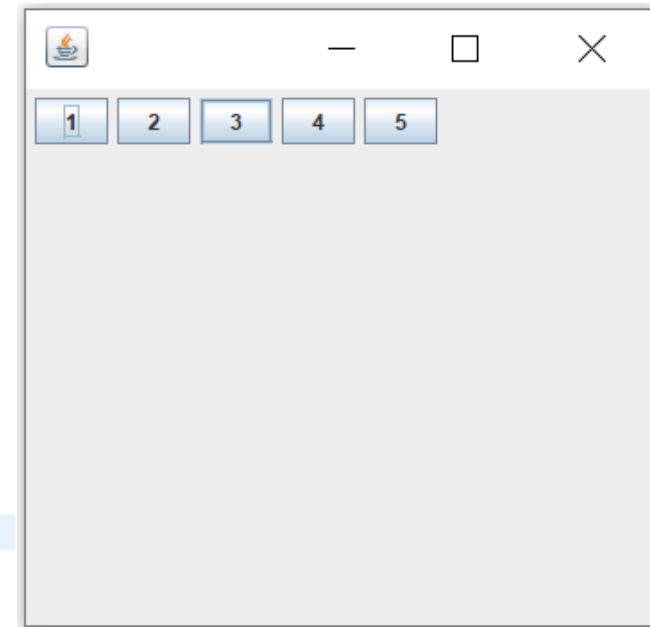
11
0

1. 13 Java FlowLayout

Let's check the left side

```
1 package josbert_new_;
2 import java.awt.*;
3 import javax.swing.*;
4
5 public class MyFlowLayout_n {
6
7     JFrame f;
8     MyFlowLayout_n(){
9         f=new JFrame();
10
11         JButton b1=new JButton("1");
12         JButton b2=new JButton("2");
13         JButton b3=new JButton("3");
14         JButton b4=new JButton("4");
15         JButton b5=new JButton("5");
16
17         // adding buttons to the frame
18         f.add(b1); f.add(b2); f.add(b3); f.add(b4); f.add(b5);
19
20         // setting flow layout of right alignment
21         f.setLayout(new FlowLayout(FlowLayout.LEFT));
22         // Test another like LEFT
23
24         f.setSize(300,300);
25         f.setVisible(true);
26     }
27     public static void main(String[] args) {
28         new MyFlowLayout_n();
29     }
30 }
```

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

11
1

1. 13 Java FlowLayout

Example of FlowLayout class: Using FlowLayout(int align, int hgap, int vgap) constructor:

```
1 package josbert_new_;
2
3 //import statement
4 import java.awt.*;
5 import javax.swing.*;
6
7 public class FlowLayoutExample1 {
8
9     JFrame frameObj;
10
11     // constructor
12     FlowLayoutExample1()
13     {
14         // creating a frame object
15         frameObj = new JFrame();
16
17         // creating the buttons
18         JButton b1 = new JButton("1");
19         JButton b2 = new JButton("2");
20         JButton b3 = new JButton("3");
21         JButton b4 = new JButton("4");
22         JButton b5 = new JButton("5");
23         JButton b6 = new JButton("6");
24         JButton b7 = new JButton("7");
25         JButton b8 = new JButton("8");
26         JButton b9 = new JButton("9");
27         JButton b10 = new JButton("10");
28
```

1. 13 Java FlowLayout

```
28
29
30     // adding the buttons to frame
31     frameObj.add(b1); frameObj.add(b2); frameObj.add(b3); frameObj.add(b4);
32     frameObj.add(b5); frameObj.add(b6); frameObj.add(b7); frameObj.add(b8);
33     frameObj.add(b9); frameObj.add(b10);
34
35     // parameterized constructor is used
36     // where alignment is left
37     // horizontal gap is 20 units and vertical gap is 25 units.
38     frameObj.setLayout(new FlowLayout(FlowLayout.LEFT, 20, 25));
39
40
41     frameObj.setSize(300, 300);
42     frameObj.setVisible(true);
43 }
44 // main method
45 public static void main(String args[])
46 {
47     new FlowLayoutExample1();
48 }
49
50 }
```

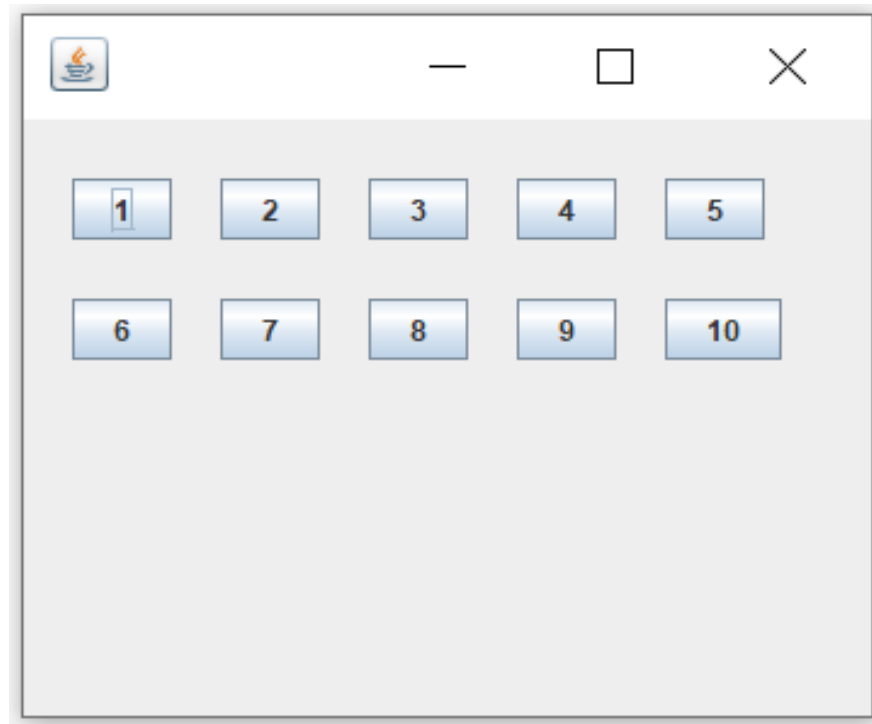
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

11
3

1. 13 Java FlowLayout

Example of FlowLayout class: Using FlowLayout(int align, int hgap, int vgap) constructor:

Sample Output:



1. 14 Java KeyListener Interface

The Java KeyListener is notified whenever you change the state of key. It is notified against KeyEvent. The KeyListener interface is found in java.awt.event package. It has three methods.

Methods of KeyListener interface

The signature of 3 methods found in KeyListener interface are given below:

1. `public abstract void keyPressed(KeyEvent e);`
2. `public abstract void keyReleased(KeyEvent e);`
3. `public abstract void keyTyped(KeyEvent e);`

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

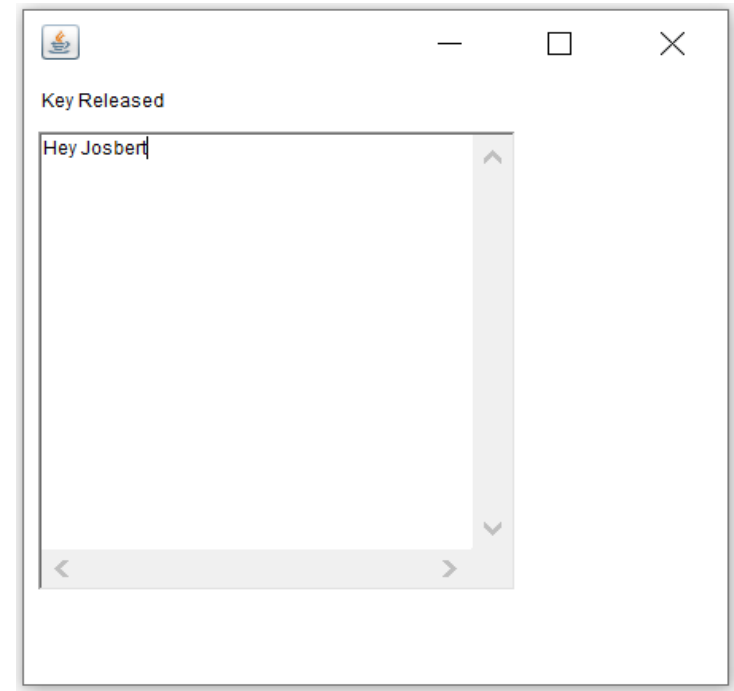
11
5

1. 14 Java KeyListener Interface

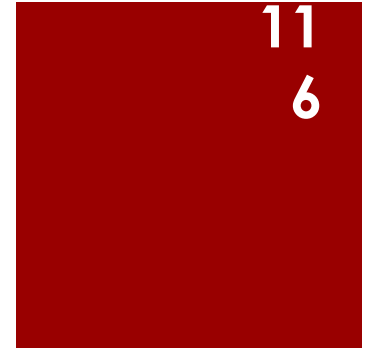
Java KeyListener Example

```
GridLayoutEx...  FlowLayoutEx...  MyFlowLayout...  FlowLayoutEx...  AEvent.java  KeyListenerE...
1 package josbert_new_src.josbert_new_GridLayoutExample1.java
2
3 import java.awt.*;
4 import java.awt.event.*;
5 public class KeyListenerExample extends Frame implements KeyListener {
6
7     Label l;
8     TextArea area;
9     KeyListenerExample(){
10         l=new Label();
11         l.setBounds(20,50,100,20);
12         area=new TextArea();
13         area.setBounds(20,80,300, 300);
14         area.addKeyListener(this);
15         add(l);add(area);
16         setSize(400,400);
17         setLayout(null);
18         setVisible(true);
19     }
20     public void keyPressed(KeyEvent e) {
21         l.setText("Key Pressed");
22     }
23     public void keyReleased(KeyEvent e) {
24         l.setText("Key Released");
25     }
26     public void keyTyped(KeyEvent e) {
27         l.setText("Key Typed");
28     }
29     public static void main(String[] args) {
30         new KeyListenerExample(); }
31
32 }
```

Sample Output:



1. 15 Java JRadioButton



The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. **It is widely used in exam systems or quiz.**

It should be added in ButtonGroup to select one radio button only.

Java JRadioButton Example

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

11
7

1. 15 Java JRadioButton

```
package josbert_new_;

import javax.swing.*;

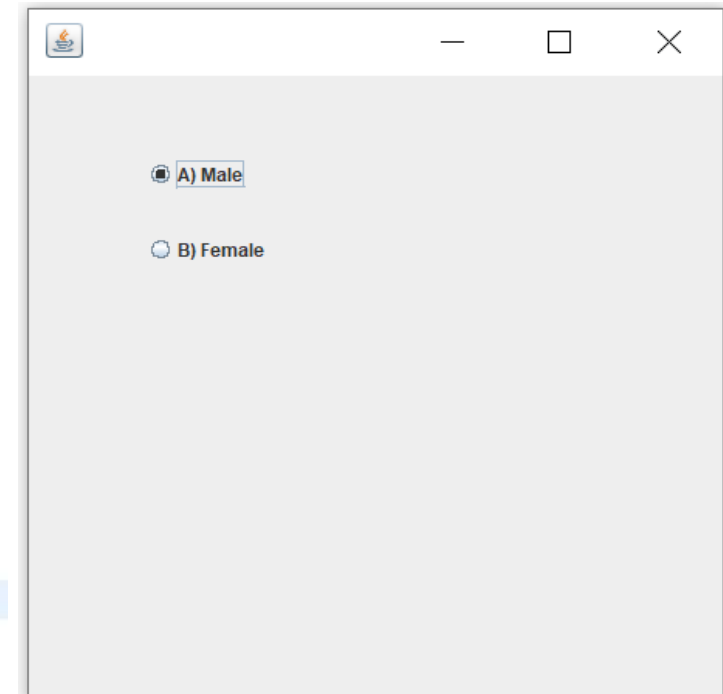
public class RadioButtonExample {

    JFrame f;

    RadioButtonExample(){
        f=new JFrame();
        JRadioButton r1=new JRadioButton("A) Male");
        JRadioButton r2=new JRadioButton("B) Female");
        r1.setBounds(75,50,100,30);
        r2.setBounds(75,100,100,30);
        ButtonGroup bg=new ButtonGroup();
        bg.add(r1);bg.add(r2);
        f.add(r1);f.add(r2);
        f.setSize(300,300);
        f.setLayout(null);
        f.setVisible(true);
    }

    public static void main(String[] args) {
        new RadioButtonExample();
    }
}
```

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

11
8

1. 15 Java JRadioButton

Lets add more comments to this java program

```
1 package josbert_new_;
2 import javax.swing.*; // Importing the Swing library for
3 //creating graphical user interfaces
4 public class RadioButtonExample_new {
5
6     // Declare a JFrame variable to hold the main window
7     JFrame f;
8
9     // Constructor for the RadioButtonExample class
10    RadioButtonExample_new() {
11        // Initialize the JFrame
12        f = new JFrame();
13
14        // Create two radio buttons for selecting gender
15        JRadioButton r1 = new JRadioButton("A) Male"); // Radio button for Male option
16        JRadioButton r2 = new JRadioButton("B) Female"); // Radio button for Female option
17
18        // Set the position and size of the first radio button
19        r1.setBounds(75, 50, 100, 30); // x, y, width, height
20
21        // Set the position and size of the second radio button
22        r2.setBounds(75, 100, 100, 30); // x, y, width, height
23
24        // Create a ButtonGroup to group the radio buttons together
25        ButtonGroup bg = new ButtonGroup();
26
27        // Add the radio buttons to the ButtonGroup to ensure only one can be selected at a time
28        bg.add(r1);
29        bg.add(r2);
30    }
```

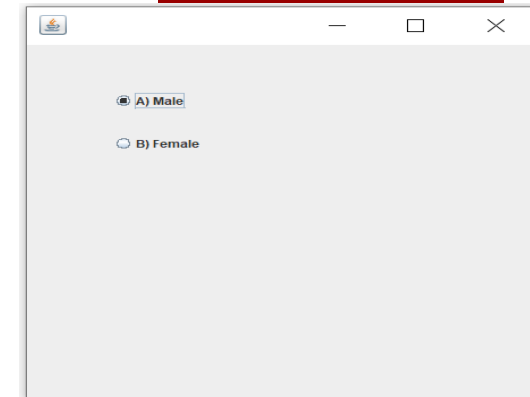
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

11
9

1. 15 Java JRadioButton

Lets add more comments to this java program

```
31      // Add the radio buttons to the JFrame
32      f.add(r1);
33      f.add(r2);
34
35      // Set the size of the JFrame (width, height)
36      f.setSize(300, 300);
37
38      // Set the layout manager to null for absolute positioning
39      f.setLayout(null);
40
41      // Make the frame visible
42      f.setVisible(true);
43  }
44
45  // Main method to run the program
46  public static void main(String[] args) {
47      new RadioButtonExample_new(); // Create an instance of
48      //RadioButtonExample to display the GUI
49  }
50
51 }
```



1. 16 Java JTree

The JTree class is used to display the tree structured data or hierarchical data. JTree is a complex component. It has a root node at the top most which is a parent for all nodes in the tree. It inherits JComponent class.

JTree class declaration

Let's see the declaration for javax.swing.JTree class.

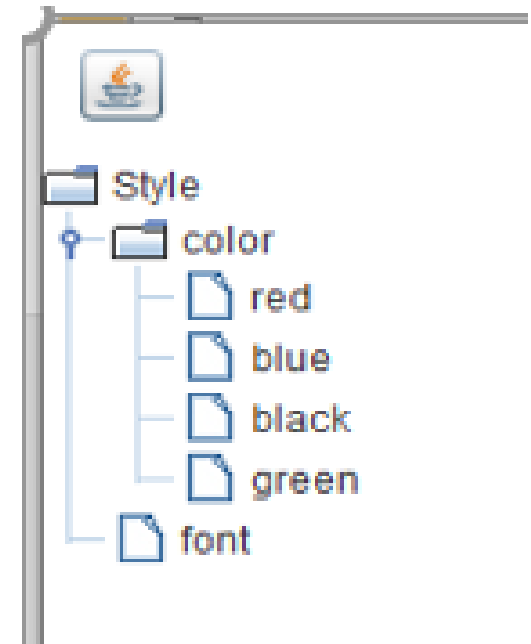
```
1. public class JTree extends JComponent implements Scrollable, Accessible
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
1

1. 16 Java JTree Java JTree Example

```
1 package josbert_new_;
2 import javax.swing.*;
3 import javax.swing.tree.DefaultMutableTreeNode;
4 public class TreeExample_1 {
5
6     JFrame f;
7     TreeExample_1(){
8         f=new JFrame();
9         DefaultMutableTreeNode style=new DefaultMutableTreeNode("Style");
10        DefaultMutableTreeNode color=new DefaultMutableTreeNode("color");
11        DefaultMutableTreeNode font=new DefaultMutableTreeNode("font");
12        style.add(color); //add color in style
13        style.add(font); // add font in style
14
15        // create colors
16        DefaultMutableTreeNode red=new DefaultMutableTreeNode("red");
17        DefaultMutableTreeNode blue=new DefaultMutableTreeNode("blue");
18        DefaultMutableTreeNode black=new DefaultMutableTreeNode("black");
19        DefaultMutableTreeNode green=new DefaultMutableTreeNode("green");
20        color.add(red); // add red in color
21        color.add(blue); // add blue in color
22        color.add(black); // add black in color
23        color.add(green); // add green in color
24        JTree jt=new JTree(style);
25        f.add(jt);
26        f.setSize(200,200);
27        f.setVisible(true);
28    }
29    public static void main(String[] args) {
30        new TreeExample_1();
31    }
32 }
```



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
2

1. 16 Java JTree Java JTree Example

Lets add more comments to this java program

```
1 package josbert_new_;
2
3 import javax.swing.*; // Import the Swing library for creating GUI components
4 import javax.swing.tree.DefaultMutableTreeNode; // Import the class for creating tree nodes
5 public class TreeExample_2 {
6
7     // Declare a JFrame object to serve as the main window
8     JFrame f;
9
10    // Constructor for TreeExample_1
11    TreeExample_2() {
12        // Initialize the JFrame
13        f = new JFrame();
14        // Create the root node for the tree with the label "Style"
15        DefaultMutableTreeNode style = new DefaultMutableTreeNode("Style");
16
17        // Create child nodes for the "Style" node
18        DefaultMutableTreeNode color = new DefaultMutableTreeNode("Color");
19        DefaultMutableTreeNode font = new DefaultMutableTreeNode("Font");
20
21        // Add the "Color" and "Font" nodes to the "Style" node
22        style.add(color); // Add the "Color" node under "Style"
23        style.add(font); // Add the "Font" node under "Style"
24
25        // Create nodes for each color
26        DefaultMutableTreeNode red = new DefaultMutableTreeNode("Red");
27        DefaultMutableTreeNode blue = new DefaultMutableTreeNode("Blue");
28        DefaultMutableTreeNode black = new DefaultMutableTreeNode("Black");
29        DefaultMutableTreeNode green = new DefaultMutableTreeNode("Green");
30
31        // Add each color node to the "Color" node
32        color.add(red); // Add the "Red" node under "Color"
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
3

1. 16 Java JTree Java JTree Example

```
33 color.add(blue); // Add the "Blue" node under "Color"
34 color.add(black); // Add the "Black" node under "Color"
35 color.add(green); // Add the "Green" node under "Color"
```

```
36
37 // Create a JTree with the "Style" node as the root
38 JTree jt = new JTree(style);
```

```
39
40 // Add the JTree to the JFrame
41 f.add(jt);
```

```
42
43 // Set the size of the JFrame
44 f.setSize(200, 200);
```

```
45
46 // Set the default close operation for the JFrame
47 f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
48
49 // Make the JFrame visible
50 f.setVisible(true);
```

```
51 }
```

```
52
53 // Main method to run the program
```

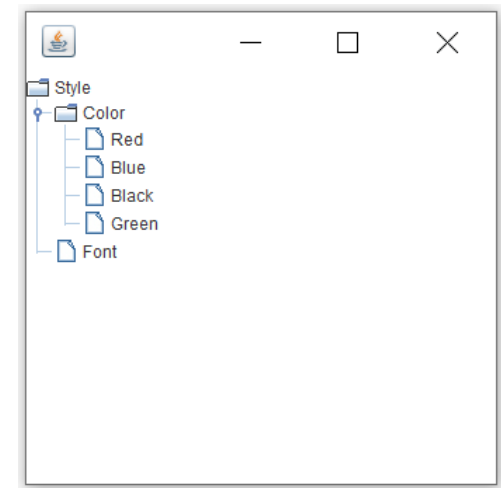
```
54 public static void main(String[] args) {
```

```
55     new TreeExample_2(); // Create an instance of TreeExample_1 to display the GUI
```

```
56 }
```

```
57 }
```

Output:



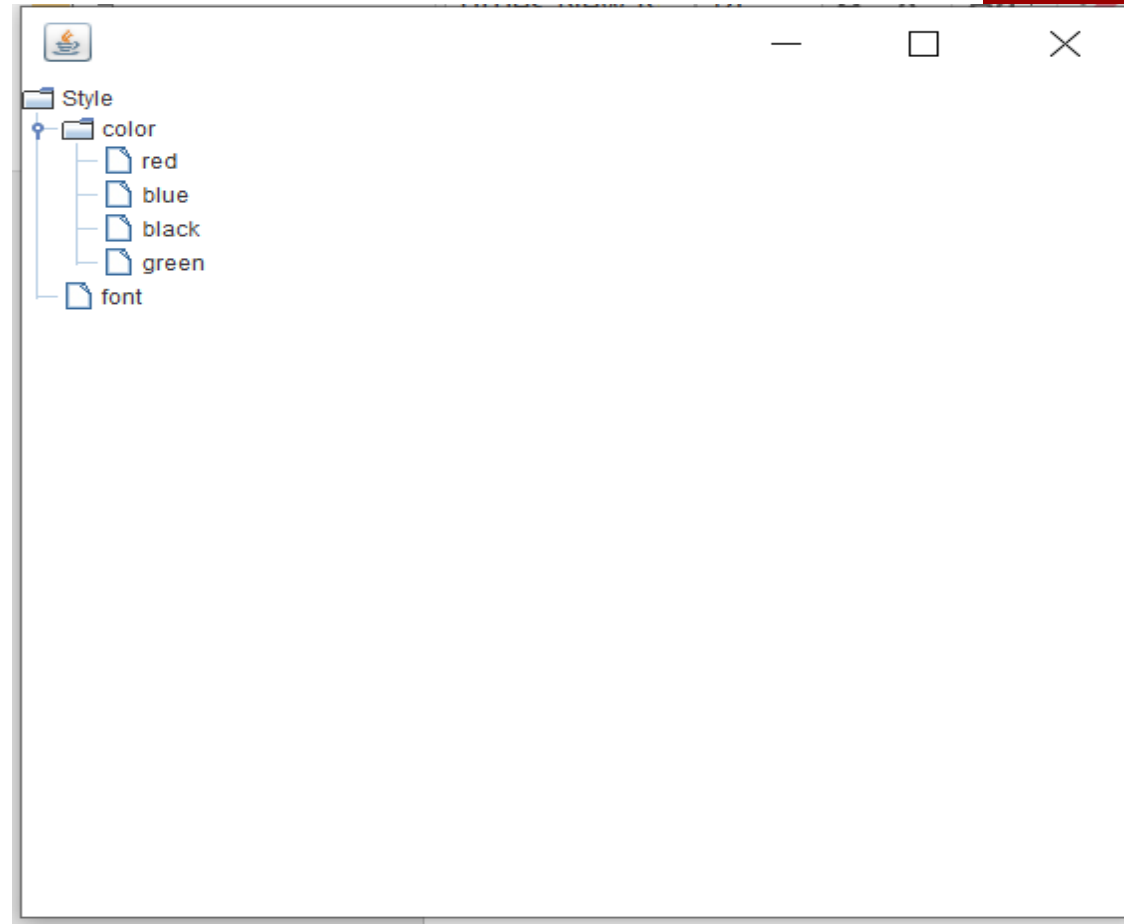
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
4

1. 16 Java JTree

Java JTree Example

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
5

Exercises:

1. Design Menu File Using JAVA GUI, Menu File is composed of New, Open, Save and Exit

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
6

Solution:

```
*FileMenuExa...  BasicMenuExa...  SubmenuExamp...  MenuActionEx...  
1 package josbert_new_;  
2 import javax.swing.*; import java.awt.event.*;  
4 public class FileMenuExample {  
5  
6     public static void main(String[] args) {  
7         JFrame frame = new JFrame("File Menu Example");  
8         JMenuBar menuBar = new JMenuBar();  
9  
10        //create Menus  
11        JMenu fileMenu = new JMenu("File");  
12  
13        JMenuItem newItem = new JMenuItem("New");  
14        JMenuItem openMenuItem = new JMenuItem("Open");  
15        JMenuItem saveMenuItem = new JMenuItem("Save");  
16        JMenuItem exitMenuItem = new JMenuItem("Exit");  
17  
18        //add Menus to Menu File  
19        fileMenu.add(newMenuItem);  
20        fileMenu.add(openMenuItem);  
21        fileMenu.add(saveMenuItem);  
22        fileMenu.addSeparator();  
23        fileMenu.add(exitMenuItem);  
24  
25        menuBar.add(fileMenu);  
26        frame.setJMenuBar(menuBar);  
27  
28        exitMenuItem.addActionListener(new ActionListener() {  
29            @Override  
30            public void actionPerformed(ActionEvent e) {  
31                System.exit(0);  
32            }  
33        });  
35        frame.setSize(300, 200);  
36        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
37        frame.setVisible(true);  
38    }  
39  
40 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
7

Solution:

Lets add more comments to this java program

```
1 package josbert.new;
2 import javax.swing.*; // Import the Swing library for creating GUI components
3 import java.awt.event.*; // Import classes for handling events
4
5 public class FileMenuExample_2 {
6
7     // Main method: entry point of the program
8     public static void main(String[] args) {
9         // Create a JFrame to serve as the main window of the application
10        JFrame frame = new JFrame("File Menu Example");
11
12        // Create a JMenuBar to hold the menu
13        JMenuBar menuBar = new JMenuBar();
14
15        // Create the "File" menu
16        JMenu fileMenu = new JMenu("File");
17
18        // Create menu items for the "File" menu
19        JMenuItem newItem = new JMenuItem("New"); // Menu item for creating a new file
20        JMenuItem openMenuItem = new JMenuItem("Open"); // Menu item for opening an existing file
21        JMenuItem saveMenuItem = new JMenuItem("Save"); // Menu item for saving the current file
22        JMenuItem exitMenuItem = new JMenuItem("Exit"); // Menu item for exiting the application
23
24        // Add menu items to the "File" menu
25        fileMenu.add(newMenuItem); // Add "New" option to the menu
26        fileMenu.add(openMenuItem); // Add "Open" option to the menu
27        fileMenu.add(saveMenuItem); // Add "Save" option to the menu
28        fileMenu.addSeparator(); // Add a separator for visual separation
29        fileMenu.add(exitMenuItem); // Add "Exit" option to the menu
30    }
```

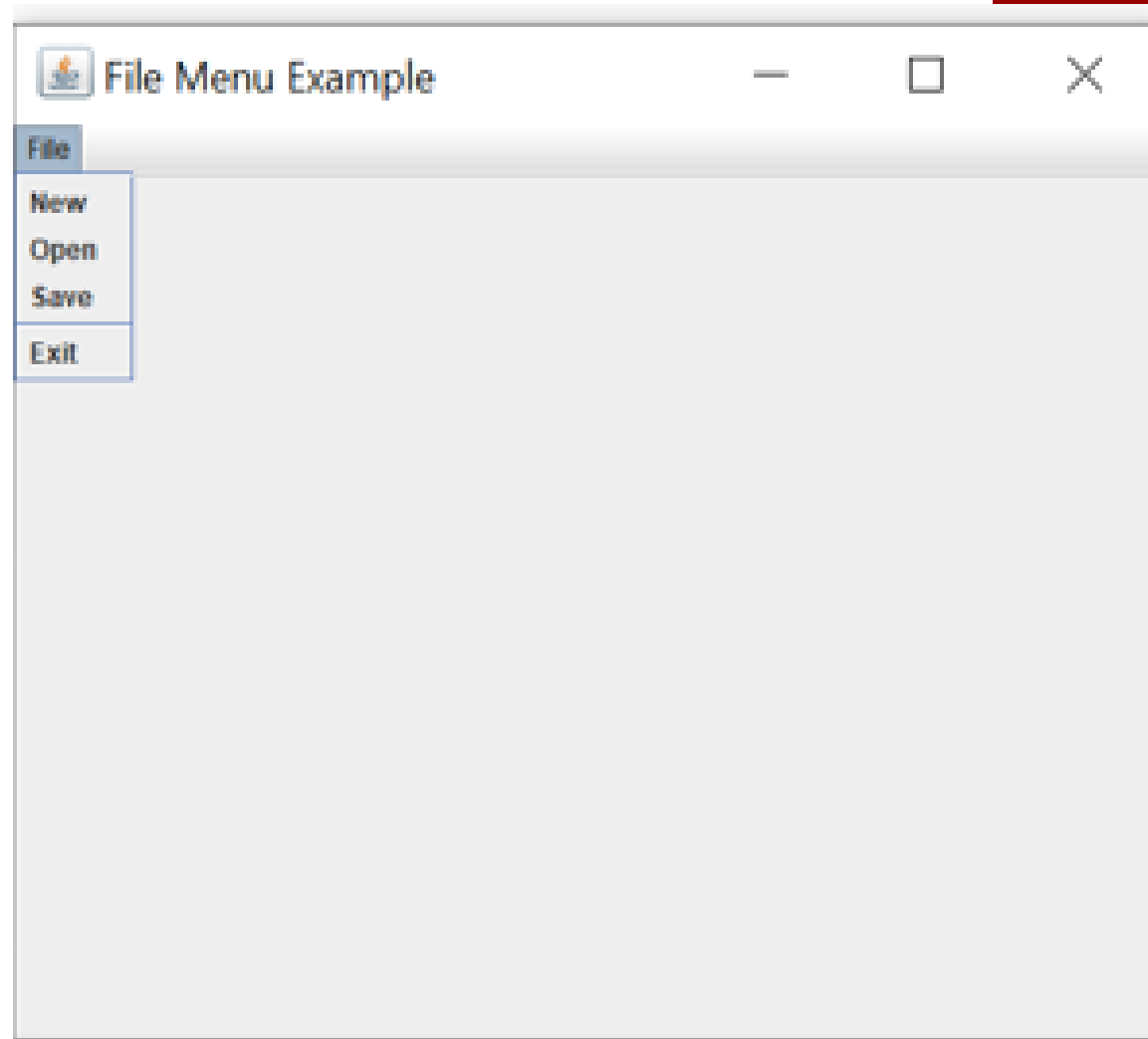
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

12
8

```
31 // Add the "File" menu to the menu bar
32 menuBar.add(fileMenu);
33
34 // Set the JFrame's menu bar to the created menuBar
35 frame.setJMenuBar(menuBar);
36
37 // Add an action listener to the "Exit" menu item
38 exitMenuItem.addActionListener(new ActionListener() {
39     // Override the actionPerformed method to define the exit behavior
40     public void actionPerformed(ActionEvent e) {
41         // Exit the application when "Exit" is selected
42         System.exit(0);
43     }
44 });
45
46 // Set the size of the JFrame
47 frame.setSize(300, 200);
48 // Set the default close operation for the JFrame
49 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
50 // Make the JFrame visible
51 frame.setVisible(true);
52 }
53 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Sample Output:



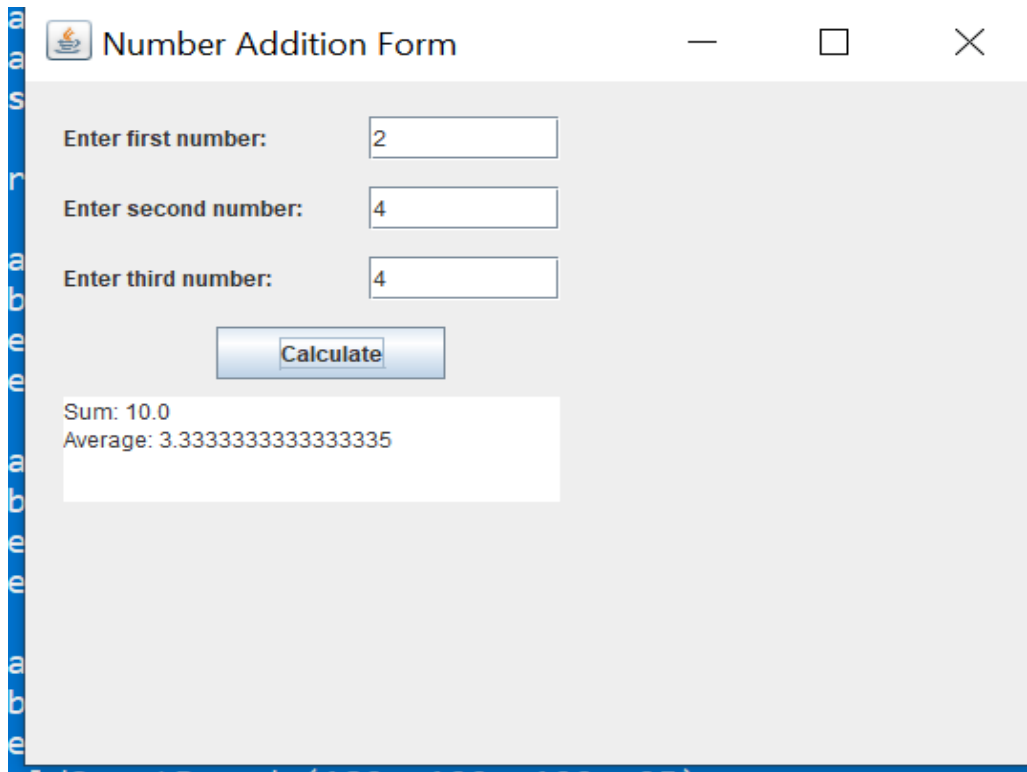
12
9

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

13
0

Exercises:

1. a. Use Swing Container and make a form to add three(3) numbers and get the results/sum and average of numbers as shown below:



The screenshot shows a Java Swing window titled "Number Addition Form". Inside the window, there are three input fields labeled "Enter first number:", "Enter second number:", and "Enter third number:". The first field contains the value "2", the second field contains "4", and the third field contains "4". Below these fields is a button labeled "Calculate". At the bottom of the window, there is a text area displaying the results: "Sum: 10.0" and "Average: 3.3333333333333335".

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

13
1

E

```
1 package BankAccount;
2 import javax.swing.*;
3 import java.awt.event.*;
4 public class NumberForm {
5     public static void main(String[] args) {
6         JFrame frame = new JFrame("Number Addition Form");
7
8         JLabel label1 = new JLabel("Enter first number:");
9         label1.setBounds(20, 20, 150, 25);
10        JTextField field1 = new JTextField();
11        field1.setBounds(180, 20, 100, 25);
12
13        JLabel label2 = new JLabel("Enter second number:");
14        label2.setBounds(20, 60, 150, 25);
15        JTextField field2 = new JTextField();
16        field2.setBounds(180, 60, 100, 25);
17
18        JLabel label3 = new JLabel("Enter third number:");
19        label3.setBounds(20, 100, 150, 25);
20        JTextField field3 = new JTextField();
21        field3.setBounds(180, 100, 100, 25);
22
23        JButton calcButton = new JButton("Calculate");
24        calcButton.setBounds(100, 140, 120, 30);
25
26        JTextArea resultArea = new JTextArea();
27        resultArea.setBounds(20, 180, 260, 60);
28        resultArea.setEditable(false);
29
30        calcButton.addActionListener(new ActionListener() {
31            public void actionPerformed(ActionEvent e) {
32                try {
33                    double num1 = Double.parseDouble(field1.getText());
34                    double num2 = Double.parseDouble(field2.getText());
35                    double num3 = Double.parseDouble(field3.getText());
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

13

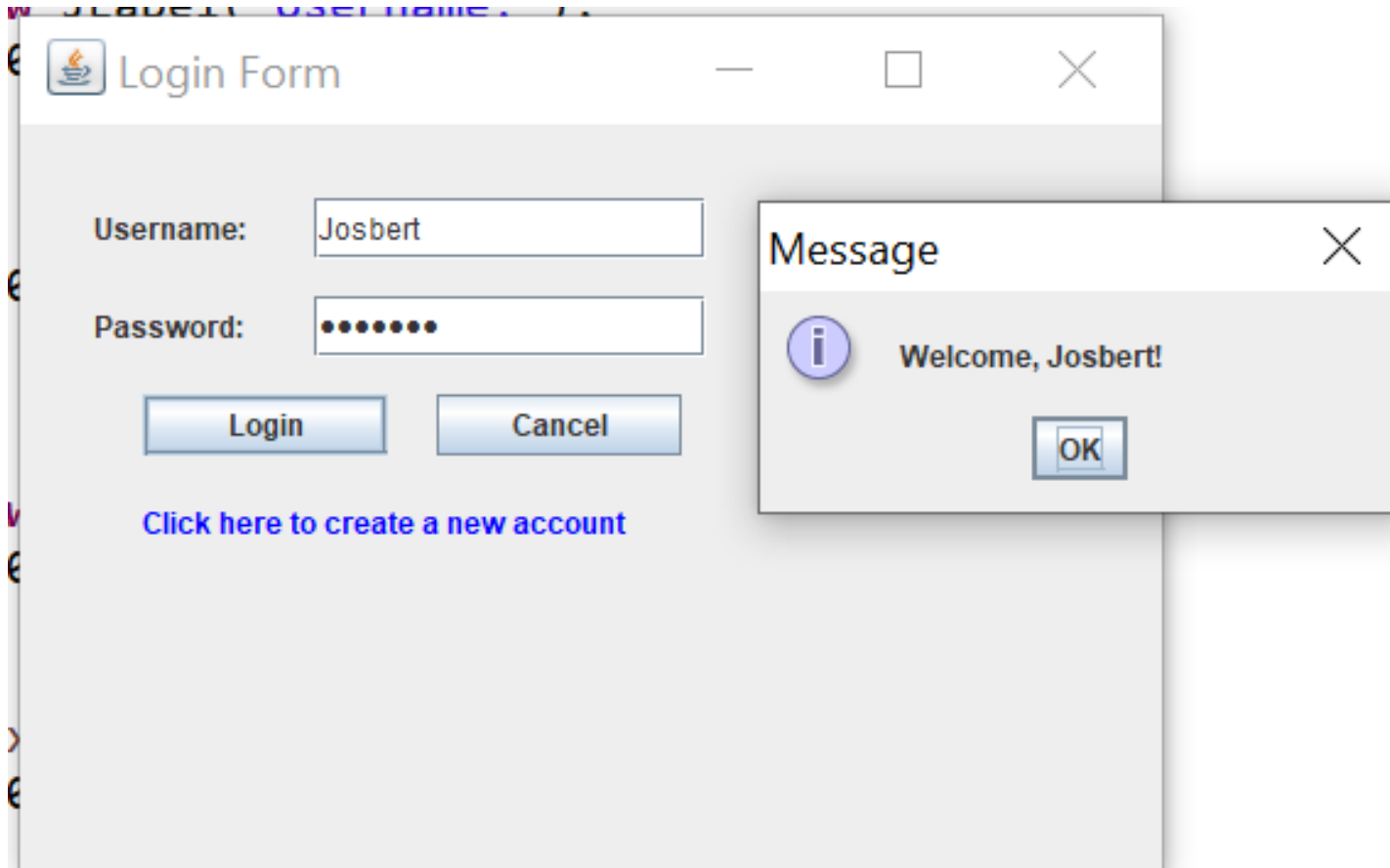
```
36
37         double sum = num1 + num2 + num3;
38         double average = sum / 3;
39
40         resultArea.setText("Sum: " + sum + "\nAverage: " + average);
41     } catch (NumberFormatException ex) {
42         resultArea.setText("Please enter valid numbers.");
43     }
44 }
45 });
46
47 frame.add(label1); frame.add(field1);
48 frame.add(label2); frame.add(field2);
49 frame.add(label3); frame.add(field3);
50 frame.add(calcButton);
51 frame.add(resultArea);
52
53 frame.setSize(320, 300);
54 frame.setLayout(null);
55 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
56 frame.setVisible(true);
57 }
58 }
59
```


UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

13
3

Exercises:

1. b. Login Form



E

1

```
1 package BankAccount;
2 import javax.swing.*;
3 import java.awt.*;
4 import java.awt.event.*;
5 public class SimpleLoginForm_impro {
6     public static void main(String[] args) {
7         // Create frame
8         JFrame frame = new JFrame("Login Form");
9         frame.setSize(350, 250);
10        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11        frame.setLayout(null);
12
13        // Username Label and TextField
14        JLabel userLabel = new JLabel("Username:");
15        userLabel.setBounds(30, 30, 80, 25);
16        frame.add(userLabel);
17
18        JTextField userText = new JTextField();
19        userText.setBounds(120, 30, 160, 25);
20        frame.add(userText);
21
22        // Password Label and PasswordField
23        JLabel passLabel = new JLabel("Password:");
24        passLabel.setBounds(30, 70, 80, 25);
25        frame.add(passLabel);
26
27        JPasswordField passText = new JPasswordField();
28        passText.setBounds(120, 70, 160, 25);
29        frame.add(passText);
30
31        // Login Button
32        JButton loginButton = new JButton("Login");
33        loginButton.setBounds(50, 110, 100, 25);
34        frame.add(loginButton);
35    }
```

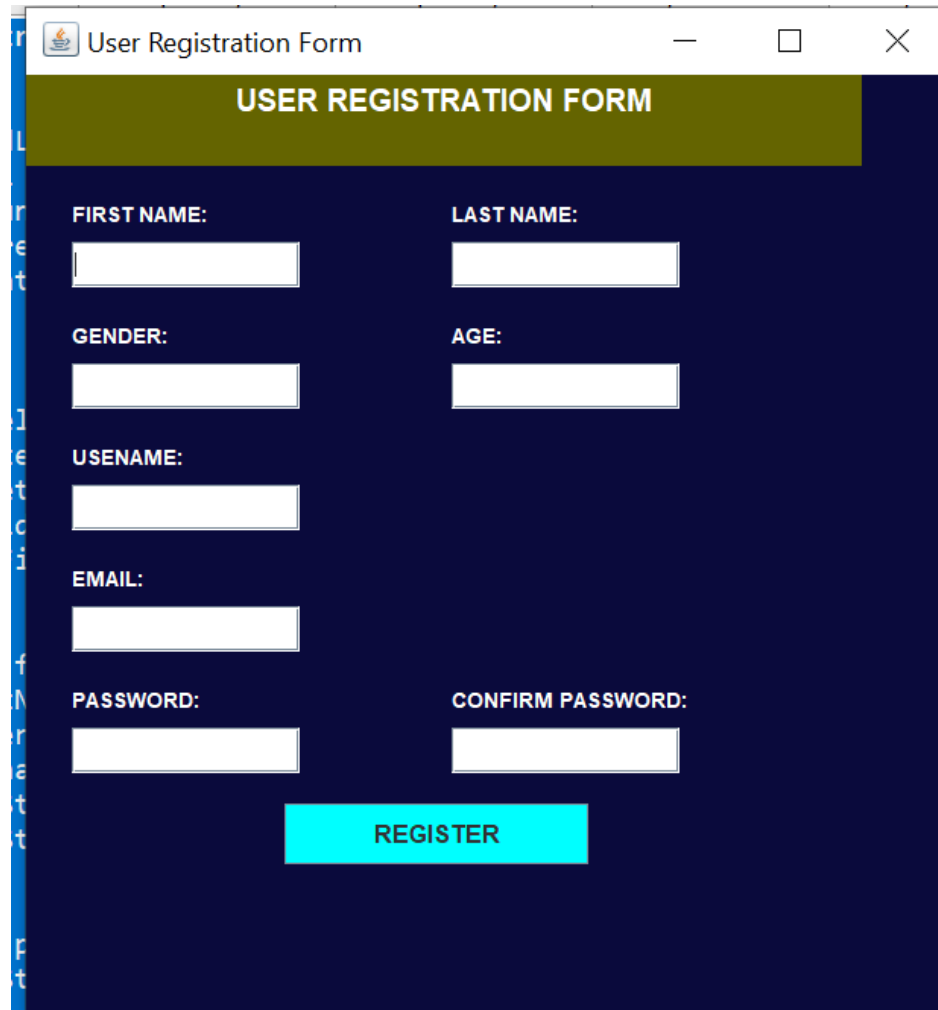
```
35
36 // Cancel Button
37 JButton cancelButton = new JButton("Cancel");
38 cancelButton.setBounds(170, 110, 100, 25);
39 frame.add(cancelButton);
40
41 // Create Account Label
42 JLabel registerLabel = new JLabel("Click here to create a new account");
43 registerLabel.setBounds(50, 150, 250, 25);
44 registerLabel.setForeground(Color.BLUE);
45 frame.add(registerLabel);
46
47 // Action listener for login button
48 loginButton.addActionListener(e -> {
49     String username = userText.getText();
50     String password = new String(passText.getPassword());
51
52     if (username.equals("Josbert") && password.equals("Josbert")) {
53         JOptionPane.showMessageDialog(frame, "Welcome, Josbert!");
54     } else {
55         JOptionPane.showMessageDialog(frame, "Username and password incorrect", "Login Failed",
56     }
57 });
58
59 // Action listener for cancel button
60 cancelButton.addActionListener(e -> {
61     userText.setText("");
62     passText.setText("");
63 });
64 // Show frame
65 frame.setLocationRelativeTo(null); // Center the frame
66 frame.setVisible(true);
67 }
68 }
69
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

13
6

Exercises:

1. c. User registration form



The image shows a screenshot of a graphical user interface for a user registration form. The window has a title bar with the text "User Registration Form" and standard minimize, maximize, and close buttons. The form itself has a dark blue background with a green header bar that says "USER REGISTRATION FORM". The form contains several input fields and labels:

- FIRST NAME:** followed by a white text input field.
- LAST NAME:** followed by a white text input field.
- GENDER:** followed by a white text input field.
- AGE:** followed by a white text input field.
- USERNAME:** followed by a white text input field.
- EMAIL:** followed by a white text input field.
- PASSWORD:** followed by a white text input field.
- CONFIRM PASSWORD:** followed by a white text input field.

At the bottom center of the form is a red button with the text "REGISTER".

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

13

```
1 package BankAccount;
2 import javax.swing.*;
3 import java.awt.*;
4 import java.awt.event.*;
5 public class RegistrationForm_imp extends JFrame {
6
7     // Declare fields
8     JTextField firstNameField, lastNameField, genderField, ageField, usernameField, emailField;
9     JPasswordField passwordField, confirmPasswordField;
10
11     public RegistrationForm_imp() {
12         // Frame setup
13         setTitle("User Registration Form");
14         setSize(550, 550);
15         setLayout(null);
16         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17         getContentPane().setBackground(new Color(10, 10, 60)); // dark blue
18
19         // Title Panel
20         JPanel titlePanel = new JPanel();
21         titlePanel.setBackground(new Color(100, 100, 0)); // olive
22         titlePanel.setBounds(0, 0, 550, 60);
23         JLabel titleLabel = new JLabel("USER REGISTRATION FORM");
24         titleLabel.setForeground(Color.WHITE);
25         titleLabel.setFont(new Font("Arial", Font.BOLD, 20));
26         titlePanel.add(titleLabel);
27         add(titlePanel);
28
29         // First row
30         addLabel("FIRST NAME:", 30, 80);
31         firstNameField = addTextField(30, 110);
32
33         addLabel("LAST NAME:", 280, 80);
34         lastNameField = addTextField(280, 110);
35
```

```
36 // Second row (Gender and Age)
37 addLabel("GENDER:", 30, 160);
38 genderField = addTextField(30, 190);
39
40 addLabel("AGE:", 280, 160);
41 ageField = addTextField(280, 190);
42
43 // Third row (Username)
44 addLabel("USERNAME:", 30, 240);
45 usernameField = addTextField(30, 270);
46
47 // Fourth row (Email)
48 addLabel("EMAIL:", 30, 320);
49 emailField = addTextField(30, 350);
50
51 // Fifth row (Password and Confirm Password)
52 addLabel("PASSWORD:", 30, 400);
53 passwordField = new JPasswordField();
54 passwordField.setBounds(30, 430, 150, 30);
55 add(passwordField);
56
57 addLabel("CONFIRM PASSWORD:", 280, 400);
58 confirmPasswordField = new JPasswordField();
59 confirmPasswordField.setBounds(280, 430, 150, 30);
60 add(confirmPasswordField);
61
62 // Register Button
63 JButton registerButton = new JButton("REGISTER");
64 registerButton.setBounds(170, 480, 200, 40);
65 registerButton.setBackground(Color.CYAN);
66 registerButton.setFont(new Font("Arial", Font.BOLD, 16));
67 add(registerButton);
```

```
69 registerButton.addActionListener(new ActionListener() {
70     public void actionPerformed(ActionEvent e) {
71         if (fieldsAreEmpty()) {
72             JOptionPane.showMessageDialog(null, "THE FIELDS CAN NOT BE EMPTY", "Error",
73                 JOptionPane.ERROR_MESSAGE);
74         } else if (!passwordsMatch()) {
75             JOptionPane.showMessageDialog(null, "Passwords do not match", "Error",
76                 JOptionPane.ERROR_MESSAGE);
77         } else {
78             JOptionPane.showMessageDialog(null, "Registration Successful");
79         }
80     }
81 });
82
83 setVisible(true);
84 }
85
86 private void addLabel(String text, int x, int y) {
87     JLabel label = new JLabel(text);
88     label.setBounds(x, y, 200, 25);
89     label.setForeground(Color.WHITE);
90     label.setFont(new Font("Arial", Font.BOLD, 14));
91     add(label);
92 }
93
94 private JTextField addTextField(int x, int y) {
95     JTextField textField = new JTextField();
96     textField.setBounds(x, y, 150, 30);
97     add(textField);
98     return textField;
99 }
```

UNIT 1.GRAPHICAL USER INTERFACE PROGRAMMING

14
0

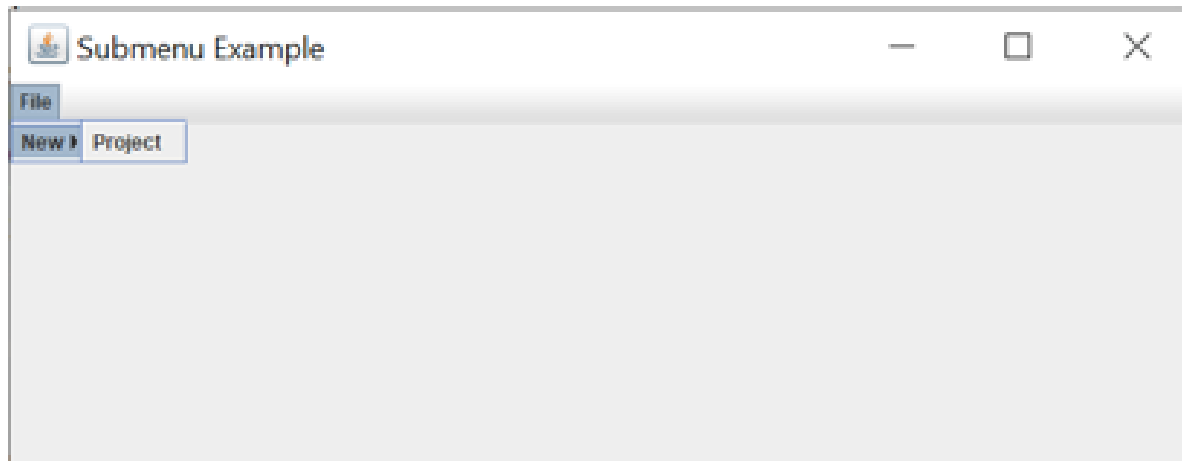
```
100  
101 private boolean fieldsAreEmpty() {  
102     return firstNameField.getText().isEmpty() || lastNameField.getText().isEmpty() ||  
103         genderField.getText().isEmpty() || ageField.getText().isEmpty() ||  
104         usernameField.getText().isEmpty() || emailField.getText().isEmpty() ||  
105         new String(passwordField.getPassword()).isEmpty() ||  
106         new String(confirmPasswordField.getPassword()).isEmpty();  
107 }  
108  
109 private boolean passwordsMatch() {  
110     return new String(passwordField.getPassword()).equals(new String(confirmPasswordField.  
111         getPassword()));  
112 }  
113  
114 public static void main(String[] args) {  
115     new RegistrationForm_imp();  
116 }  
117  
118 }
```


UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

14
1

Exercises:

2. Write the codes of the following Menu with SubMenu using JAVA GUI.

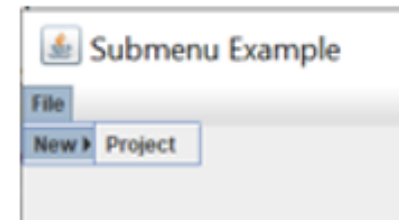


UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

14
2

Solution:

```
1 package josbert_new_;
2 import javax.swing.*;
3
4 public class SubmenuExample {
5
6     public static void main(String[] args) {
7         JFrame frame = new JFrame("Submenu Example");
8         JMenuBar menuBar = new JMenuBar();
9         JMenu fileMenu = new JMenu("File");
10        JMenu subMenu = new JMenu("New");
11        JMenuItem menuItem = new JMenuItem("Project");
12
13        subMenu.add(menuItem); // add Project to New
14        fileMenu.add(subMenu); // add New to File
15        menuBar.add(fileMenu);
16        frame.setJMenuBar(menuBar);
17
18        frame.setSize(300, 200);
19        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
20        frame.setVisible(true);
21    }
22
23 }
```



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

14
3

Let's add more comments to this program

```
1 package josbert_new_;
2 import javax.swing.*; // Import the Swing package for GUI components
3 public class Sub_menu_2 {
4
5     public static void main(String[] args) {
6         // Create a new JFrame (window) with the title "Submenu Example"
7         JFrame frame = new JFrame("Submenu Example");
8
9         // Create a new JMenuBar to hold the menu items
10        JMenuBar menuBar = new JMenuBar();
11
12        // Create a new JMenu for the top-level menu called "File"
13        JMenu fileMenu = new JMenu("File");
14
15        // Create a new JMenu for the submenu called "New"
16        JMenu subMenu = new JMenu("New");
17
18        // Create a new JMenuItem for an action called "Project"
19        JMenuItem menuItem = new JMenuItem("Project");
20
21        // Add the Project menu item to the New submenu
22        subMenu.add(menuItem); // add Project to New
23
24        // Add the New submenu to the File menu
25        fileMenu.add(subMenu); // add New to File
26
27        // Add the File menu to the menu bar
28        menuBar.add(fileMenu); // add File menu to the menu bar
29
30        // Set the created menu bar on the JFrame
31        frame.setJMenuBar(menuBar);
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

14
4

```
32
33 // Set the size of the JFrame to 300 pixels wide and 200 pixels tall
34 frame.setSize(300, 200);
35
36 // Set the default close operation to exit the application when the window is closed
37 frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38
39 // Make the JFrame visible on the screen
40 frame.setVisible(true);
41 }
42 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

14
5

Exercises:

3. Design Menu File and add Option 1 and Option 2 as Radio Button Using JAVA GUI.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

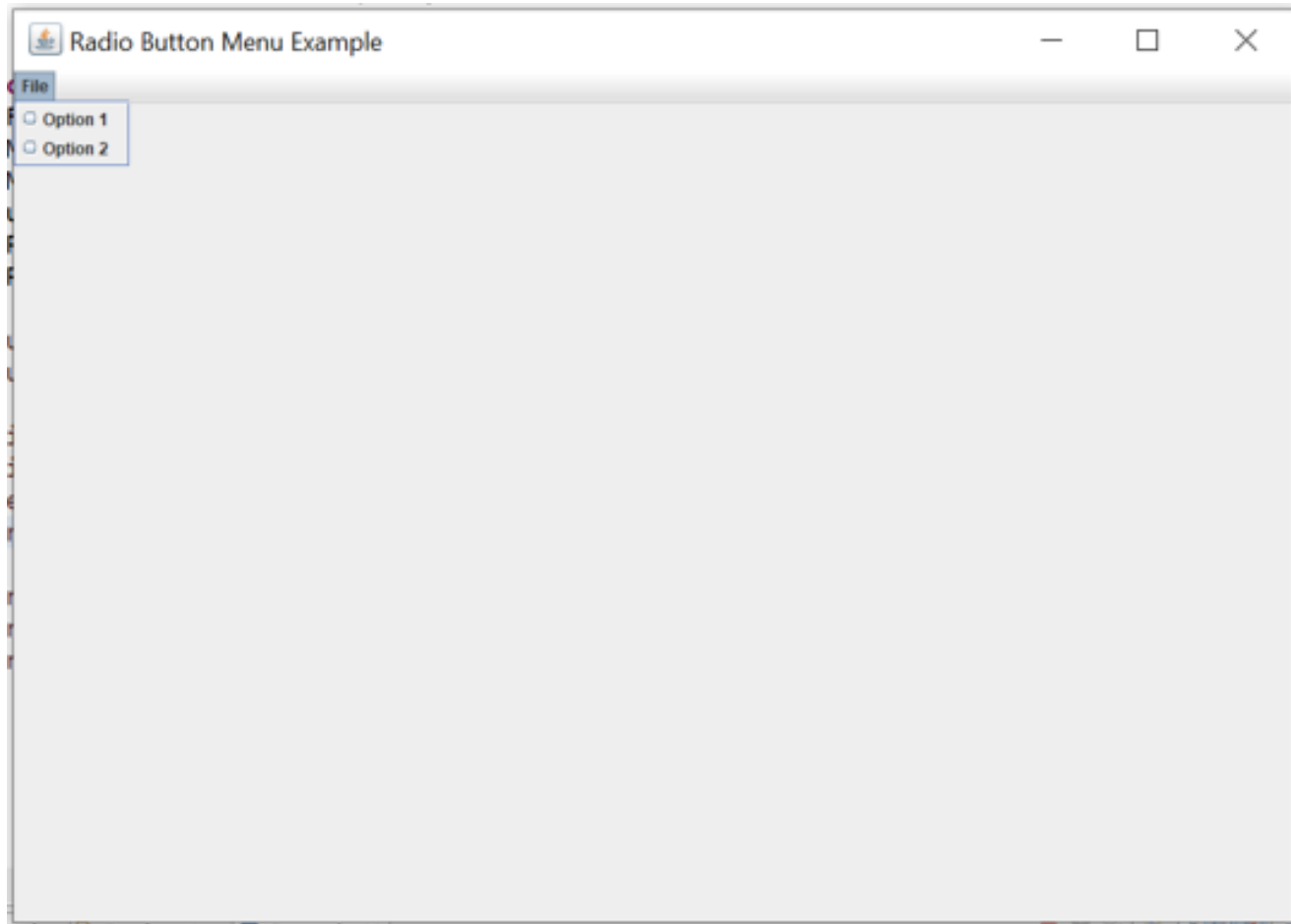
14
6

Solution:

```
1 package josbert_new_;
2 import javax.swing.*;
3
4 public class RadioButtonMenuExample {
5
6
7     public static void main(String[] args) {
8         JFrame frame = new JFrame("Radio Button Menu Example");
9         JMenuBar menuBar = new JMenuBar();
10        JMenu fileMenu = new JMenu("File");
11        ButtonGroup buttonGroup = new ButtonGroup();
12        JRadioButtonMenuItem radioButtonItem1 = new JRadioButtonMenuItem("Option 1");
13        JRadioButtonMenuItem radioButtonItem2 = new JRadioButtonMenuItem("Option 2");
14
15
16        //add buttons to frame
17        buttonGroup.add(radioButtonItem1);
18        buttonGroup.add(radioButtonItem2);
19
20        //add buttons to Menu File
21        fileMenu.add(radioButtonItem1);
22        fileMenu.add(radioButtonItem2);
23        menuBar.add(fileMenu); //add fileMenu to the bar
24        frame.setJMenuBar(menuBar); // create menu bar
25
26        frame.setSize(300, 200);
27        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
28        frame.setVisible(true);
29    }
30 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Sample Output:



14
7

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

4. Design the Color Chooser Using JAVA GUI



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

14
9

```
1 package josbert_new_;
2 import java.awt.*;           // Using AWT's Graphics and Color
3 import java.awt.event.*;     // Using AWT's event classes and listener interfaces
4 import javax.swing.*;
5
6 @SuppressWarnings("serial")
7 public class JColorChooserDemo extends JFrame {
8
9
10     //public class JColorChooserDemo extends JFrame {
11     JPanel panel;
12     Color bgColor = Color.LIGHT_GRAY; // Panel's background color
13
14     // Constructor to setup the UI components and event handlers
15     public JColorChooserDemo() {
16         panel = new JPanel(new BorderLayout());
17
18         JButton btnColor = new JButton("Change Color");
19         panel.add(btnColor, BorderLayout.SOUTH);
20         btnColor.addActionListener(new ActionListener() {
21             @Override
22             public void actionPerformed(ActionEvent evt) {
23                 Color color = JColorChooser.showDialog(JColorChooserDemo.this,
24                     "Choose a color", bgColor);
25                 if (color != null) { // new color selected
26                     bgColor = color;
27                 }
28                 panel.setBackground(bgColor); // change panel's background color
29             }
30         });
31
32         setContentPane(panel);
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

15
0

```
33
34     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
35     setTitle("JColorChooser Demo");
36     setSize(300, 200);
37     setLocationRelativeTo(null); // center the application window
38     setVisible(true);           // show it
39 }
40
41 // The entry main() method
42 public static void main(String[] args) {
43     // Run GUI codes in the Event-Dispatching thread for thread safety
44     SwingUtilities.invokeLater(new Runnable() {
45         @Override
46         public void run() {
47             new JColorChooserDemo(); // Let the constructor do the job
48         }
49     });
50 }
51
52 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

15
1

Let's add more comments to this program

```
1 package josbert_new_;
2 import java.awt.*;           // Importing AWT classes for Graphics, Color, and Layout management
3 import java.awt.event.*;     // Importing AWT event classes and listener interfaces for handling user interactions
4 import javax.swing.*;        // Importing Swing classes for building the GUI components
5 @SuppressWarnings("serial") // Suppressing serial warning for serialization
6 public class JColorChooserDemo_ extends JFrame { // Class extending JFrame to create a windowed application
7
8     JPanel panel; // Panel to hold components and set background color
9     Color bgColor = Color.LIGHT_GRAY; // Initial color for the panel's background
10
11     // Constructor to setup the UI components and event handlers
12     public JColorChooserDemo_() {
13         panel = new JPanel(new BorderLayout()); // Creating a JPanel with BorderLayout
14
15         JButton btnColor = new JButton("Change Color"); // Button to trigger color change
16         panel.add(btnColor, BorderLayout.SOUTH); // Adding button to the southern part of the panel
17
18         // Adding an action listener to the button for handling click events
19         btnColor.addActionListener(new ActionListener() {
20             @Override
21             public void actionPerformed(ActionEvent evt) {
22                 // Open a color chooser dialog to let the user select a color
23                 Color color = JColorChooser.showDialog(JColorChooserDemo_.this,
24                     "Choose a color", bgColor);
25
26                 if (color != null) { // Check if the user selected a new color
27                     bgColor = color; // Update the background color with the new selection
28                 }
29                 panel.setBackground(bgColor); // Set the background color of the panel to the selected color
30             }
31         });
32     }
33 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

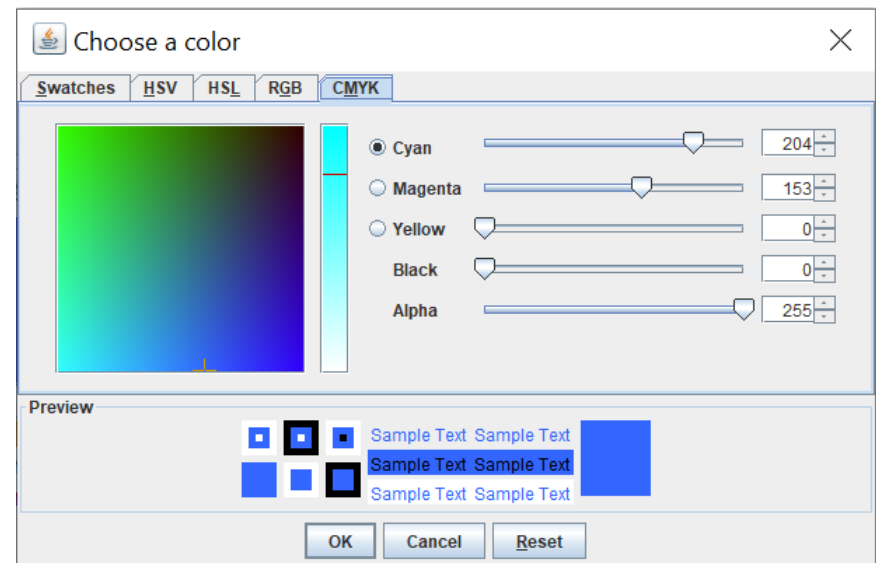
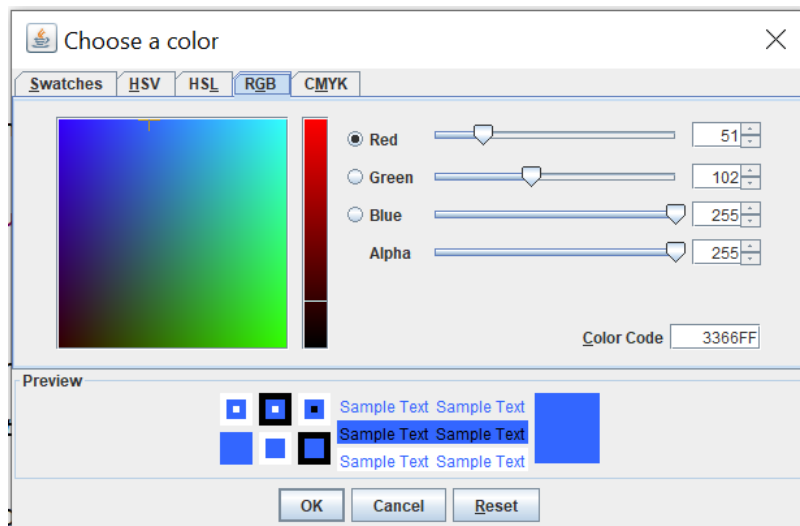
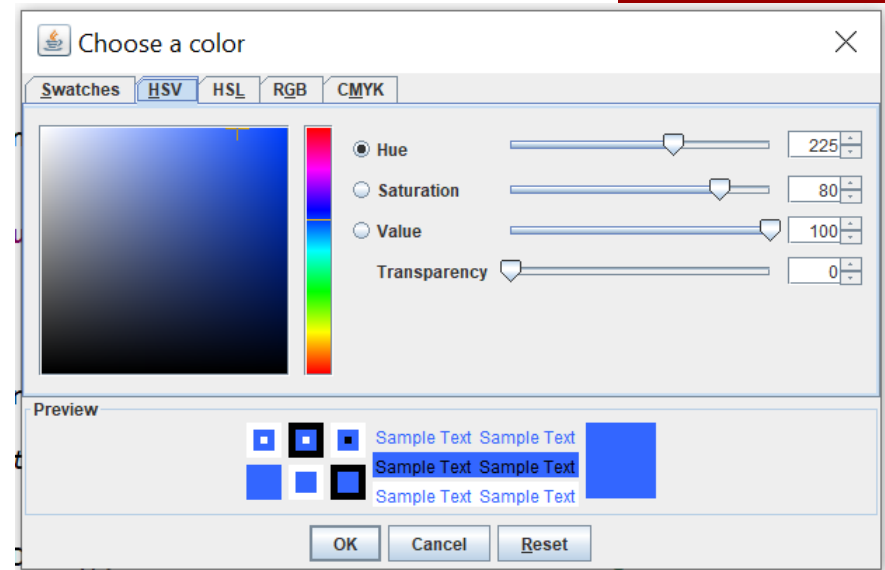
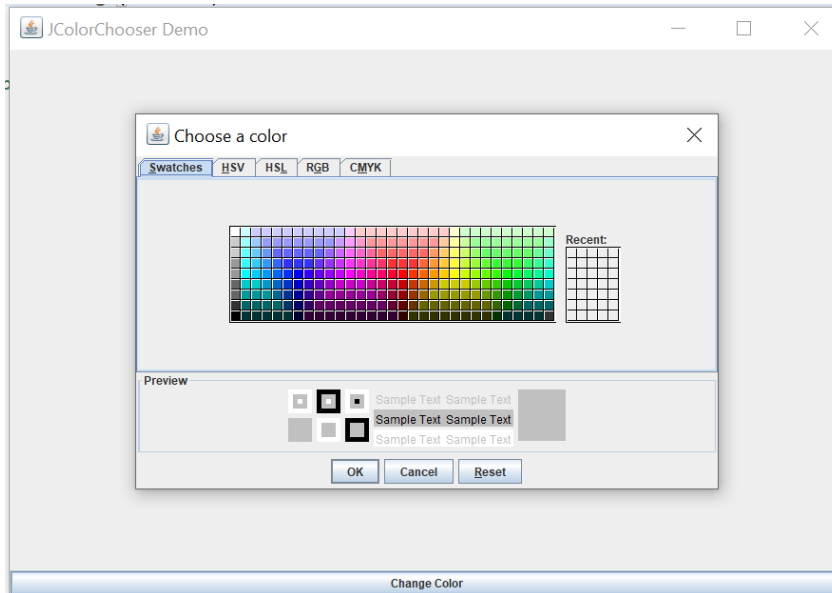
15
2

```
32
33     setContentPane(panel); // Setting the content pane of the JFrame to our panel
34
35     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Exit application on close
36     setTitle("JColorChooser Demo"); // Set title of the window
37     setSize(300, 200); // Setting the size of the window
38     setLocationRelativeTo(null); // Center the application window on the screen
39     setVisible(true); // Make the window visible
40 }
41
42 // The entry point of the program
43 public static void main(String[] args) {
44     // Run the GUI codes inside the Event-Dispatching thread for thread safety
45     SwingUtilities.invokeLater(new Runnable() {
46         @Override
47         public void run() {
48             new JColorChooserDemo_(); // Instantiate the demo class, triggering the UI setup
49         }
50     });
51 }
52
53 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Sample Output:

15
3

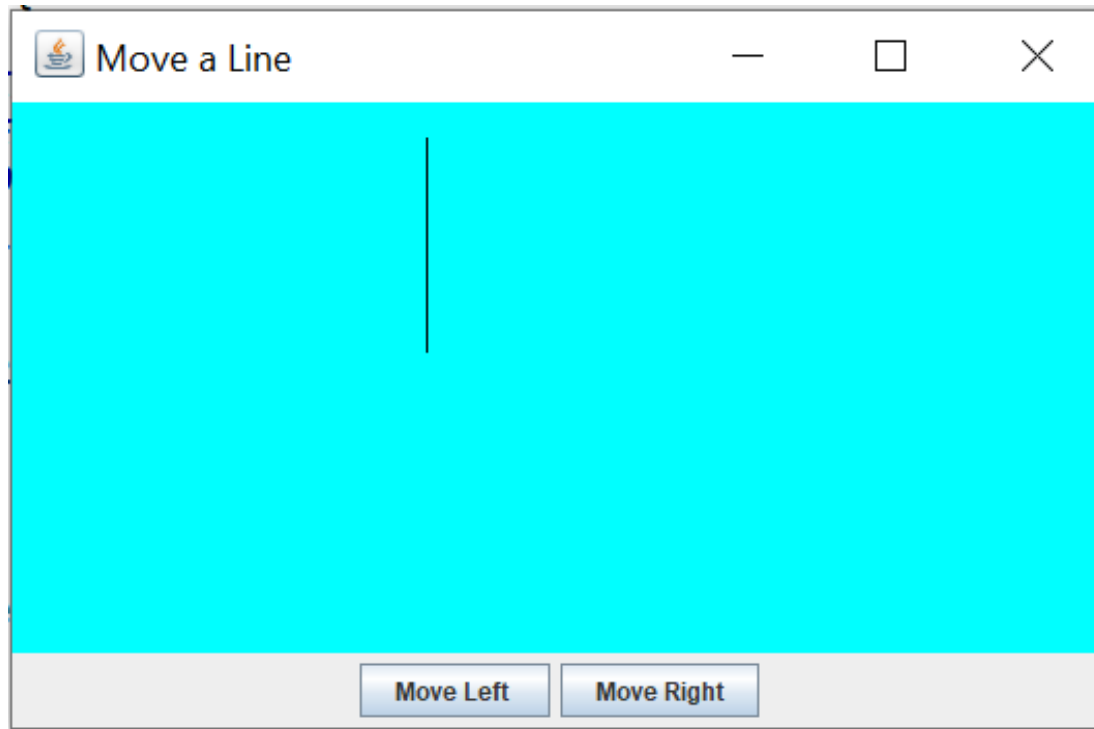


UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

15
4

Exercises:

5. Using key/button to move a line left or right.



Codes are available

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

15
5

6. Design a program to develop an Accumulator approach.

```
1 package josbert_new_;
2 import java.awt.*;           // Using AWT container and component classes
3 import java.awt.event.*;
4
5 public class AWTAccumulator extends Frame {
6
7     //private Label lblInput;   // Declare input Label (to use anonymous)
8     //private Label lblOutput; // Declare output Label (to use anonymous)
9     private TextField tfInput; // Declare input TextField
10    private TextField tfOutput; // Declare output TextField
11    private int sum = 0;        // Accumulated sum, init to 0
12
13    // Constructor to setup the GUI components and event handlers
14    public AWTAccumulator() {
15        setLayout(new GridLayout(2, 2));
16        // "super" Frame (Container) sets layout to GridLayout of 2 rows 2 columns.
17
18        add(new Label("Enter an Integer: ")); // "super" Frame adds an anonymous Label
19
20        tfInput = new TextField(10); // Construct TextField
21        add(tfInput);                // "super" Frame adds TextField
22
23        tfInput.addActionListener(new TFInputListener()); // anonymous instance
24        // "tfInput" is the source object that fires an ActionEvent upon entered.
25        // The source add an anonymous instance of TFInputListener as an ActionEvent
26        // listener, which provides an ActionEvent handler called actionPerformed().
27        // Hitting "enter" on tfInput invokes actionPerformed().
28
29        add(new Label("The Accumulated Sum is: ")); // "super" Frame adds an anonymous Label
30
31        tfOutput = new TextField(10); // allocate TextField
32        tfOutput.setEditable(false);  // read-only
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

15
6

```
33         add(tfOutput);                // "super" Frame adds TextField
34
35         setTitle("AWT Accumulator"); // "super" Frame sets title
36         setSize(350, 120); // "super" Frame sets initial window size
37         setVisible(true); // "super" Frame shows
38     }
39
40     // The entry main() method
41     public static void main(String[] args) {
42         // Invoke the constructor to setup the GUI, by allocating an anonymous instance
43         new AWTAccumulator();
44     }
45
46     // Define an inner class to handle the input TextField.
47     // An ActionEvent listener must implement ActionListener interface.
48     private class TFInputListener implements ActionListener {
49         // ActionEvent handler - Called back upon hitting "enter" key on TextField
50         @Override
51         public void actionPerformed(ActionEvent evt) {
52             // Get the String entered into the TextField tfInput, convert to int
53             int numberIn = Integer.parseInt(tfInput.getText());
54             sum += numberIn; // Accumulate numbers entered into sum
55             tfInput.setText(""); // Clear input TextField
56             tfOutput.setText(sum + ""); // Display sum on the output TextField
57                                     // convert int to String
58         }
59     }
60
61 }
```


UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

15
7

Sample Output:



The screenshot shows a Java AWT window titled "AWT Accumulator". The window contains two text input fields. The first field is labeled "Enter an Integer:" and contains the value "235". The second field is labeled "The Accumulated Sum is:" and contains the value "979". The window has standard Mac OS X window controls (red, yellow, and green buttons) in the top-left corner.

Label	Value
Enter an Integer:	235
The Accumulated Sum is:	979

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

7. Counter Radio Combo

15
8

```
1 package josbert_new_;
2 import java.awt.*;
3 import java.awt.event.*;
4 import javax.swing.*;
5 public class SwingCounterRadioCombo extends JFrame {
6     private JTextField tfCount;
7     private int count = 0; // counter's value
8     private boolean countingUp = true; // counting up or down
9     private int step = 1; // increment step size
10
11     /** Constructor to setup the UI */
12     public SwingCounterRadioCombo () {
13         Container cp = getContentPane();
14         cp.setLayout(new FlowLayout());
15
16         // Create JLabel and JTextField
17         cp.add(new JLabel("Counter:"));
18         tfCount = new JTextField("0", 5);
19         tfCount.setEditable(false);
20         tfCount.setHorizontalAlignment(JTextField.RIGHT);
21         cp.add(tfCount);
22
23         // Create JRadioButton for counting up and down
24         JRadioButton rbUp = new JRadioButton("Up", true);
25         rbUp.setMnemonic(KeyEvent.VK_U);
26         cp.add(rbUp);
27         rbUp.addActionListener(new ActionListener() {
28             @Override
29             public void actionPerformed(ActionEvent e) {
30                 countingUp = true;
31             }
32         });
33     }
34 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

15
9

Exercises:

```
33     JRadioButton rbDown = new JRadioButton("Down", true);
34     rbDown.setMnemonic(KeyEvent.VK_D);
35     cp.add(rbDown);
36     rbDown.addActionListener(new ActionListener() {
37         @Override
38         public void actionPerformed(ActionEvent e) {
39             countingUp = false;
40         }
41     });
42     // Setup a ButtonGroup to ensure exclusive selection
43     ButtonGroup btnGp = new ButtonGroup();
44     btnGp.add(rbUp);
45     btnGp.add(rbDown);
46
47     // Create JComboBox for setting the count step size
48     add(new JLabel("Step:"));
49     final Integer[] steps = {1, 2, 3, 4, 5}; // auto-upcast
50     final JComboBox<Integer> comboCount = new JComboBox<Integer>(steps);
51     comboCount.setPreferredSize(new Dimension(60, 20));
52     cp.add(comboCount);
53     comboCount.addItemListener(new ItemListener() {
54         @Override
55         public void itemStateChanged(ItemEvent e) {
56             if (e.getStateChange() == ItemEvent.SELECTED) {
57                 step = (Integer)comboCount.getSelectedItem();
58             }
59         }
60     });
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
0

Exercises:

```
61
62     // Create JButton for "Count"
63     JButton btnCount = new JButton("Count");
64     btnCount.setMnemonic(KeyEvent.VK_C);
65     cp.add(btnCount);
66     btnCount.addActionListener(new ActionListener() {
67         @Override
68         public void actionPerformed(ActionEvent e) {
69             if (countingUp) {
70                 count += step;
71             } else {
72                 count -= step;
73             }
74             tfCount.setText(count + "");
75         }
76     });
77
78     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
79     setTitle("Swing Counter with RadioButton & ComboBox");
80     setSize(480, 100);
81     setVisible(true);
82 }
83
84 /** The entry main() method */
85 public static void main(String[] args) {
86     // Run the GUI codes in the Event-Dispatching thread for thread-safety
87     SwingUtilities.invokeLater(new Runnable() {
88         @Override
```

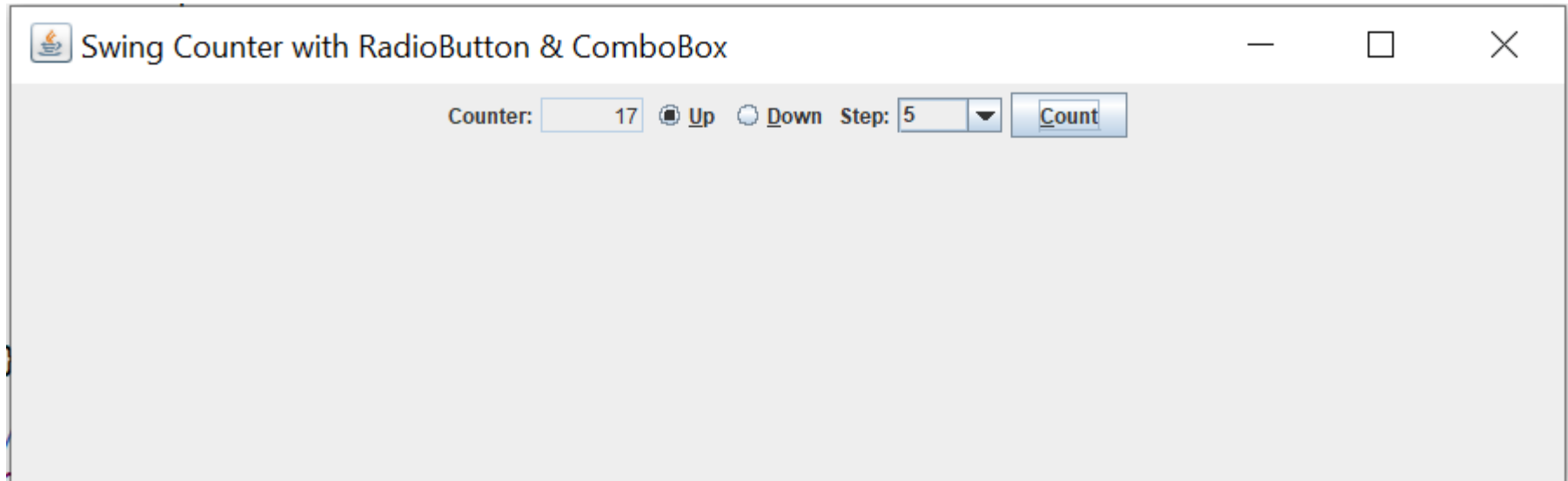
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
1

Exercises:

```
89         public void run() {  
90             new SwingCounterRadioCombo(); // Let the constructor do the job  
91         }  
92     });  
93 }  
94  
95 }  
96
```

Sample output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
2

Exercises:

8. a. Simple Banking System

```
1 package josbert_new_;
2 import java.awt.*;
5 public class Banking implements ActionListener{
6
7     private JFrame mainFrame;
8     private JTextField inputField;
9     private JLabel balanceLabel;
10    private int balance = 0;
11
12    public Banking() {
13
14        mainFrame = new JFrame("Simple Bank Application");
15        mainFrame.setVisible(true);
16        mainFrame.setSize(400, 400);
17        mainFrame.setFont(new Font("Arial", Font.BOLD, 18));
18        mainFrame.setLayout(new FlowLayout());
19
20        inputField = new JTextField(10);
21        balanceLabel = new JLabel("Current balance: " + balance);
22        JButton depositButton = new JButton("Deposit");
23        JButton withdrawButton = new JButton("Withdraw");
24
25        mainFrame.add(inputField);
26        mainFrame.add(depositButton);
27        mainFrame.add(withdrawButton);
28        mainFrame.add(balanceLabel);
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
3

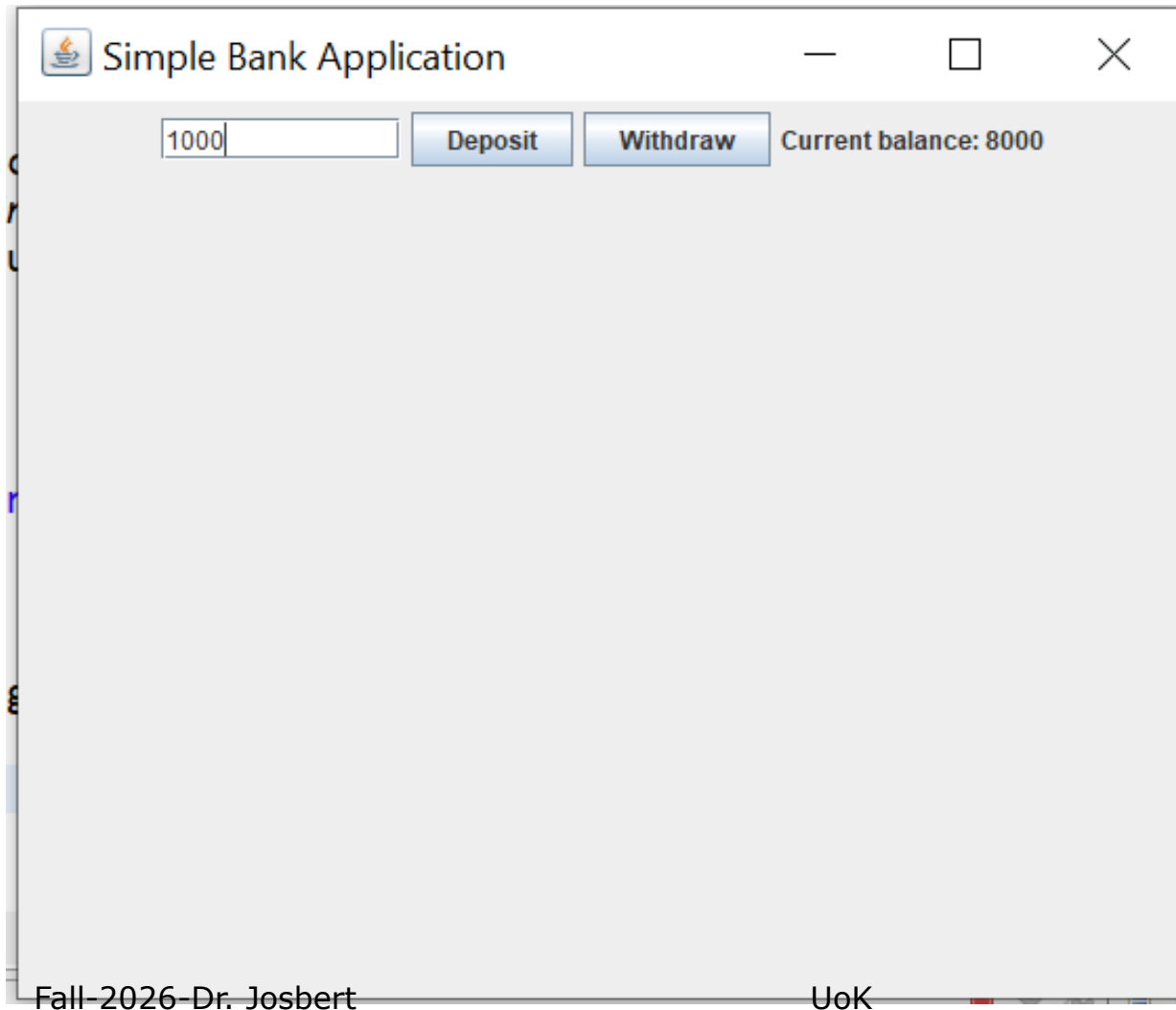
Exercises:

```
29
30     depositButton.addActionListener(this);
31     withdrawButton.addActionListener(this);
32
33     mainFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34 }
35
36 @Override
37 public void actionPerformed(ActionEvent e) {
38     int amount = Integer.parseInt(inputField.getText());
39     if (e.getActionCommand().equals("Deposit")) {
40         balance += amount;
41     } else {
42         balance -= amount;
43     }
44     balanceLabel.setText("Current balance: " + balance);
45     inputField.setText("");
46 }
47
48 public static void main(String[] args) {
49     new Banking();
50 }
51 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

Sample Output:



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
5

Exercises:

8. b. Simple Banking System

Bank Accounts Created by Josbert

Input Details

Name:

Account Number:

Balance:

Lodgement:

Withdraw:

Account Details:

Codes are available

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16

Exercises:

9. NOTEPAD Program

```
1 package josbert_new_;
2 import javax.swing.*;
3 import java.awt.*;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6 import java.io.*;
7 public class NotepadApp extends JFrame {
8
9     private JTextArea textArea;
10
11     public NotepadApp() {
12         setTitle("Simple Notepad");
13         setSize(800, 600);
14         setDefaultCloseOperation(EXIT_ON_CLOSE);
15
16         textArea = new JTextArea();
17         JScrollPane scrollPane = new JScrollPane(textArea);
18         add(scrollPane, BorderLayout.CENTER);
19
20         JMenuBar menuBar = new JMenuBar();
21         JMenu fileMenu = new JMenu("File");
22         JMenuItem openItem = new JMenuItem("Open");
23         JMenuItem saveItem = new JMenuItem("Save");
24
25         openItem.addActionListener(new ActionListener() {
26             @Override
27             public void actionPerformed(ActionEvent e) {
28                 JFileChooser fileChooser = new JFileChooser();
```

```
29         int result = fileChooser.showOpenDialog(null);
30         if (result == JFileChooser.APPROVE_OPTION) {
31             File file = fileChooser.getSelectedFile();
32             try {
33                 BufferedReader reader = new BufferedReader(new FileReader(file));
34                 String line;
35                 while ((line = reader.readLine()) != null) {
36                     textArea.append(line + "\n");
37                 }
38                 reader.close();
39             } catch (IOException ex) {
40                 ex.printStackTrace();
41             }
42         }
43     }
44
45     saveItem.addActionListener(new ActionListener() {
46         @Override
47         public void actionPerformed(ActionEvent e) {
48             JFileChooser fileChooser = new JFileChooser();
49             int result = fileChooser.showSaveDialog(null);
50             if (result == JFileChooser.APPROVE_OPTION) {
51                 File file = fileChooser.getSelectedFile();
52                 try {
53                     BufferedWriter writer = new BufferedWriter(new FileWriter(file));
54                     writer.write(textArea.getText());
55                     writer.close();
56                 } catch (IOException ex) {
57                     ex.printStackTrace();
58                 }
59             }
60         }
61     });
62
63     fileMenu.add(openItem);
64     fileMenu.add(saveItem);
65     menuBar.add(fileMenu);
66     setJMenuBar(menuBar);
67
68 }
69
70 public static void main(String[] args) {
71     SwingUtilities.invokeLater(new Runnable() {
72         @Override
73         public void run() {
74             new NotepadApp().setVisible(true);
75         }
76     });
77 }
78
79 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
7

Let's add more comments to this program:

```
1 package josbert_new_;
2 import javax.swing.*; // Importing Swing components for the GUI
3 import java.awt.*; // Importing AWT components for layout management
4 import java.awt.event.ActionEvent; // Importing ActionEvent class for handling action events
5 import java.awt.event.ActionListener; // Importing ActionListener interface for event handling
6 import java.io.*; // Importing I/O classes for file operations
7
8 // Main class for the Notepad application
9 public class NotepadApp extends JFrame {
10
11     private JTextArea textArea; // TextArea component for user text input
12
13     public NotepadApp_() {
14         // Setting up the frame properties
15         setTitle("Simple Notepad"); // Title of the Notepad window
16         setSize(800, 600); // Setting the dimensions of the window
17         setDefaultCloseOperation(EXIT_ON_CLOSE); // Exit application on close
18
19         // Initializing the text area where the user can write text
20         textArea = new JTextArea();
21         // Adding a scroll pane to the text area for scrolling capability
22         JScrollPane scrollPane = new JScrollPane(textArea);
23         // Adding the scroll pane to the center of the frame
24         add(scrollPane, BorderLayout.CENTER);
25
26         // Creating a menu bar for the application
27         JMenuBar menuBar = new JMenuBar();
28
29         // Creating 'File' menu
30         JMenu fileMenu = new JMenu("File");
31
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
8

```
32 // Menu items for 'Open' and 'Save' functionality
33 JMenuItem openItem = new JMenuItem("Open");
34 JMenuItem saveItem = new JMenuItem("Save");
35
36 // Action listener for the 'Open' menu item
37 openItem.addActionListener(new ActionListener() {
38     @Override
39     public void actionPerformed(ActionEvent e) {
40         // Creating a file chooser dialog for opening files
41         JFileChooser fileChooser = new JFileChooser();
42         // Showing the open dialog and capturing the user's choice
43         int result = fileChooser.showOpenDialog(null);
44         if (result == JFileChooser.APPROVE_OPTION) {
45             // Getting the selected file from the file chooser
46             File file = fileChooser.getSelectedFile();
47             try {
48                 // Creating a BufferedReader to read the file
49                 BufferedReader reader = new BufferedReader(new FileReader(file));
50                 String line;
51                 // Reading each line until EOF
52                 while ((line = reader.readLine()) != null) {
53                     textArea.append(line + "\n"); // Appending line to text area
54                 }
55                 reader.close(); // Closing the reader after use
56             } catch (IOException ex) {
57                 ex.printStackTrace(); // Printing stack trace in case of an exception
58             }
59         }
60     }
61 });
62
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

16
9

```
63 // Action listener for the 'Save' menu item
64 saveItem.addActionListener(new ActionListener() {
65     @Override
66     public void actionPerformed(ActionEvent e) {
67         // Creating a file chooser dialog for saving files
68         JFileChooser fileChooser = new JFileChooser();
69         int result = fileChooser.showSaveDialog(null);
70         if (result == JFileChooser.APPROVE_OPTION) {
71             // Getting the selected file from the file chooser
72             File file = fileChooser.getSelectedFile();
73             try {
74                 // Creating a BufferedWriter to write to the file
75                 BufferedWriter writer = new BufferedWriter(new FileWriter(file));
76                 writer.write(textArea.getText()); // Writing the text area content to the file
77                 writer.close(); // Closing the writer after use
78             } catch (IOException ex) {
79                 ex.printStackTrace(); // Printing stack trace in case of an exception
80             }
81         }
82     }
83 });
84
85 // Adding the 'Open' and 'Save' menu items to the file menu
86 fileMenu.add(openItem);
87 fileMenu.add(saveItem);
88 // Adding the file menu to the menu bar
89 menuBar.add(fileMenu);
90 // Setting the menu bar for the frame
91 setJMenuBar(menuBar);
92 }
```

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

17
0

```
93
94 // Main method to launch the application
95 public static void main(String[] args) {
96     // Ensuring that the GUI code runs on the Event Dispatch Thread
97     SwingUtilities.invokeLater(new Runnable() {
98         @Override
99         public void run() {
100             new NotepadApp_().setVisible(true); // Making the NotepadApp window visible
101         }
102     });
103 }
104
105 }
106
```

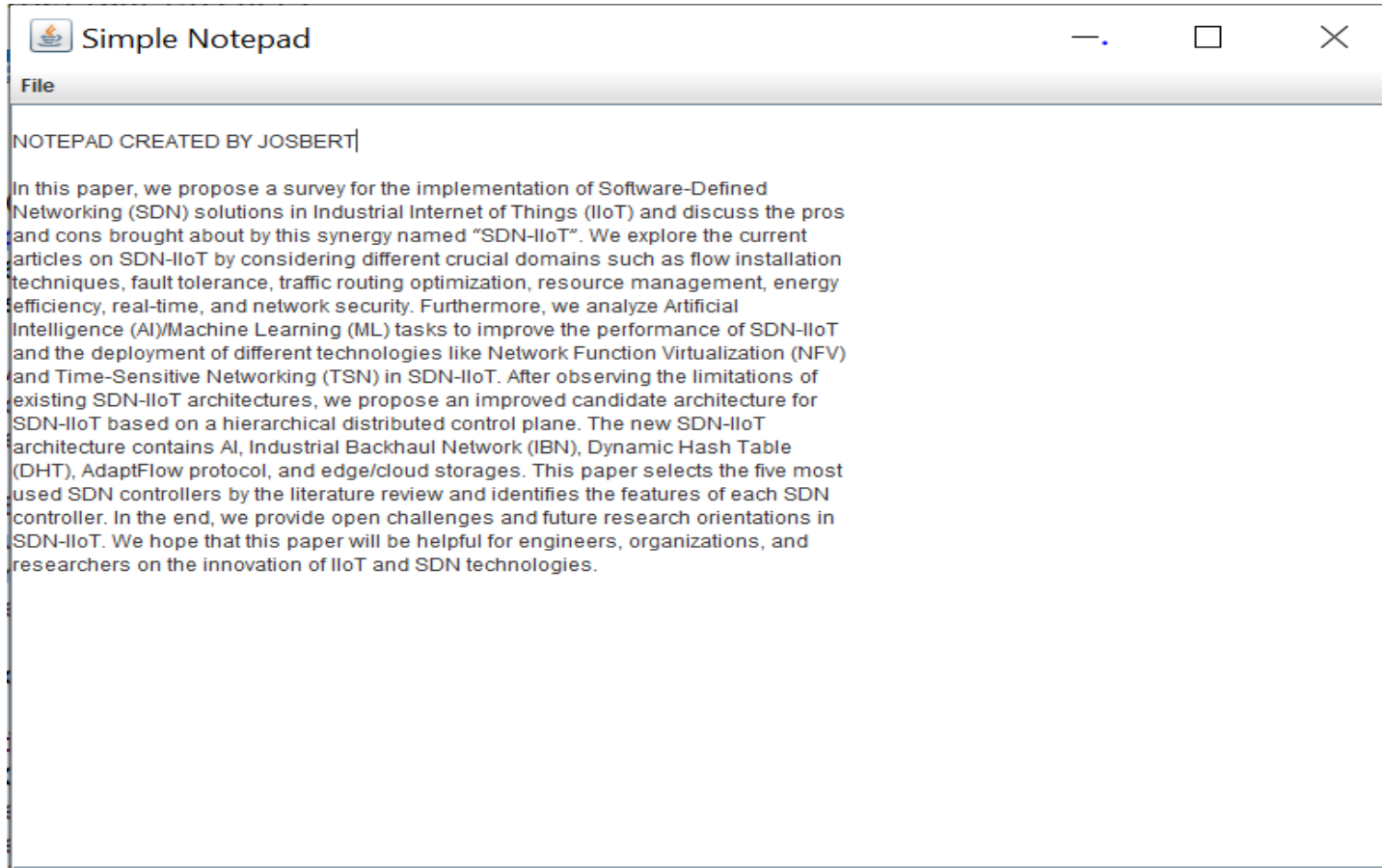
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

17
1

Exercises:

9. NOTEPAD Program

Sample Output:



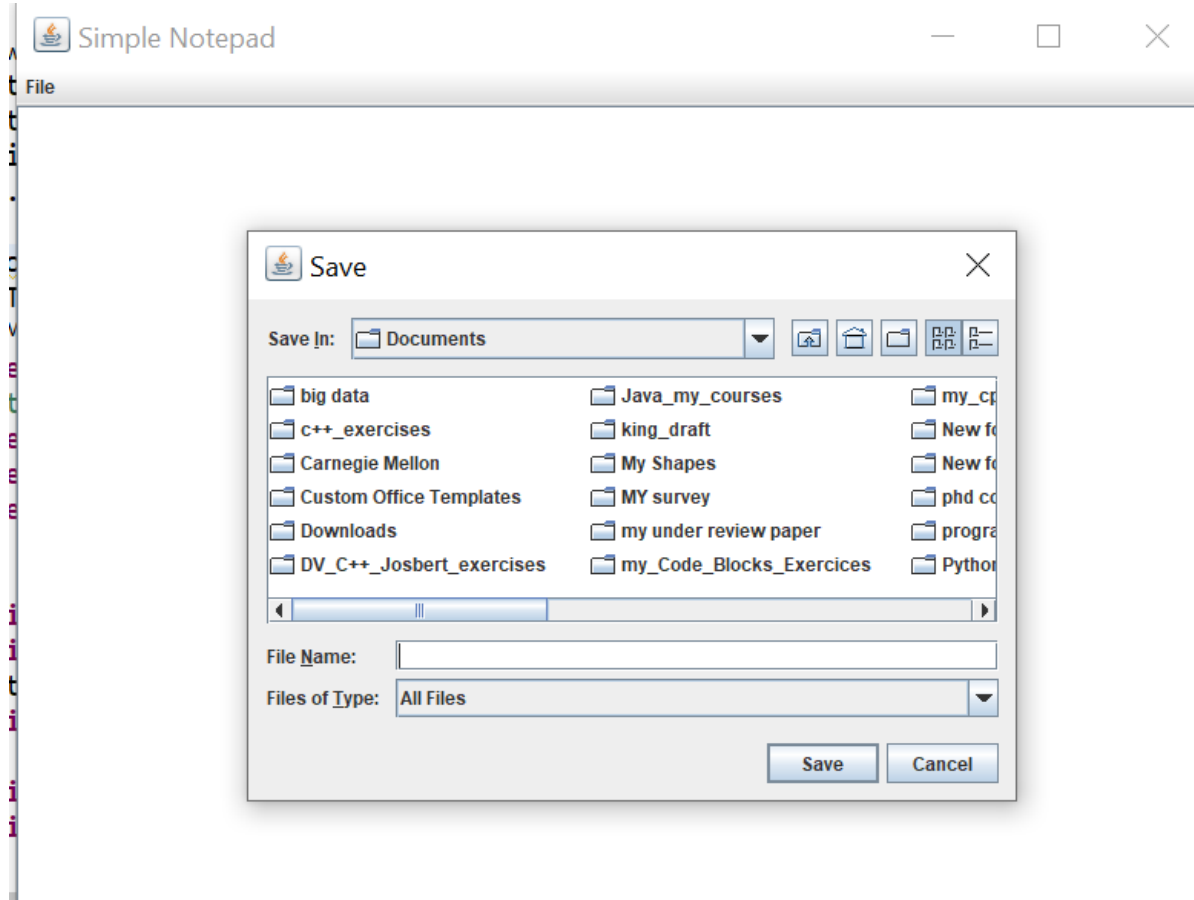
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

17
2

Exercises:

9. NOTEPAD Program

Sample Output:



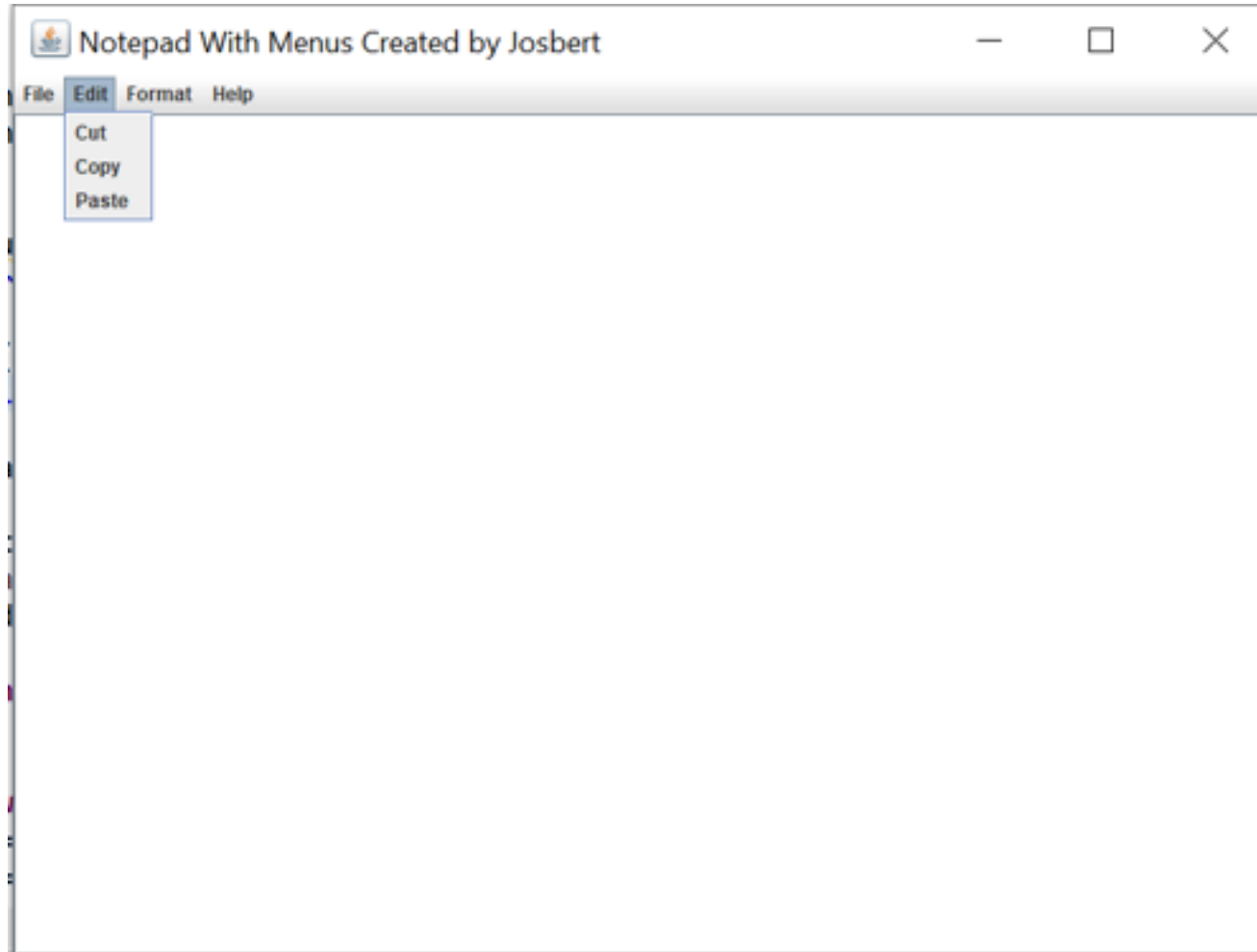
File Menu -> Save

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

10. NOTEPAD Program with Full Menus

17
3



Codes are available

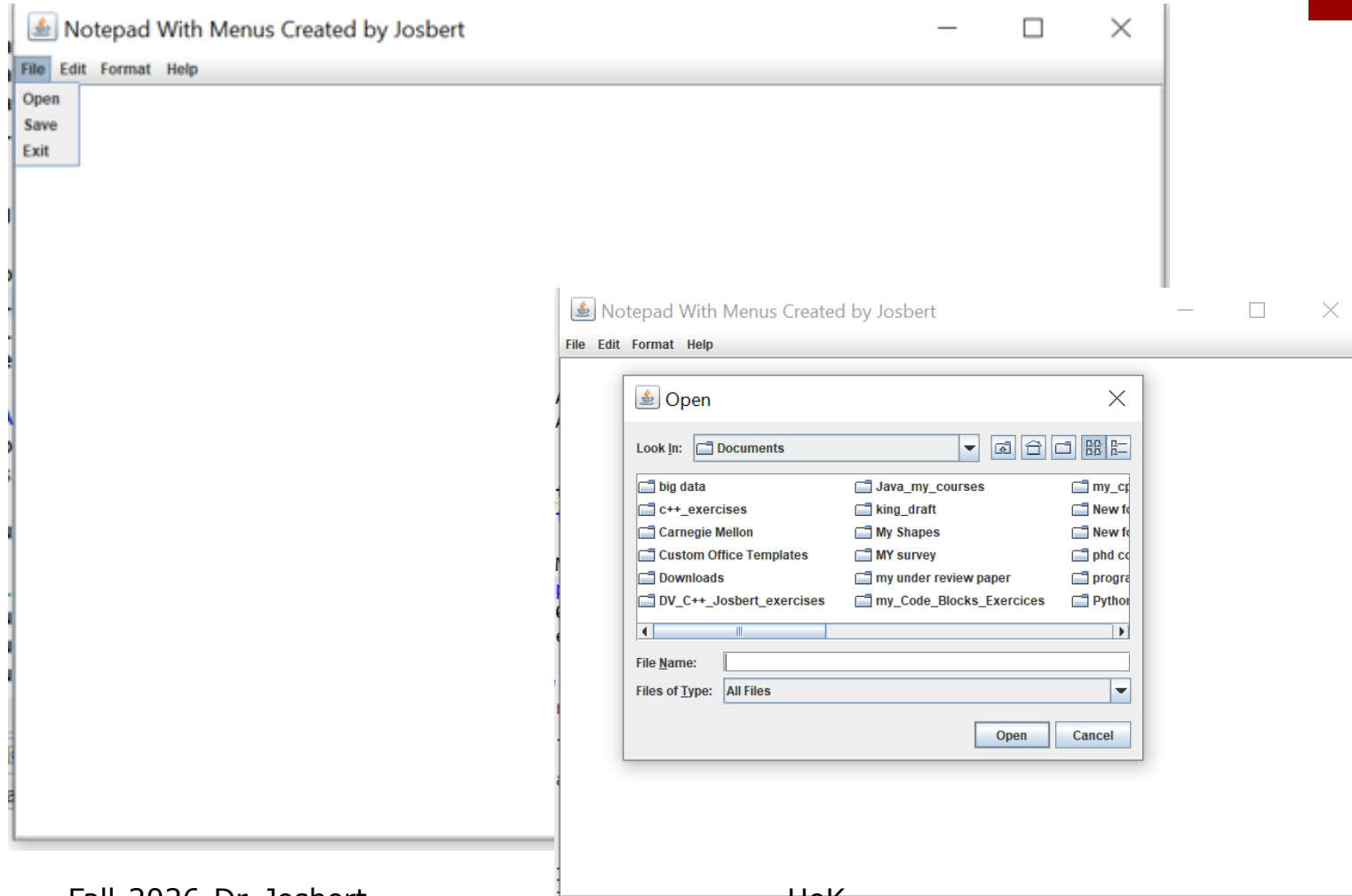
[NotepadWithMenus](#)

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

17
4

Exercises:

10. NOTEPAD Program with Full Menu



File -> Open

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

11. a. Creating an online bus ticket buying system using Java

Bus Ticket Booking System Created by Josbert

Select Route:

Select Date (YYYY-MM-DD):

Number of Tickets:

Book Ticket

Kigali-Musanze

Kigali-Musanze

Kigali-Rubavu

Musanze-Kigali

Rubavu-Musanze

Musanze-Rubavu

Rubavu-Kigali

BusTicketBookingGUI

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

17
6

Exercises:

11. a. Creating an online bus ticket buying system using Java

Bus Ticket Booking System Created by Josbert

Select Route: Kigali-Musanze

Select Date (YYYY-MM-DD): 2024-06-10

Number of Tickets: 3

Book Ticket

Message

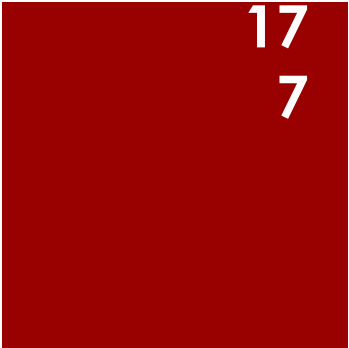
Ticket booked for Route: Kigali-Musanze, Date: 2024-06-10, Number of Tickets: 3

OK

Fall-2026-Dr. Josbert UoK BusTicketBookingGUI

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:



Online Bus Ticket Booking System

Route:

Kigali-Musanze

Passenger Name:

NTEZIRIZA NKERABAHIZI Josbert

Phone Number:

0788547461

Date of Travel:

01/08/2024

Time of Travel:

17:50

Number of Tickets:

5

Book Tickets

Clear

Booking Details:

Passenger Name: NTEZIRIZA NKERABAHIZI Josbert

Phone Number: 455766886

Route: Kigali-Musanze

Date of Travel: Thu Aug 01 16:40:39 CAT 2024

Time of Travel: Thu Aug 01 16:40:40 CAT 2024

Number of Tickets: 5

Payment Code: *182*8*1*596633*5000rwf#

Booking Details:

Passenger Name: NTEZIRIZA NKERABAHIZI Josbert

Phone Number: 455766886

Route: Kigali-Musanze

Date of Travel: Sat Aug 03 00:00:00 CAT 2024

Time of Travel: Thu Aug 01 16:40:40 CAT 2024

Number of Tickets: 5

Payment Code: *182*8*1*596633*5000rwf#

Booking Details:

Passenger Name: NTEZIRIZA NKERABAHIZI Josbert

Phone Number: 0788547461

Route: Kigali-Musanze

Date of Travel: Thu Aug 01 16:50:20 CAT 2024

Time of Travel: Thu Jan 01 17:50:00 CAT 1970

Number of Tickets: 5

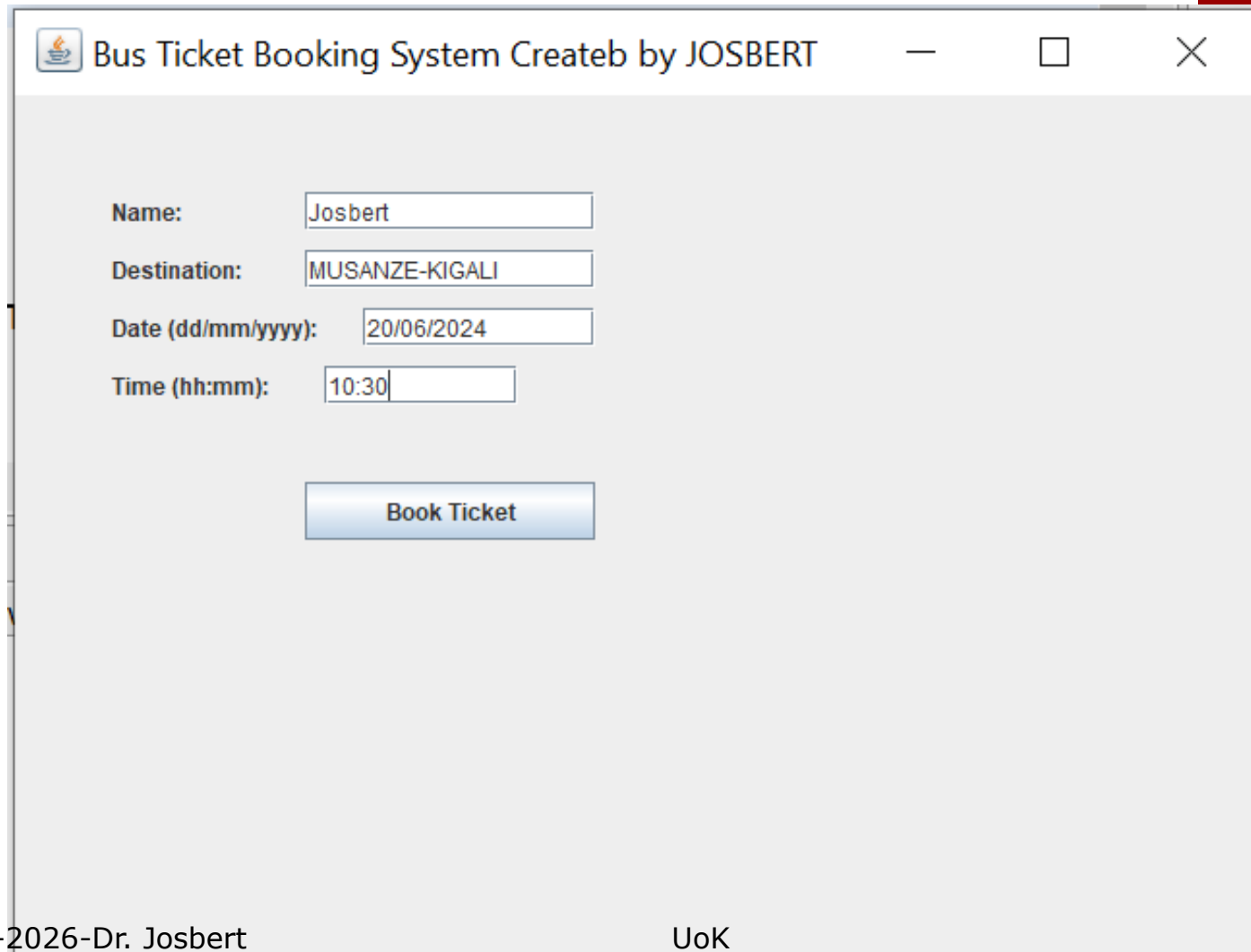
Payment Code: *182*8*1*596633*5000rwf#

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

17
8

Exercises:

11. b. Creating an online bus ticket buying system using Java



Bus Ticket Booking System Createb by JOSBERT

Name:

Destination:

Date (dd/mm/yyyy):

Time (hh:mm):

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

17
9

Exercises:

11. b. Creating an online bus ticket buying system using Java

Bus Ticket Booking System Createb by JOSBERT

Name:

Destination:

Date (dd/mm/yyyy):

Time (hh:mm):

Message

 Ticket booked successfully for Josbert to MUSANZE-KIGALI on 20/06/2024 10:30

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

18
0

Exercises:

12. Online registration of students using Graphical User Interface (GUI) with Swing

The screenshot shows a Java Swing window titled "Student Registration". The window contains a registration form with the following fields and values:

Field	Value
Name:	NTEZIRIZA NKERABAHIZI JOSBERT
Age:	100
Sex:	Male
Nationality:	RWANDAISE
Section:	Software
Department:	Computer Science and Technology

At the bottom of the form is a blue button labeled "Register".

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

18
1

Exercises:

12. Online registration of students using Graphical User Interface (GUI) with Swing.

The screenshot displays a Java Swing window titled "Student Registration". The window contains a registration form with the following fields and values:

- Name: NTEZIRIZA NKERABAHIZI JOSBERT
- Age: 100
- Sex: Male
- Nationality: RWANDAISE
- Section: Software
- Department: Computer Science and Technology

A confirmation message dialog box is open over the form, displaying the following information:

- Name: NTEZIRIZA NKERABAHIZI JOSBERT
- Age: 100
- Sex: Male
- Nationality: RWANDAISE
- Section: Software
- Department: Computer Science and Technology

The dialog box has an "OK" button. At the bottom of the registration form, there is a "Register" button.

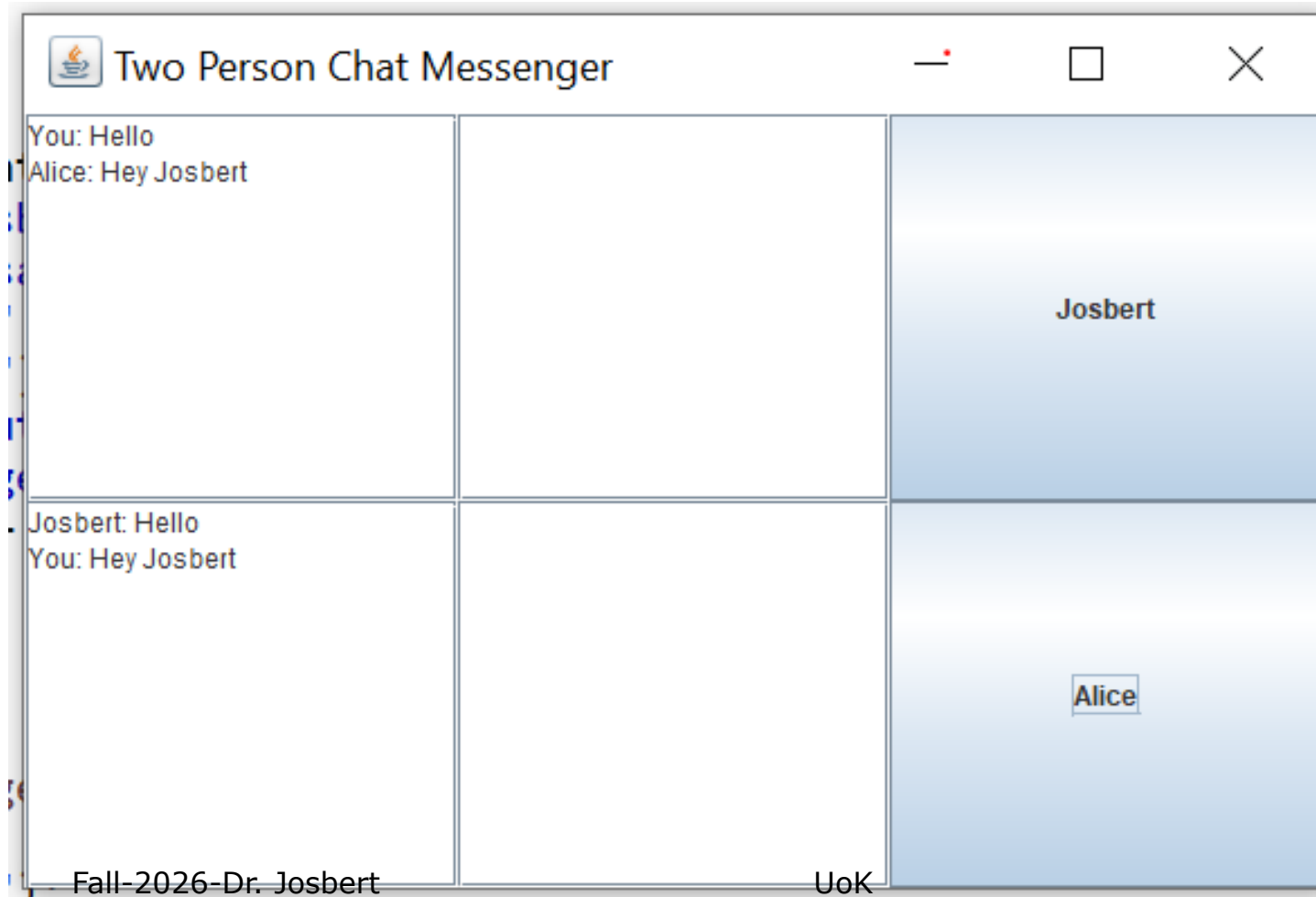
StudentRegistration

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

13. An example of a simple two-person chat messenger application using GUI in Java:

`TwoPersonChatMessenger` new



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

14. Create a simple Payroll System using Java GUI.

18
3

The screenshot shows a Java GUI window titled "Payroll System". It contains three input fields for "Employee Name:", "Hours Worked:", and "Hourly Rate:". The "Employee Name" field contains "Celine", "Hours Worked" contains "10", and "Hourly Rate" contains "150". Below these fields is a button labeled "Calculate Payslip". To the right of the button is a large text area displaying the calculated payroll data for three employees: Josbert, Cena, and Celine. The data for each employee includes their name, hours worked, hourly rate, and total payment.

Employee Name	Hours Worked	Hourly Rate	Total Payment
Josbert	8.0	\$100.0	\$800.0
Cena	6.0	\$200.0	\$1200.0
Celine	10.0	\$150.0	\$1500.0

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

15. A simple Hospital Management System using GUI in Java

18
4

The screenshot shows a Java Swing window titled "Hospital Management System". The window has a standard title bar with minimize, maximize, and close buttons. The main content area is divided into two columns. The left column contains four labels: "Patient ID:", "Patient Name:", "Patient Address:", and "Patient Contact:". The right column contains four corresponding text input fields. The first field contains "Louis233", the second contains "HABAGUSENGA Louis", the third contains "KIGALI-KICUKIRO", and the fourth contains "0788098764". Below the input fields, there are three buttons: "Add Patient" (left), "Update Patient" (left), and "View Patients" (right). The buttons have a blue gradient background.

Patient ID:	Louis233
Patient Name:	HABAGUSENGA Louis
Patient Address:	KIGALI-KICUKIRO
Patient Contact:	0788098764
Add Patient	View Patients
Update Patient	

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

15. A simple Hospital Management System using GUI in Java

18
5

Hospital Management System

Patient ID:

Patient Name:

Patient Address:

Patient Contact:

Add Patient

Update Patient

Message

Patient ID: Josbert250
Patient Name: null
Patient Address: null
Patient Contact: null

Patient ID: Josbert250
Patient Name: NTEZIRIZA NKERABAHIZI Josbert
Patient Address: MU SANZE-INE S
Patient Contact: 0794322298

Patient ID: Josbert250
Patient Name: NTEZIRIZA NKERABAHIZI Josbert
Patient Address: MU SANZE-INE S
Patient Contact: 0794322298

Patient ID: RUGWIRO345
Patient Name: RUGWIRO Hubert
Patient Address: NYANZA-CITY
Patient Contact: 0798343300

Patient ID: Claire234
Patient Name: UWIMANA Claire
Patient Address: KIGALI-MUHIMA
Patient Contact: 0799547342

Patient ID: Louis233
Patient Name: HABAGUSENGA Louis
Patient Address: KIGALI-KICUKIRO
Patient Contact: 0788098764

OK

View Patients

String patientsString = "";

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

15. A simple Hospital Management System using GUI in Java

18
6

Hospital Management System

Patient ID: Jeanne244

Patient Name: UMURISA Jeanne

Patient Address:

Patient Contact: 0788987043

Message

i Patient added successfully!

OK

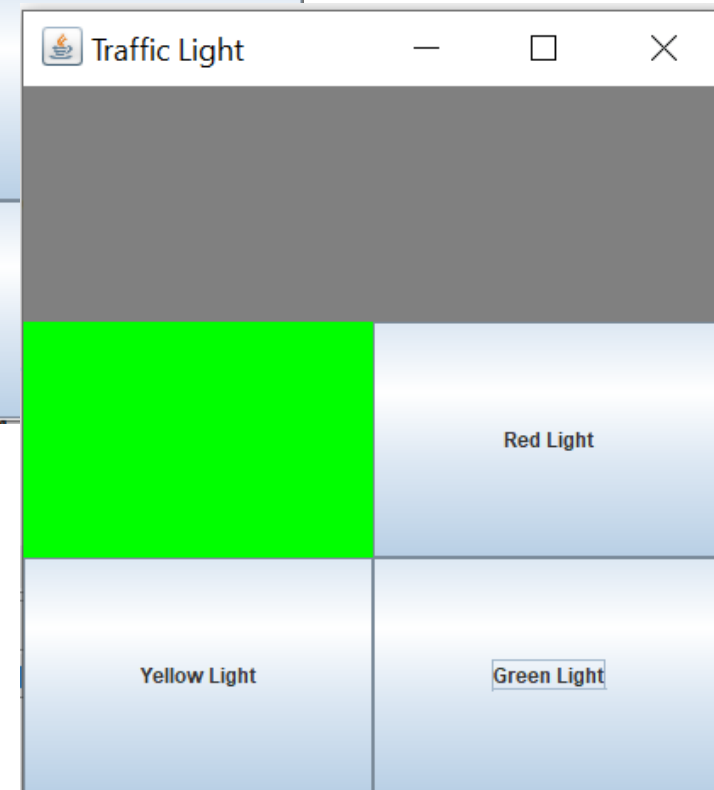
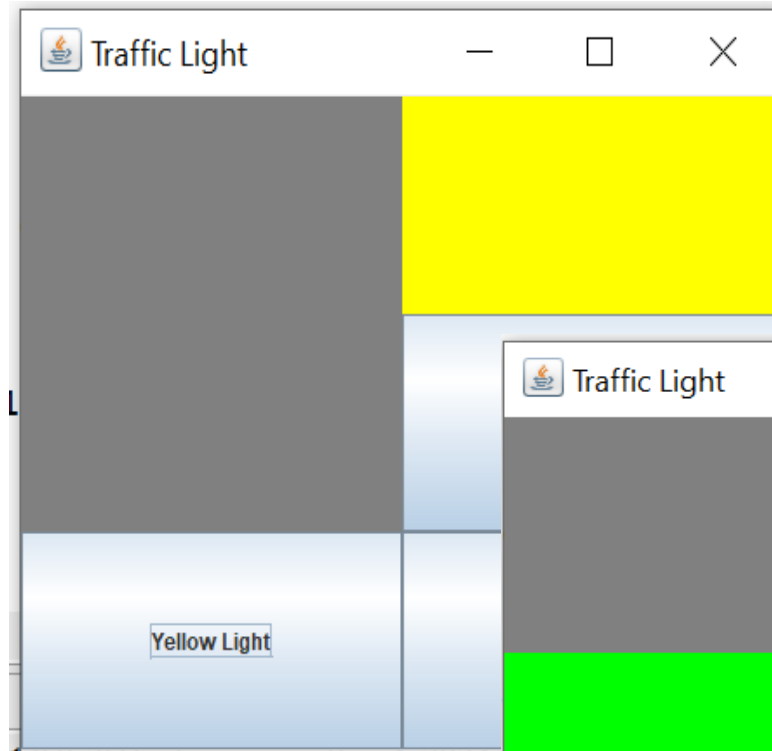
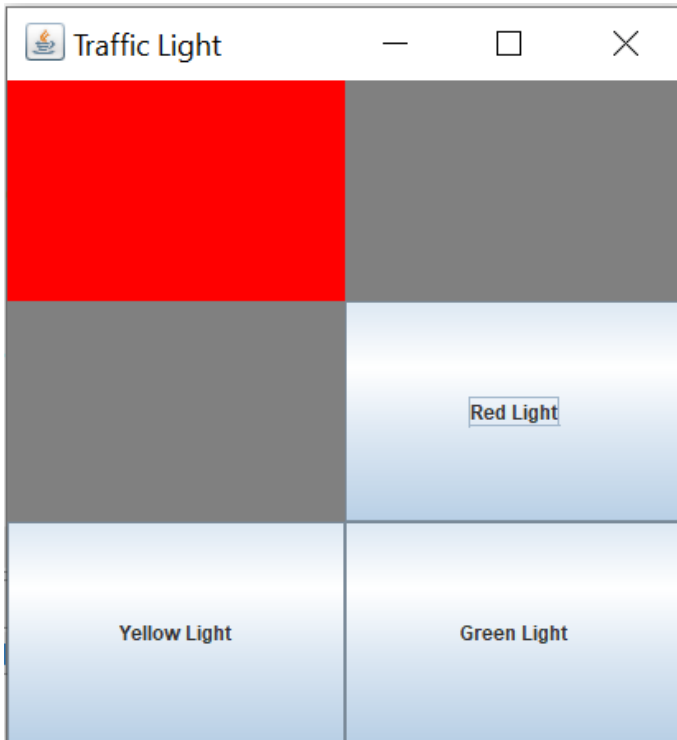
Add Patient **View Patients** **Update Patient**

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

16. A traffic light application in Java GUI

18
7



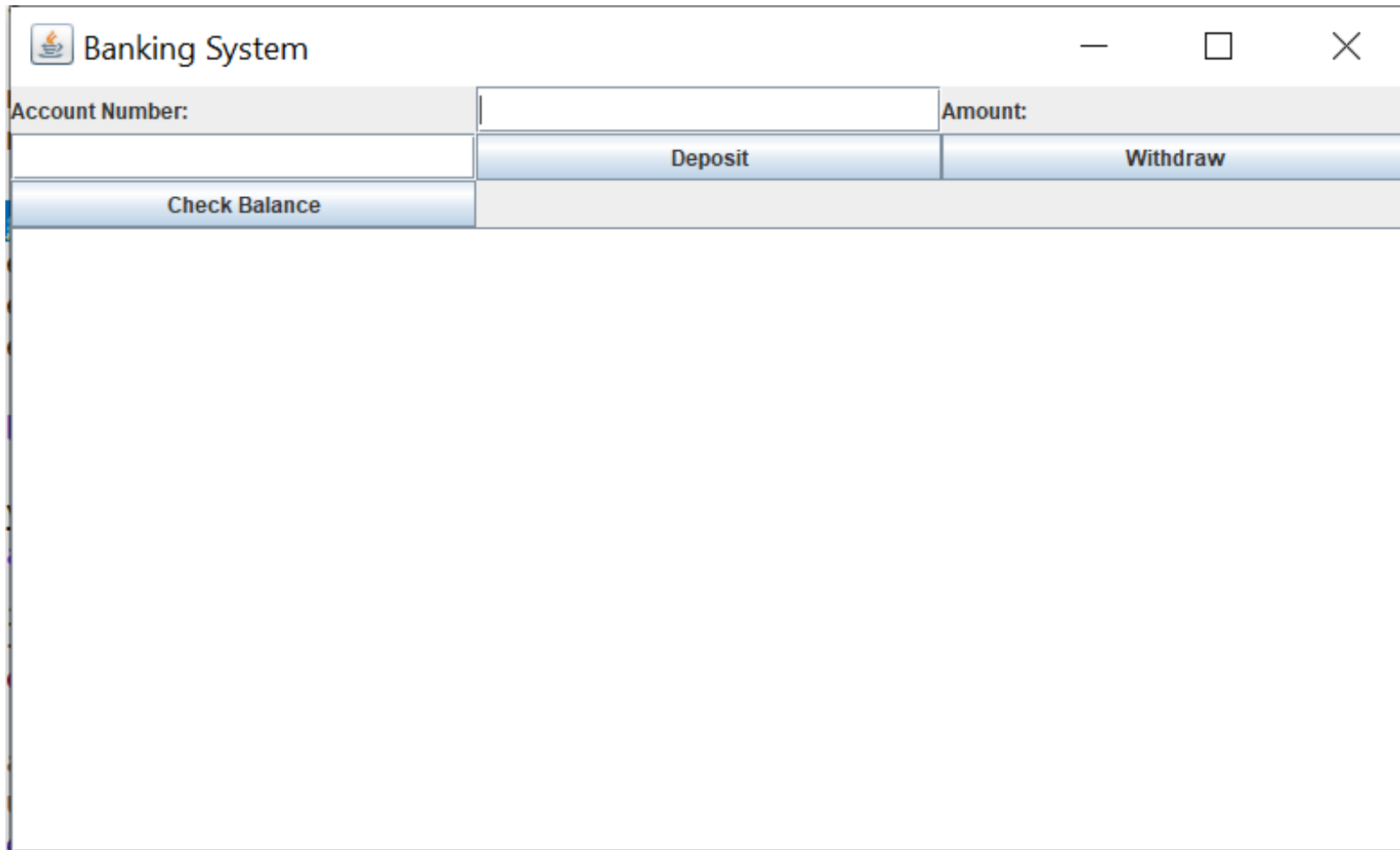
UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

17. Another banking system

Class name: BankingSystemGUI

18
8



The image shows a Java Swing window titled "Banking System" with a standard Mac OS-style title bar (red, yellow, and green buttons). The window contains a form with the following elements:

- Account Number:** A text label followed by an empty text input field.
- Amount:** A text label followed by an empty text input field.
- Buttons:** Three buttons are arranged in a grid:
 - Deposit:** A blue button with white text, located below the "Amount" field.
 - Withdraw:** A blue button with white text, located to the right of the "Deposit" button.
 - Check Balance:** A blue button with white text, located below the "Account Number" field.
- Content Area:** A large, empty rectangular area at the bottom of the window, likely for displaying transaction history or account details.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

17. Another banking system

Class name: BankingSystemGUI

The image shows a Java Swing window titled "Banking System" with a standard title bar (minimize, maximize, close buttons). The window contains a form with the following elements:

- Account Number:** A text field containing "188200JNN".
- Amount:** A text field, currently empty.
- Buttons:** Three buttons are visible: "Deposit", "Withdraw", and "Check Balance".
- Transaction History:** A text area displaying the following text:
Deposited \$5000.0. New balance: \$5000.0
Current balance: \$5000.0
Deposited \$10000.0. New balance: \$15000.0
Deposited \$9000.0. New balance: \$24000.0
Withdrawn \$7000.0. New balance: \$17000.0
Withdrawn \$900.0. New balance: \$16100.0
Insufficient funds
Withdrawn \$400.0. New balance: \$15700.0

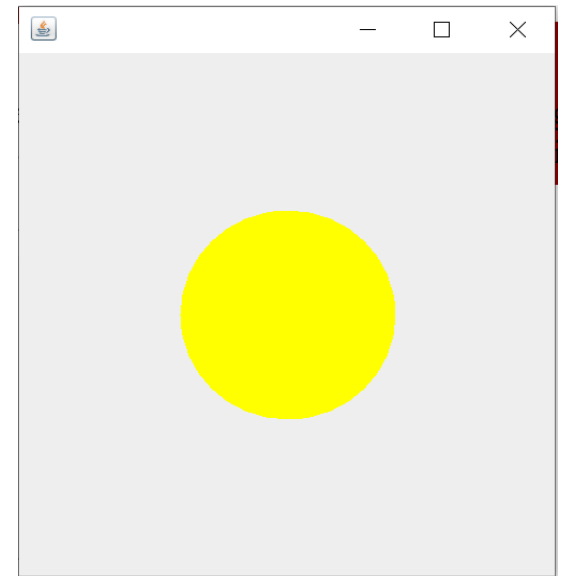
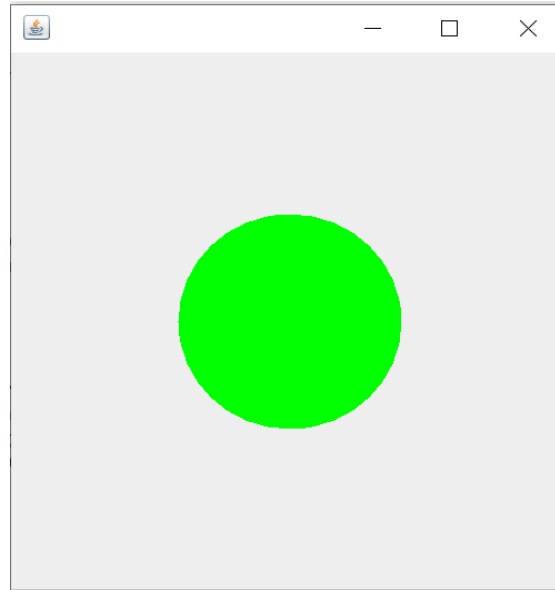
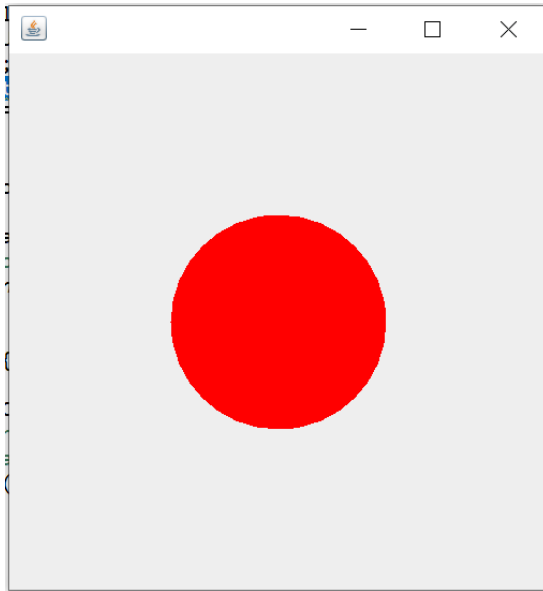
At the bottom of the window, there is a footer area with the text "Fall-2026-Dr. Josbert" on the left and "UoK" on the right.

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

19. Creates a simple traffic light GUI with red, yellow, and green colors. The colors change automatically every 5 seconds using a **Timer**

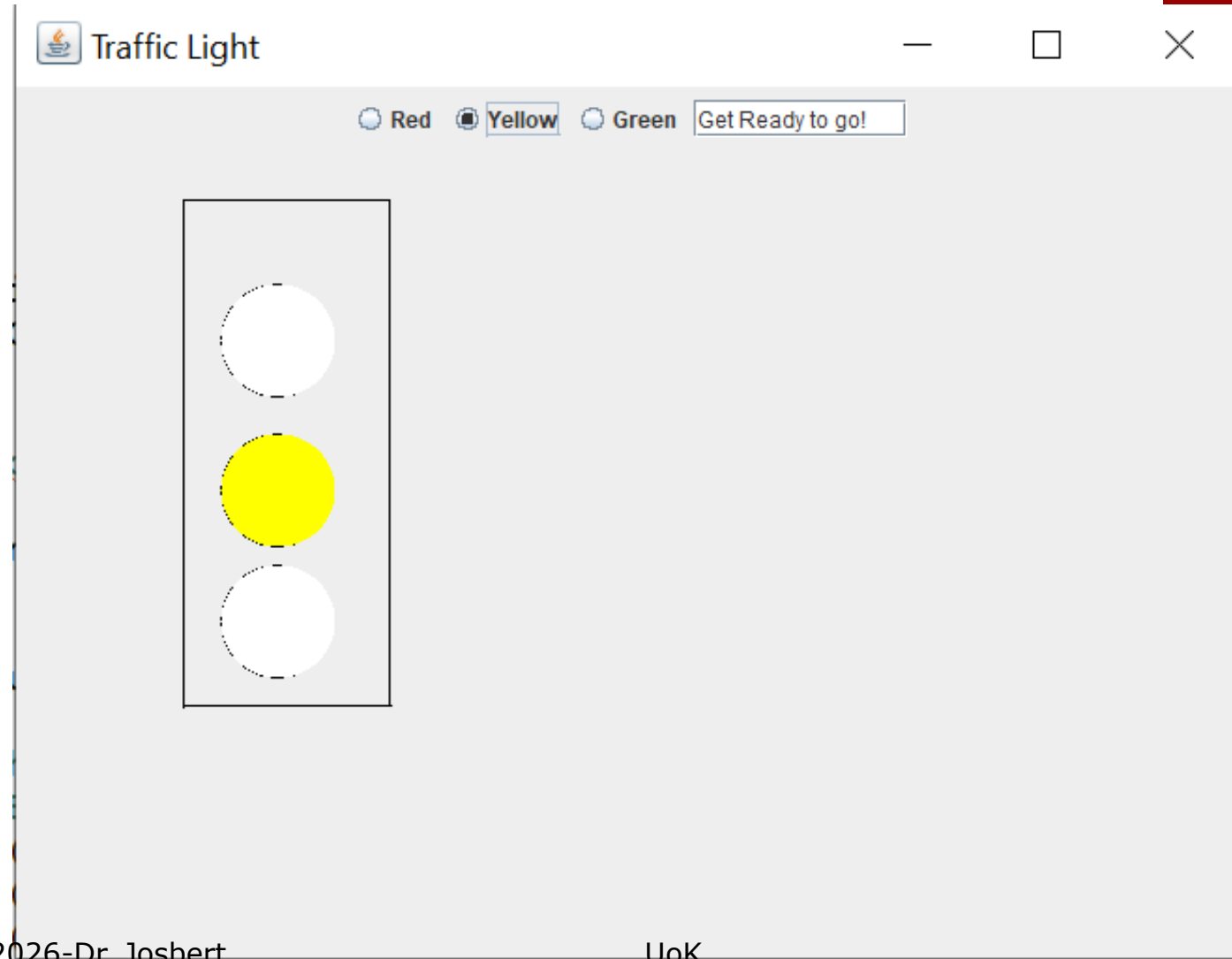
19
0



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

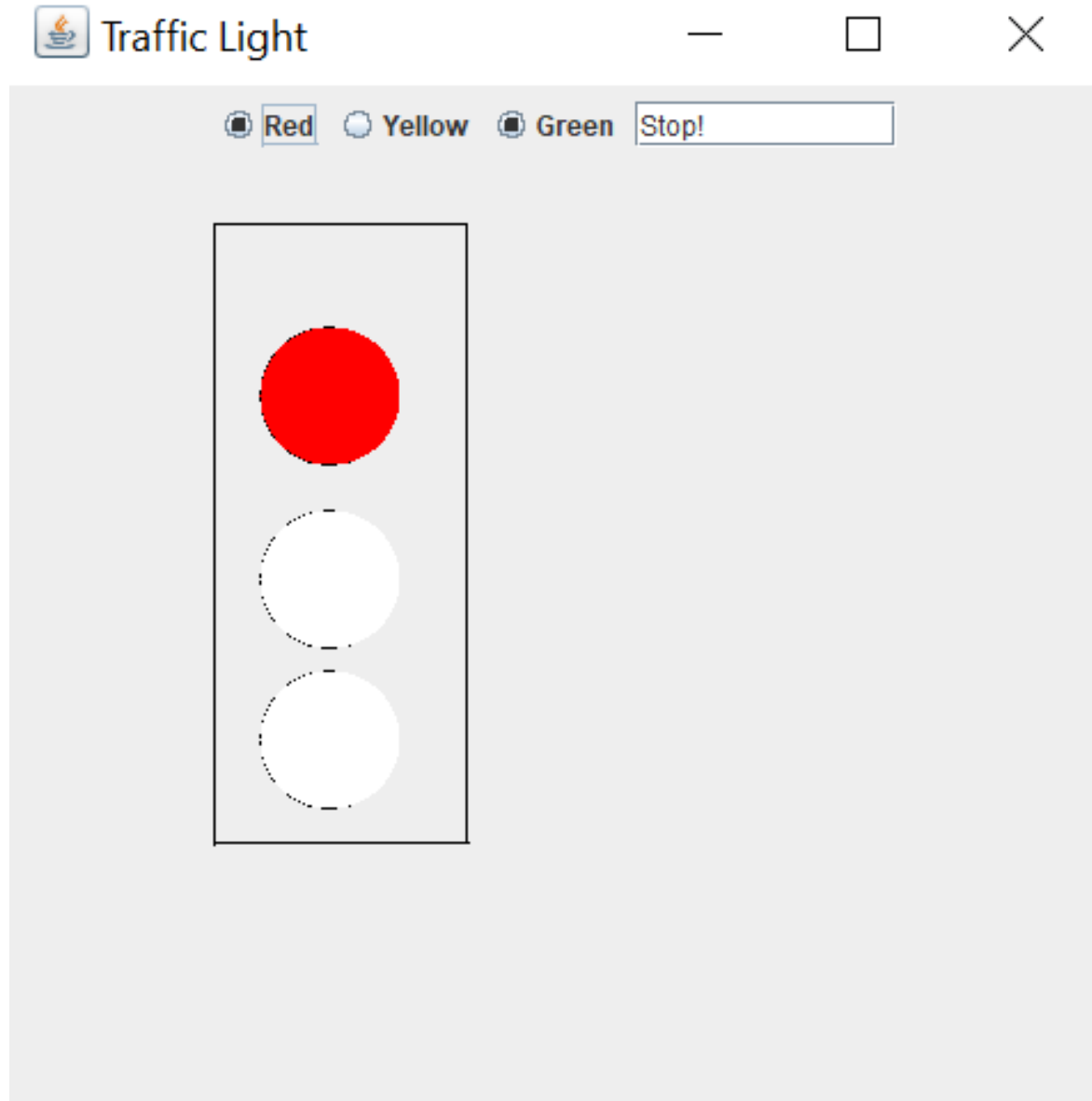
20. Another example of traffic light using Java GUI



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

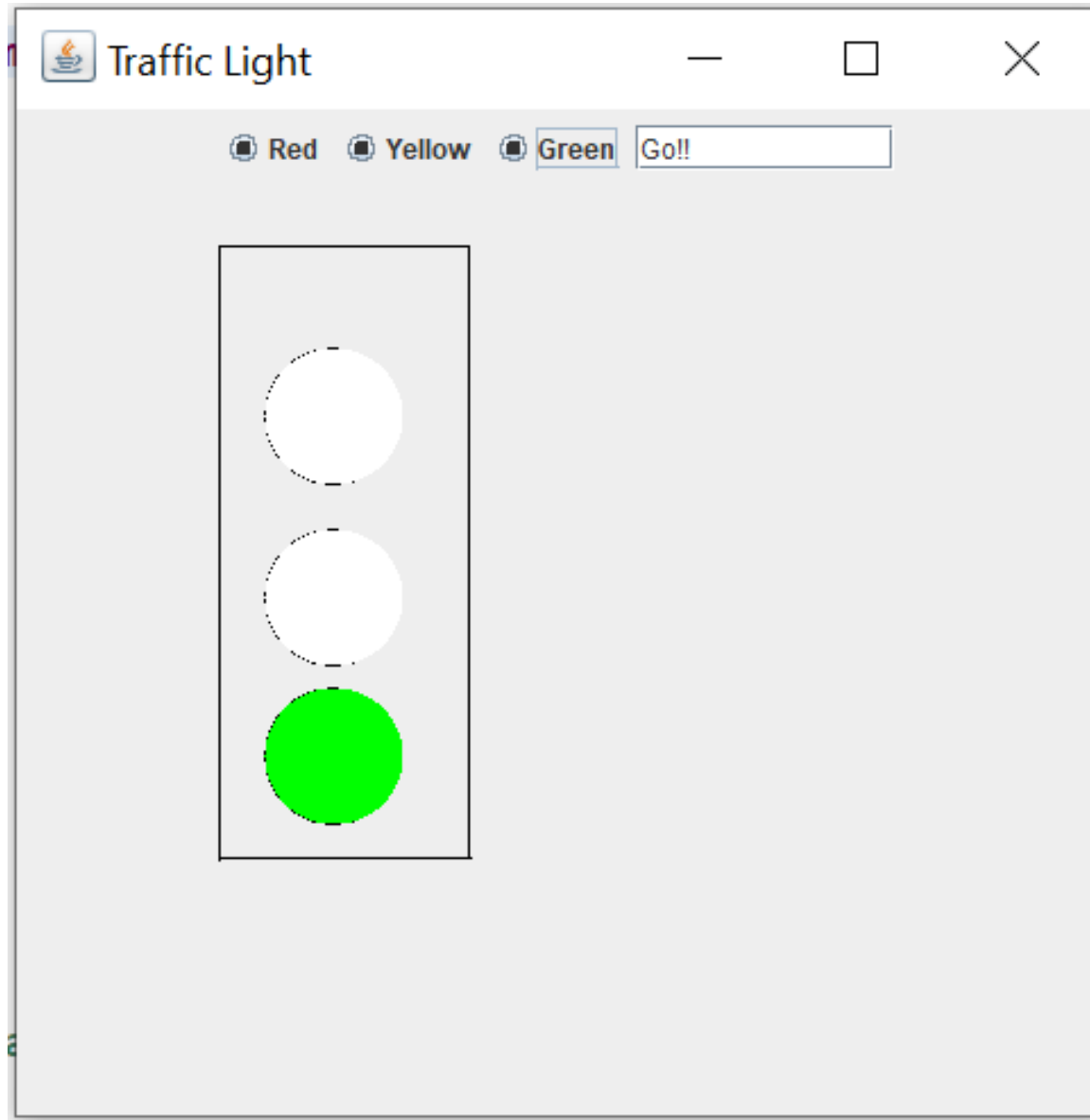
Exercises:

19
2



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

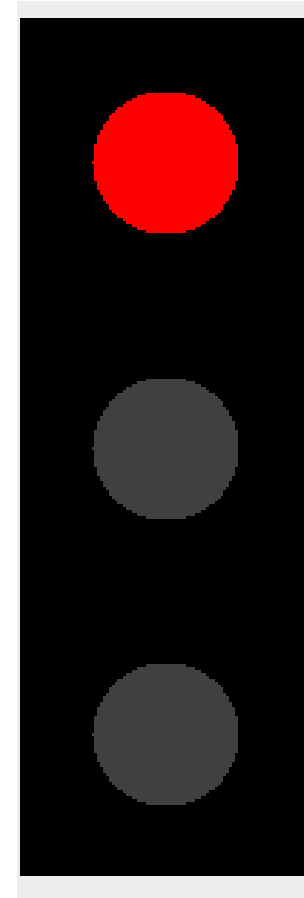
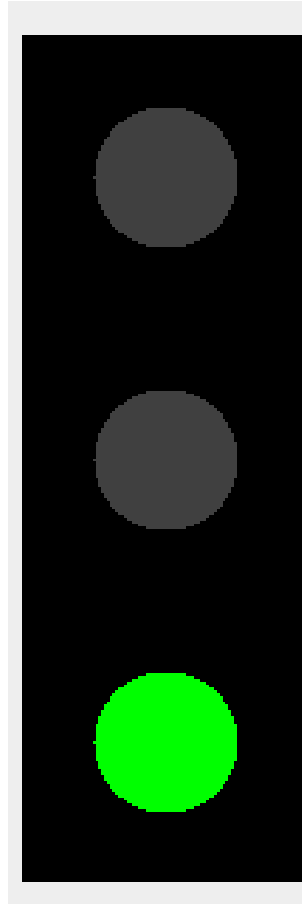
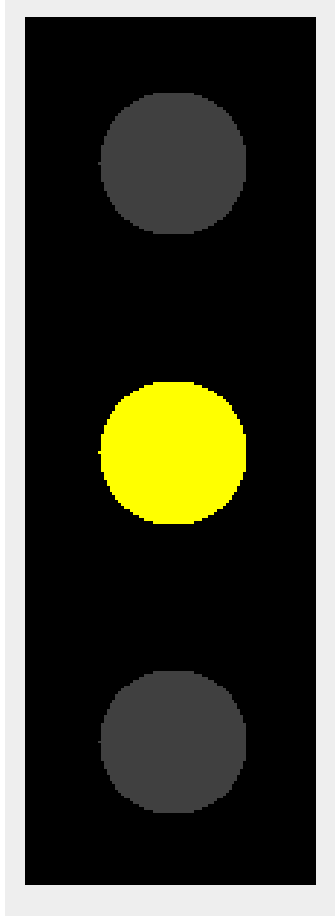


19
3

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

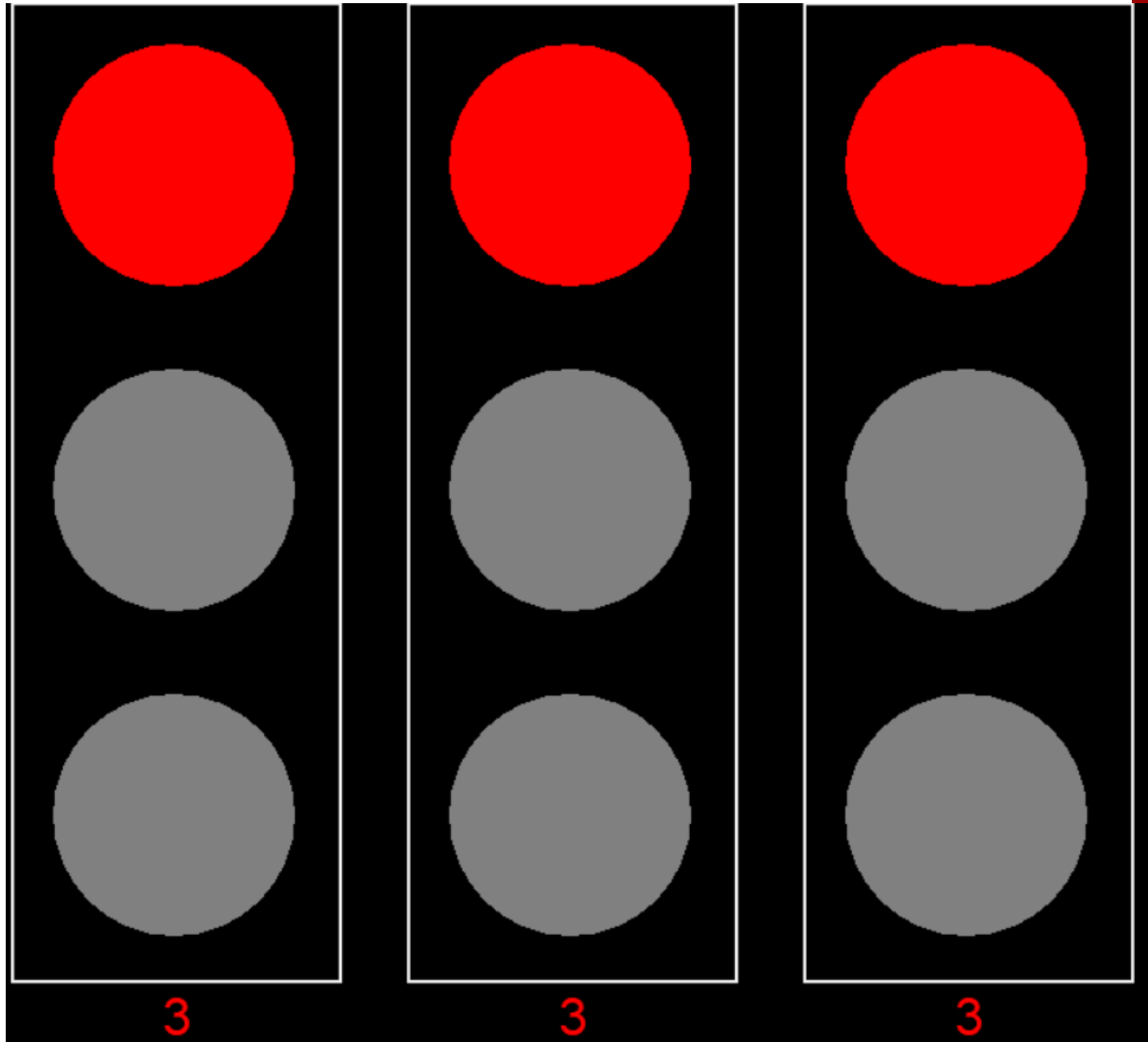
Exercises:

21. Another example of automatic traffic light using Java GUI



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

19
5

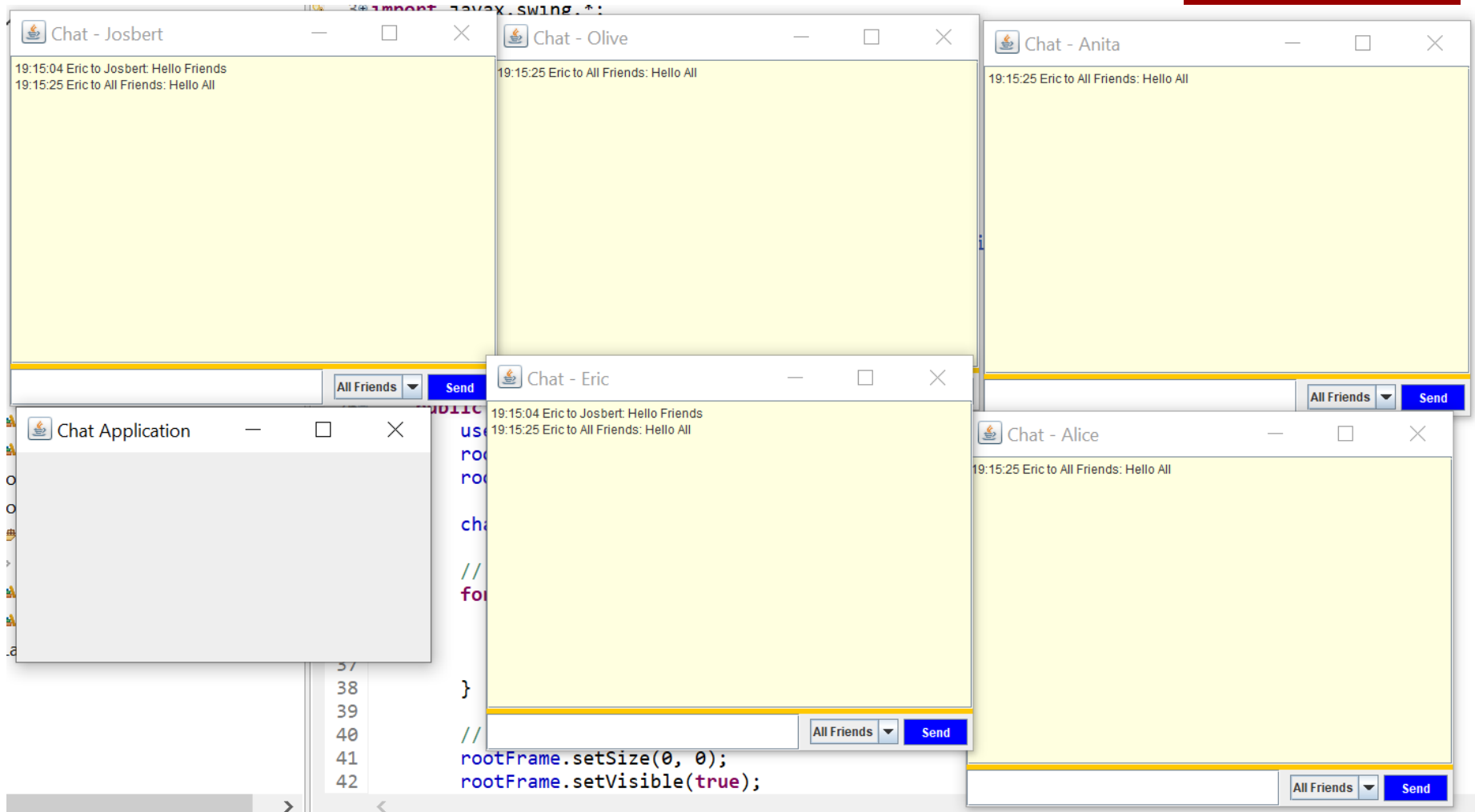


UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

22. Chat application using Java GUI

19
6



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

23. RITCO Company Booking Ticket

19
7

 RITCO COMPANY TICKET BOOKING

RITCO COMPANY BOOKING TICKET

BOOKED TICKETS

Ticket Code	Name	Phone	Price	Status	Route	Date	Time
TKT-6170	Josbert	0794322298	3000	Paid	Muhanga - Kigali	2025-04-29	20:17
TKT-3478	Isaline	0788547461	5000	Paid	Rubavu - Kigali	2025-05-02	22:19

Available Buses
5

Available Tickets
18

Booked Tickets
2

Remaining Seats: 18

Name:

Phone Number:

Price:

Status:

Paid

Route:

Kigali - Muhanga

Date:

2025-04-29

Time:

20:20

Ticket Code:

TKT-1448

[Book Ticket](#)

Fall-2026-Dr. Josbert

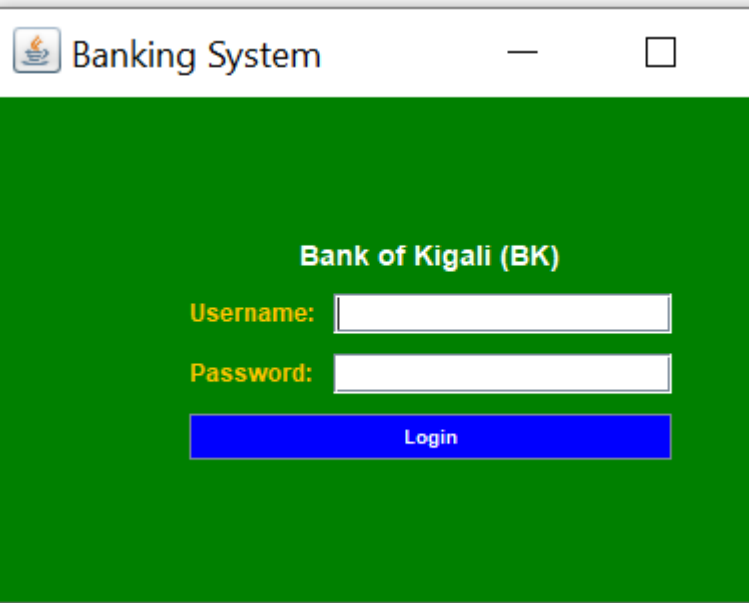
UoK

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

23. BK System

19
9



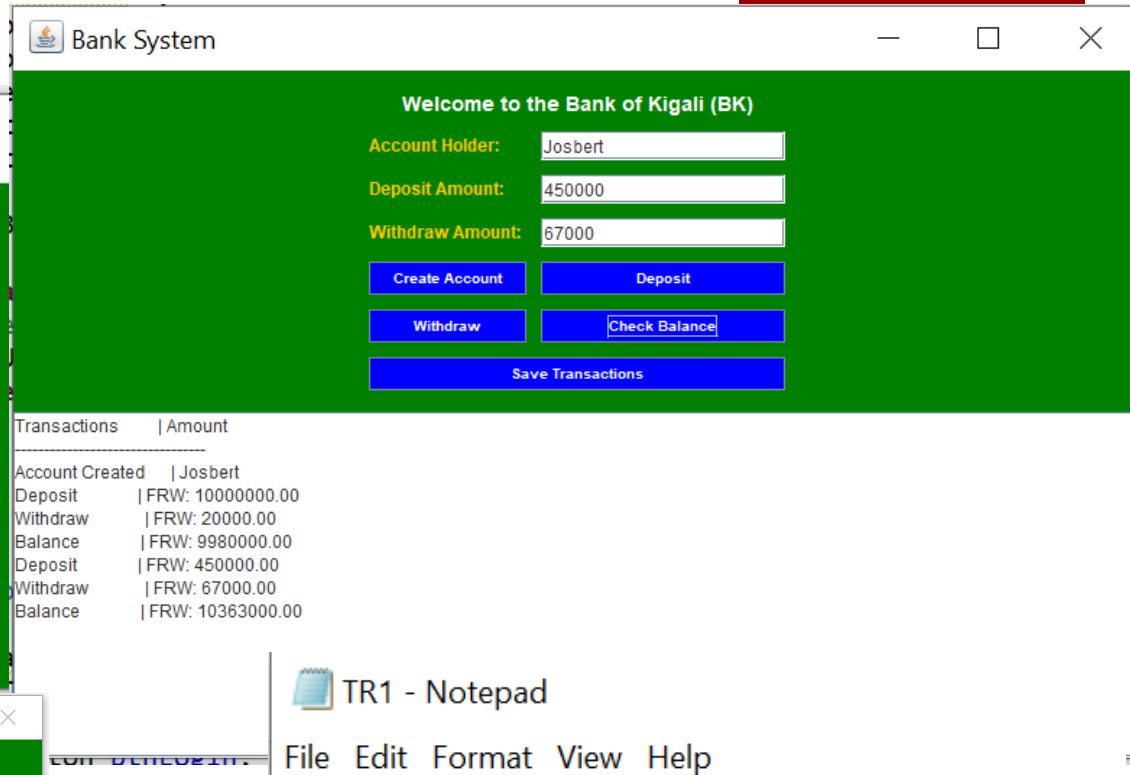
Banking System

Bank of Kigali (BK)

Username:

Password:

Login



Bank System

Welcome to the Bank of Kigali (BK)

Account Holder:

Deposit Amount:

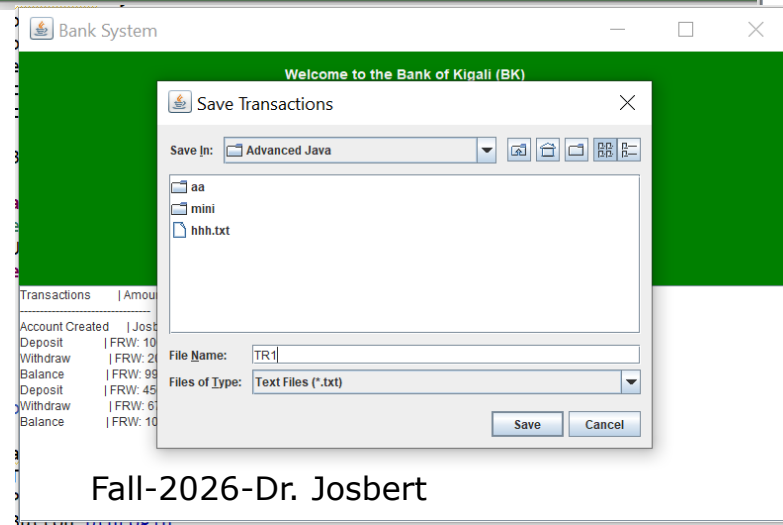
Withdraw Amount:

Create Account Deposit

Withdraw Check Balance

Save Transactions

Transactions	Amount
Account Created	Josbert
Deposit	FRW: 10000000.00
Withdraw	FRW: 20000.00
Balance	FRW: 9980000.00
Deposit	FRW: 450000.00
Withdraw	FRW: 67000.00
Balance	FRW: 10363000.00



Bank System

Welcome to the Bank of Kigali (BK)

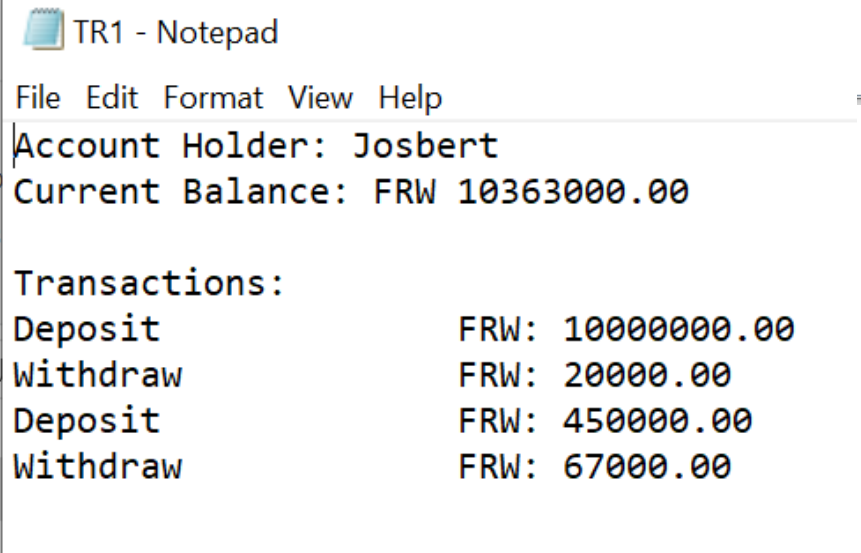
Save Transactions

Save In:

File Name:

Files of Type:

Save Cancel



TR1 - Notepad

File Edit Format View Help

Account Holder: Josbert

Current Balance: FRW 10363000.00

Transactions:

Deposit	FRW: 10000000.00
Withdraw	FRW: 20000.00
Deposit	FRW: 450000.00
Withdraw	FRW: 67000.00

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

24. Attendance Software

20
0

Attendance Register created by Dr. Josbert

ATTENDANCE SOFTWARE

University of Kigali
Department of Computer Science, SWE
OOP-Advanced Java Programming

No.	Student No.	First Name	Surname	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10	Day 11	Day 12	Day 13	Day 14
1	2111	Olivier	KARENZI	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	2222	Jeanne	UWIRINGIY...	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	2333	Rachel	MUKESHIM...	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	2444	Jehovanis	KIGELI	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Student No.: 2111

Day: 13

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

24. Attendance Software

20
1

Attendance Register created by Dr. Josbert

ATTENDANCE SOFTWARE

University of Kigali
Department of Computer Science, SWE
OOP-Advanced Java Programming

No.	Student No.	First Name	Surname	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10	Day 11	Day 12	Day 13	Day 14
1	2111	Olivier	KARENZI	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	2222	Jeanne	UWIRINGIY...	0	0	0	0	0	0	0	0	0	0	0	0	0	0
3	2333	Rachel	MUKESHIM...	0	0	0	0	0	0	0	0	0	0	0	0	0	0
4	2444	Jehovanis	KIGELI	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Exam Eligibility

Students allowed to sit for the exam:
Student No: 2111 (Olivier KARENZI) • 80.00%

Students NOT allowed to sit for the exam:
Student No: 2222 (Jeanne UWIRINGIYIMANA) • 0.00%
Student No: 2333 (Rachel MUKESHIMANA) • 0.00%
Student No: 2444 (Jehovanis KIGELI) • 0.00%

OK

ATTENDANCE REPORT

University of Kigali
Department of Computer Science, SWE
OOP-Advanced Java Programming
Lecturer: Dr. NTEZIRIZA NKERABAHIZI Josbert
Date: 2026-04-06

No.	Student No.	First Name	Surname	Day 1	Day 2	Day 3	Day 4	Day 5
Day 6	Day 7	Day 8	Day 9	Day 10	Day 11	Day 12	Day 13	Day 14
Day 15	Attendance %							
1	2111	Olivier	KARENZI	1	0	1	1	1
1	1	1	1	1	1	1	1	0
0	80.00%							
2	2222	Jeanne	UWIRINGIYIMANA	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0.00%							
3	2333	Rachel	MUKESHIMANA	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0.00%							
4	2444	Jehovanis	KIGELI	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0.00%							

Student No.:

2111

Day:

13

Mark Attendance

Add Student

Delete Student

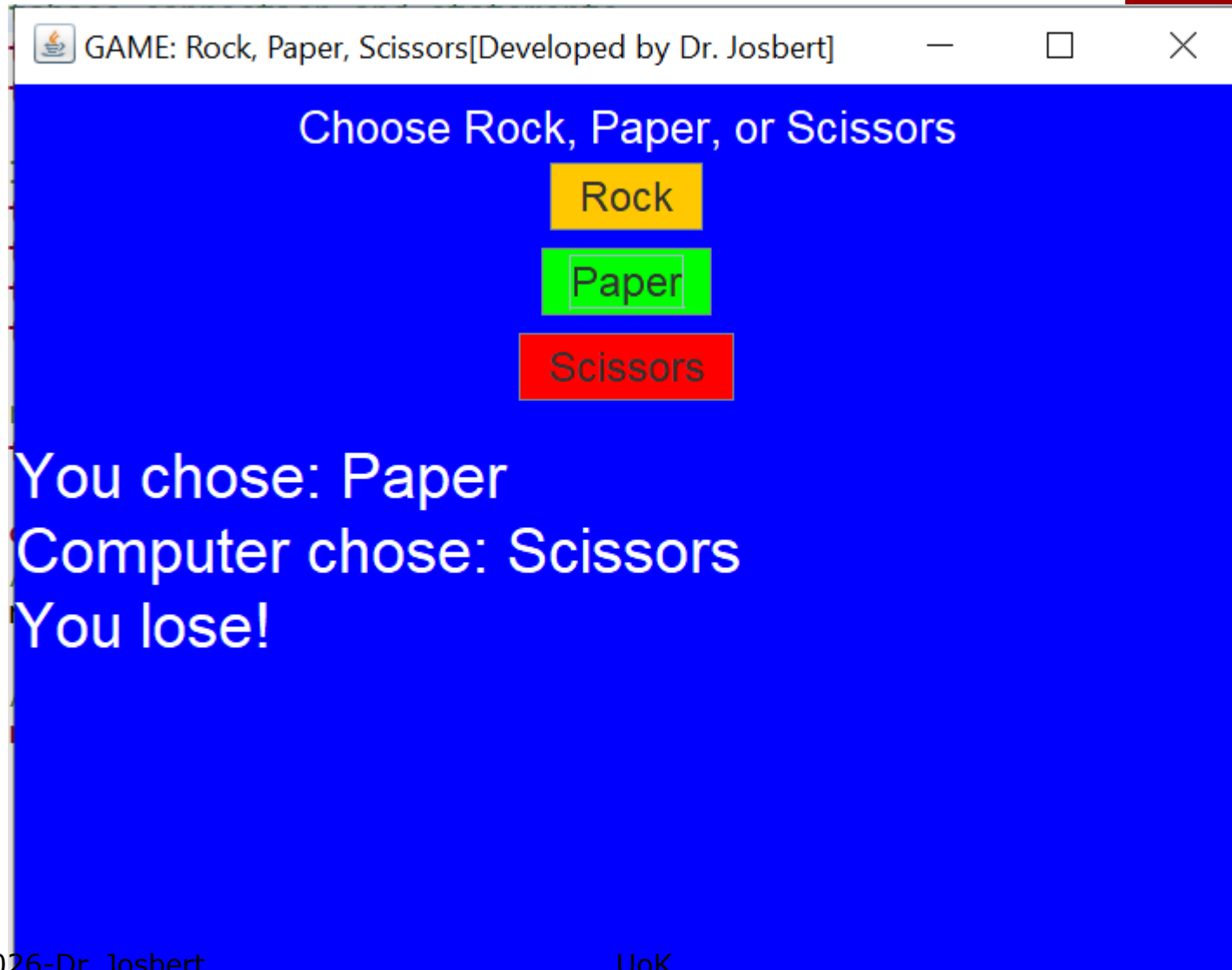
Calculate Attendance

Print to Notepad

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

24. Game: Rock, paper, Scissors



20
2

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

25. Person Management Data

20
3

Person Management

First Name:

Middle Name:

Last Name:

Email:

Phone:

ID	First Name	Middle Name	Last Name	Email	Phone
1	Josbert	NTEZIRIZA	NKERABAHIZI	josbert@gmail.com	0794322298
2	Isaline	UMUKUNZI	UWASE	isaline@gmail.com	09434343434
3	Solange	UMWALI	MWIZA	solange@gmail.com	0743434343
4	Musabe	John	Eric	Eric@gmail.com	4343434343434
5	Hubert	Rugwiro	musabe	hubert@gmail.com	434343434343

SELECT * FROM `person`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

←T→

personId

firstName

middleName

lastName

email

phone

☐	Edit	Copy	Delete	1	Josbert	NTEZIRIZA	NKERABAHIZI	josbert@gmail.com	0794322298
☐	Edit	Copy	Delete	2	Isaline	UMUKUNZI	UWASE	isaline@gmail.com	09434343434
☐	Edit	Copy	Delete	3	Solange	UMWALI	MWIZA	solange@gmail.com	0743434343
☐	Edit	Copy	Delete	4	Musabe	John	Eric	Eric@gmail.com	4343434343434
☐	Edit	Copy	Delete	5	Hubert	Rugwiro	musabe	hubert@gmail.com	434343434343

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

New

Insert

Update

Delete

First

Previous

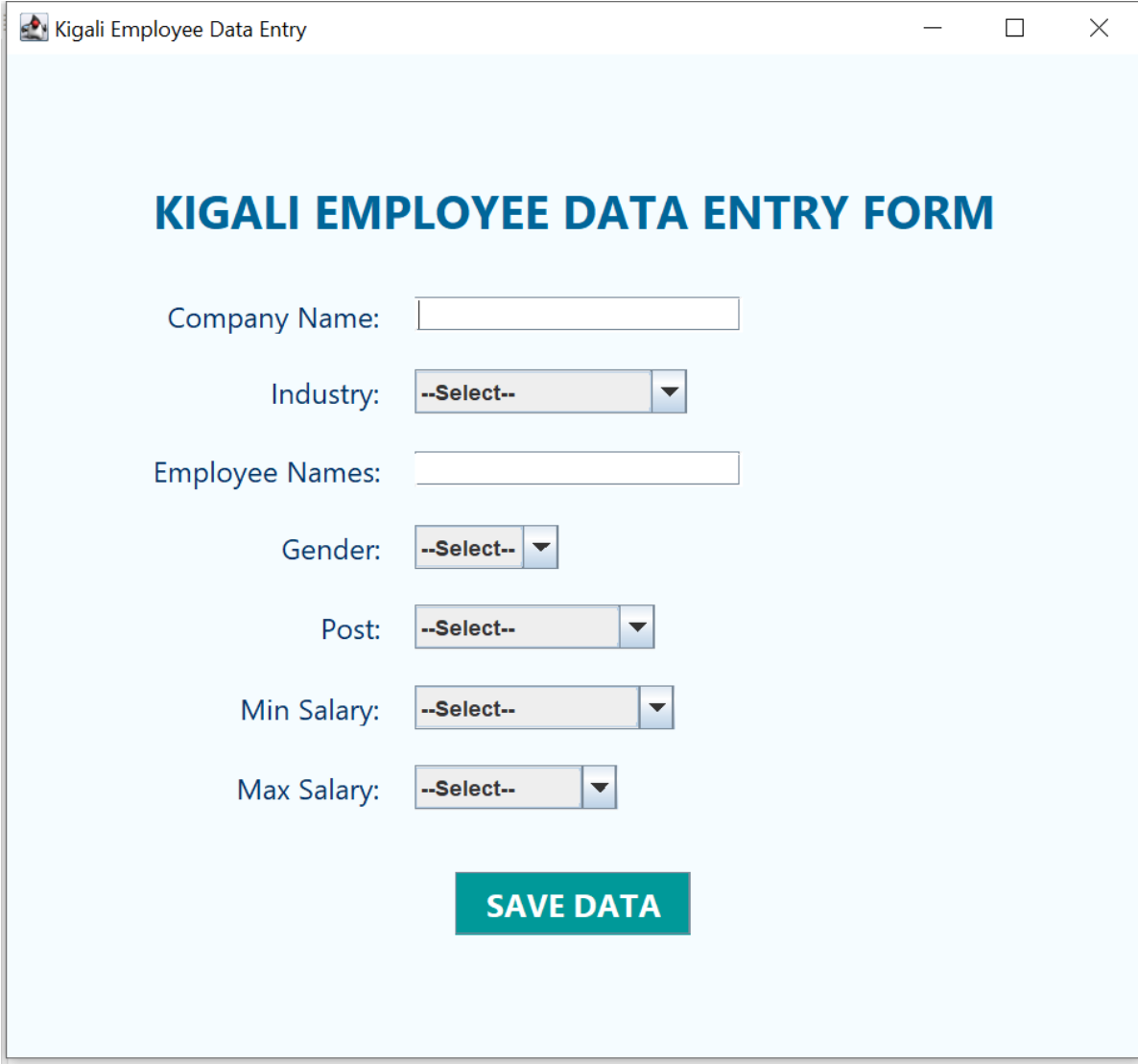
Next

Last

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

26. Kigali Employee Data Entry Form



The screenshot shows a window titled "Kigali Employee Data Entry" with a light blue background. The title bar includes a standard Windows icon, the text "Kigali Employee Data Entry", and minimize, maximize, and close buttons. The main content area features the title "KIGALI EMPLOYEE DATA ENTRY FORM" in bold blue text. Below the title are several input fields: "Company Name" (a text box), "Industry" (a dropdown menu with "--Select--"), "Employee Names" (a text box), "Gender" (a dropdown menu with "--Select--"), "Post" (a dropdown menu with "--Select--"), "Min Salary" (a dropdown menu with "--Select--"), and "Max Salary" (a dropdown menu with "--Select--"). At the bottom center is a teal button labeled "SAVE DATA".

Kigali Employee Data Entry

KIGALI EMPLOYEE DATA ENTRY FORM

Company Name:

Industry:

Employee Names:

Gender:

Post:

Min Salary:

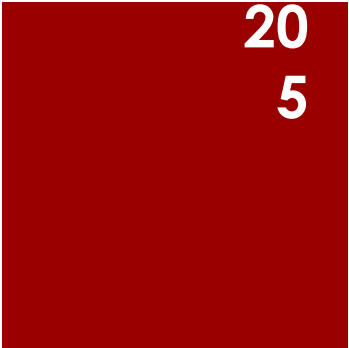
Max Salary:

SAVE DATA

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

26. Kigali Employee Data Entry Form



Kigali Employee Data Entry

KIGALI EMPLOYEE DATA ENTRY FORM

Company Name:

Industry:

--Select--

Employee Names:

Gender:

--Select--

SELECT * FROM `kigaliemployeesdata`

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				id	company_name	industry	employee_name	employee_gender	post	min_salary	max_salary	registration_date
☐	Edit	Copy	Delete	1	Tech Solutions	IT	John Doe	Male	Manager	less than 300000 rwf	above 500000	2025-04-25 00:38:33
☐	Edit	Copy	Delete	2	Green Farms	Farming	Jane Smith	Female	CEO	less than 500000 rwf	above 1000000	2025-04-25 00:38:33
☐	Edit	Copy	Delete	3	Jos_Company	Business	Big Jos	Male	Manager	less than 400000 rwf	above 1000000	2025-04-25 00:55:16
☐	Edit	Copy	Delete	4	Vivo	IT	JOSBERTR	Male	CEO	less than 100000 rwf	above 200000	2025-04-27 17:28:11

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

27. Employee Record System

Menu | Employee Record System

— □ ×

MENU

 Add Employee

 Increase Salary

 Salary Information

 View Employees Data

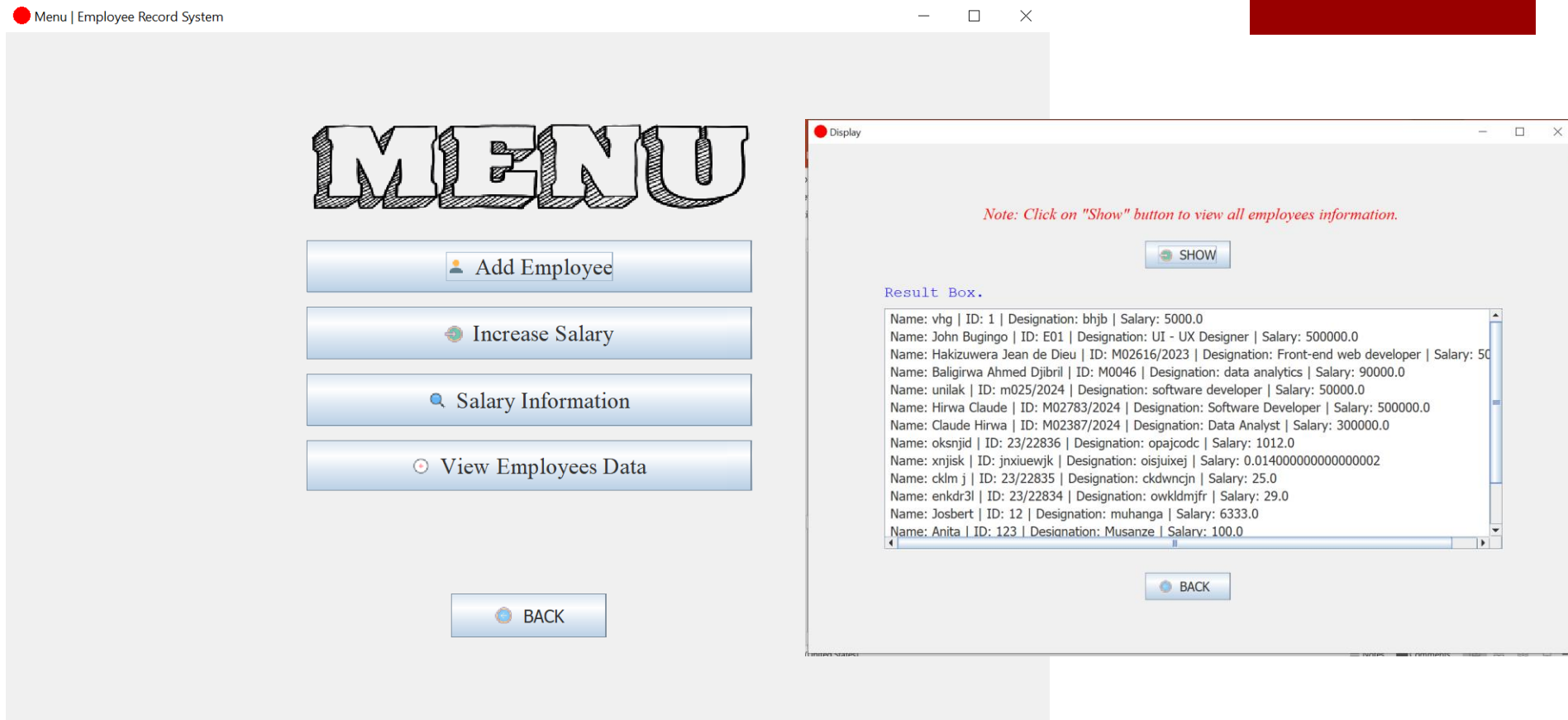
 BACK

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

27. Employee Record System

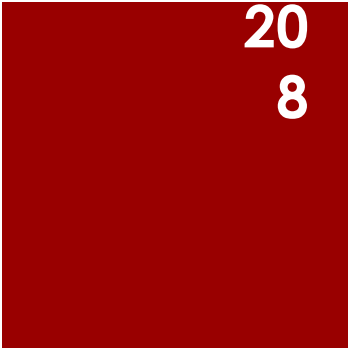
20
7



UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

28. Employee Record System



User Management System

Search User

Search (ID/Name):

Search

User Details

Full Name:

Email:

Phone:

Country:

Actions

Submit

Update

Delete

Reset

ID	FULL NAME	EMAIL	PHONE	COUNTRY
1	John Doe	john.doe@example.com	1234567890	United States
2	Jane Smith	jane.smith@example.com	0987654321	Canada
3	Robert Johnson	robert.j@example.com	5551234567	United Kingdom
7	Josbert NKERABAHIZI	Josbertnn@gmail.com	084343533	France

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

28. Employee Record System

20
9

User Management System

Search User

Search (ID/Name): Search

SELECT * FROM `users`

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

				id	fullname	email	phone	country	registration_date
☐	Edit	Copy	Delete	1	John Doe	john.doe@example.com	1234567890	United States	2025-04-25 00:41:17
☐	Edit	Copy	Delete	2	Jane Smith	jane.smith@example.com	0987654321	Canada	2025-04-25 00:41:17
☐	Edit	Copy	Delete	3	Robert Johnson	robert.j@example.com	5551234567	United Kingdom	2025-04-25 00:41:17
☐	Edit	Copy	Delete	7	Josbert NKERABAHIZI	Josbertnn@gmail.com	084343533	France	2025-04-25 01:12:29

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

29. Car Rent System

21
0

LOSRENT LOGIN PAGE

Login Form

Use valid Username and password to login

Username

admin

Password

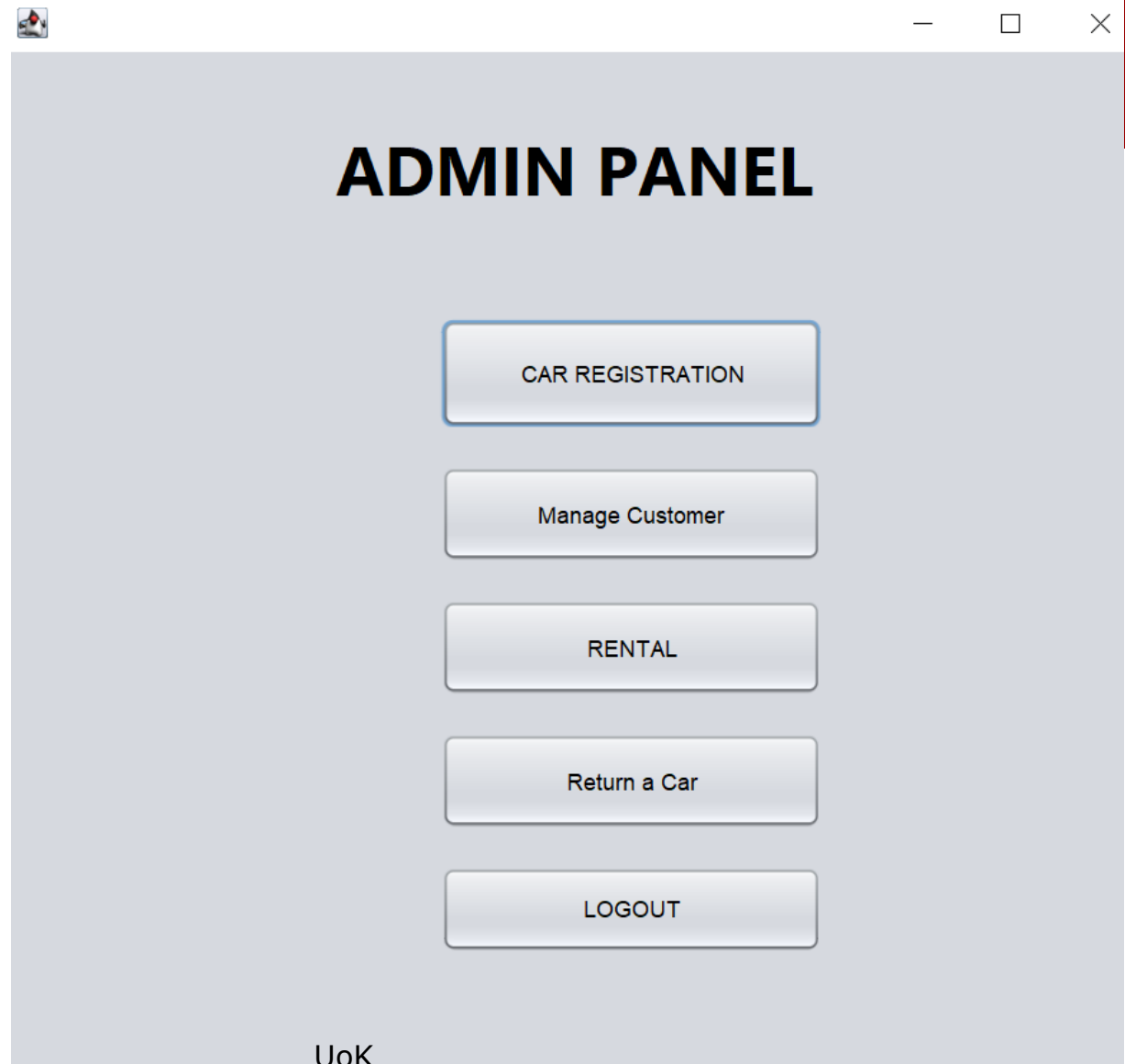
Login

Exit

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

29. Car Rent System



21
1

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

29. Car Rent System

21
2

Car_RegNo	Make	Model	Plate No	Available
CA001	Toyota	Corolla	RAA123A	No
CA002	Honda	Civic	RAB456B	Yes
CA003	Mazda	CX-5	RAC789C	Yes

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

29. Car Rent System

21
3

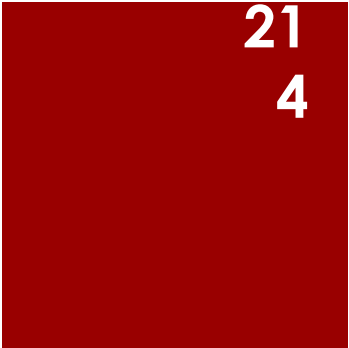
The screenshot displays a graphical user interface for a Car Rent System. On the left, there is a 'Customer Form' titled 'Customer Registration Form'. It contains five input fields: 'Customer ID' (with 'C0004' entered), 'Customer Names', 'Address', 'Phone No', and 'Password'. Below these fields are five buttons: 'Add', 'Delete', 'Update', 'Back', and 'Refresh'. On the right, there is a table with four columns: 'Customer ID', 'Names', 'Address', and 'Phone No'. The table contains three rows of data.

Customer ID	Names	Address	Phone No
C0001	John Doe	123 Main St, Kigali	0781234567
C0002	fdfdf	fdfdfdf	0788843434
C0003	Josbert	Rwanda, Musanze	0794322298

CHAPTER 17 GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

29. Car Rent System



Customer Form

Customer Registration Form

Customer ID

c0004

Customer Names

Address

Phone No

Password

Add

Delete

Update

Back

Refresh

Customer ID	Names	Address	Phone No
C0001	John Doe	123 Main St, Kigali	0781234567
C0002	felid	felid	0788843434
C0003	Josbert	Rwanda, Musanze	0794322298

Filters

Containing the word:

	Table	Action	Rows	Type	Collation	Size	Overhead
☐	car	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐	customer	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐	rent	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐	returncar	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 KiB	-
☐	user	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
	5 tables	Sum	10	InnoDB	utf8mb4_general_ci	160.0 KiB	0 B

☐ Check all

With selected:

UNIT 1. GRAPHICAL USER INTERFACE PROGRAMMING

Exercises:

30. Calculator

